

2026年9月北美芯片设计/验证题目汇总

根据同学回忆整理 · 92道真题含答案与解析

Amazon · Apple · IBM · Intel · NXP · Nvidia · Optiver · Scaleflux · Smith & Nephew

目录

Digital Design

Q1. Digital Design: Rising Edge Detector for a Single-Bit Signal
Q2. Digital Design: Why Async FIFO Pointers Must Be Gray-Coded
Q3. Digital Design: The ASIC Design Flow, Stage by Stage
Q4. Digital Design: Camera Sensor to FPGA Image Data Interface
Q5. Digital Design: Async Assert, Sync Deassert Reset
Q6. Digital Design: FSM Sequence Detector for 1,0,1,1,0
Q7. Digital Design: Fibonacci Sequence Generator with Enable
Q8. Digital Design: Implement Parameterizable Barrel Shifter for Shift Operations
Q9. Digital Design: Design APB Slave Peripheral Following AMBA APB Protocol
Q10. Digital Design: Build AXI-Lite Write-Only Control Block
Q11. Digital Design: Design Write-Back Set-Associative Cache with LRU or FIFO Replacement
Q12. Digital Design: Design Pipelined ALU with Hazard Detection and Forwarding
Q13. Digital Design: Design Vending Machine FSM with Coin Input and Change Output
Q14. Digital Design: Detect Rising, Falling, and Any Transition (Registered Outputs)
Q15. Digital Design: Mealy vs Moore FSM -- the Sync/Async Output Distinction
Q16. Digital Design: Synchronous vs Asynchronous Sequential Logic, and Why Async Matters for Speed
Q17. Digital Design: Timing Requirements of Combinational Logic Between Two Pipeline Flip-Flops
Q18. Digital Design: Debug the data_gain Module -- Bias, Absolute Value, Fixed-Point Gain, Saturation (Designer Question)
Q19. Digital Design: What Is an AXI Burst, and What Is a Narrow Transfer
Q20. Digital Design: Logic Circuit Analysis -- NAND/XNOR Combination (FPGA Engineer Question)
Q21. Digital Design: Logic Circuit Analysis -- Reverse-Engineer Inputs for Target Outputs (FPGA Engineer Question)
Q22. Digital Design: Boolean Simplification -- Minimum Gate Count (FPGA Engineer Question)
Q23. Digital Design: Match a Truth Table to a Boolean Equation (FPGA Engineer Question)
Q24. Digital Design: Hardware Logic Utilization -- Subtract vs Increment vs Multiply-by-8 (FPGA Engineer Question)
Q25. Digital Design: Hardware Logic Utilization -- Slice Compare vs Zero-Detect vs Magnitude Compare (FPGA Engineer Question)
Q26. Digital Design: Combinational BitReverser -- Latency, LUTs, and Flip-Flops (FPGA Engineer Question)
Q27. Digital Design: Matcher Module -- Reset Type and Flip-Flop Count (FPGA Engineer Question)
Q28. Digital Design: LogicModule -- Latency, LUTs, and Flip-Flops (with a Real Syntax Bug) (FPGA Engineer Question)
Q29. Digital Design: Identify the Correct DUT Waveform (Async Reset + Bit Indexing) (FPGA Engineer Question)
Q30. Digital Design: Debug a 2-Cycle-Latency Boolean ALU (FPGA Engineer Question)
Q31. Digital Design: Greenhouse Sensor Protocol Decoder (FPGA Engineer Question)

SystemVerilog -- Constraints & Randomization

Q32. SystemVerilog: No-Two-Adjacent-Equal Array Constraint (One Direction Is Enough)
Q33. SystemVerilog: Constrained Array -- Size 10-16, Strictly Descending
Q34. SystemVerilog: 10-bit Value with Exactly 5 Non-Adjacent 1-Bits
Q35. SystemVerilog: Generate Knight's Tour on Chessboard Using Constraints
Q36. SystemVerilog: Dynamic Array Size 300 with No Adjacent Zeros
Q37. SystemVerilog: Pick Ball from 10 Colors, No Repeat in Next 3 Picks
Q38. SystemVerilog: Generate Matrix and Its 90° Counterclockwise Rotation
Q39. SystemVerilog: Implement Instruction Generation Rules Without SystemVerilog Constraints
Q40. SystemVerilog: 100-bit Variable with Exactly One Run of 5 Consecutive 1s
Q41. SystemVerilog: Constrain Address to Range 2-10
Q42. SystemVerilog: Constrain Address to Multiple Ranges (2-10, 11-15, 19)

SystemVerilog -- Assertions (SVA)

Q43. SystemVerilog: Pulse-Width Assertion (Min 2, Max 6 Cycles) -- Three Iterations to Get It Right
Q44. SystemVerilog: Immediate vs Concurrent Assertions, with a Handshake Example
Q45. SystemVerilog: Start and End Signals Do Not Overlap
Q46. SystemVerilog: Sequence A Occurs Before Sequence B
Q47. SystemVerilog: Data Remains Stable When Enable Asserted
Q48. SystemVerilog: Write UVM RAL Assertion for Register Reads and Writes
Q49. SystemVerilog: Verify Mutual Exclusion in Shared Resource Access
Q50. SystemVerilog: Signal Always High in Clock Domain
Q51. SystemVerilog: Assertion Combining a Bounded-Window and an Exact-Offset Timing Requirement

SystemVerilog -- Coding

Q52. SystemVerilog: Asymmetric Fork-Join Priority (thread1 Must Finish, thread2 May Be Killed)
Q53. SystemVerilog: $cast Usage -- Static vs Dynamic Casting

Verification -- Test Plan

Q54. Verification: Code Coverage vs Functional Coverage, and Writing a Covergroup

UVM

Q55. UVM: Design Out-of-Order Transaction Matching Scoreboard Using Transaction IDs
Q56. UVM: Design Monitor that Reassembles 64-bit Transactions from Two 32-bit Beats
Q57. UVM: Two-Monitor Check-Cycle Scoreboard (Duplicate & Timeout Detection)
Q58. UVM: N-by-M Packet Switch Scoreboard (Per-Flow Ordering)

Algorithms & Scripting

Q59. Algorithms: Triangle Bug Meeting Probability (Brainteaser)
Q60. Algorithms: Minimum Weighings to Find the Heavy Ball Among 8 (Brainteaser)
Q61. Algorithms: Python -- Find and Print Lines Containing "ERROR" in a File
Q62. Algorithms & Scripting: Merge Two Sorted Arrays
Q63. Algorithms & Scripting: Maximum Product Subarray
Q64. Algorithms & Scripting: Container With Most Water
Q65. Algorithms & Scripting: Move Zeroes
Q66. Algorithms & Scripting: Maximum Subarray
Q67. Algorithms & Scripting: Longest Increasing Subsequence
Q68. Algorithms & Scripting: Coin Change
Q69. Algorithms & Scripting: First and Last Position of Element
Q70. Algorithms & Scripting: Median of Two Sorted Arrays
Q71. Algorithms & Scripting: Longest Palindromic Substring
Q72. Algorithms & Scripting: Find All Occurrences of Substring
Q73. Algorithms & Scripting: Valid Palindrome
Q74. Algorithms & Scripting: First Repeating Character
Q75. Algorithms & Scripting: String Compression
Q76. Algorithms & Scripting: Date Comparison (LessThan) in C (FPGA Engineer Question)

Computer Architecture

Q77. Computer Architecture: FPGA Feasibility Analysis for LLM Layers (Bandwidth / DSP / Dataflow Scheduling)
Q78. Computer Architecture: Implement Signed Addition Overflow Detection Logic
Q79. Computer Architecture: Implement Pseudo-LRU Using Binary Tree Structure
Q80. Computer Architecture: Implement Miss Status Holding Register (MSHR) for Non-Blocking Cache
Q81. Computer Architecture: Design Functional Unit Busy/Free Tracking Module
Q82. Computer Architecture: Implement Pipeline Registers with Stall and Flush Controls
Q83. Computer Architecture: Implement 2-Read 1-Write Register File

Q84. Computer Architecture: MESI and MOESI Cache Coherence Protocols

Q85. Computer Architecture: Set-Associative Cache vs Direct-Mapped vs Fully-Associative

Q86. Computer Architecture: TAGE vs gshare Branch Predictors

Behavioral Questions

Q87. Behavioral: Ensuring Something You Created Worked Properly

Q88. Behavioral: Responding to a Major Setback After Significant Effort

Q89. Behavioral: Using an AI Tool to Help Create Something

Q90. Behavioral: Something You Created Did Not Work as Expected

Q91. Behavioral: Creating Something With Other People (Team Role)

Q92. Behavioral: Explaining Something You Created to an Unfamiliar Audience

Digital Design

Q1. Digital Design: Rising Edge Detector for a Single-Bit Signal

QUESTION

Write SystemVerilog code that detects a rising edge on a single-bit signal (i.e. produces a one-cycle pulse the cycle the signal transitions from 0 to 1), and explain the approach.

ANSWER

```
module rising_edge_detector (
  input  logic clk,
  input  logic rst_n,
  input  logic sig_in,
  output logic edge_detected
);
  logic sig_d;   // sig_in, delayed by one clk cycle

  always_ff @(posedge clk or negedge rst_n)
    if (!rst_n) sig_d <= 1'b0;
    else        sig_d <= sig_in;

  // edge_detected is high exactly when sig_in is HIGH now but was LOW
  // last cycle -- i.e. "just rose"
  assign edge_detected = sig_in & ~sig_d;
endmodule

// Verified by simulation: driving sig_in high for 3 cycles then low,
// then high again for 1 cycle -- edge_detected pulses for exactly 1
// cycle at each 0->1 transition, and stays 0 the rest of the time
// (including while sig_in remains steadily high), confirming this is a
// true EDGE detector, not a level detector.
```

解释

考点：单比特上升沿检测是数字设计里最基础的构建模块之一，"打一拍再做与运算"是标准做法，几乎所有跨时钟域同步、按键消抖、脉冲展宽这类电路都会用到这个基本单元。

关键原理：用一个触发器把信号延迟一拍（sig_d），当前值和延迟值做"当前为1、上一拍为0"的组合逻辑判断（sig_in & ~sig_d），这样只有在信号真正从0跳变到1的那一拍，两个条件才会同时成立，输出一个宽度恰好1个时钟周期的脉冲。

延伸考点：如果要检测下降沿，只需要把逻辑换成~sig_in & sig_d；如果要同时检测上升和下降两种边沿，可以用异或 sig_in ^ sig_d；如果输入信号来自异步时钟域（比如外部按键、跨时钟域信号），在打这一拍之前还需要先加两级同步器消除亚稳态风险，这是这道题在真实场景里最容易被继续追问的延伸点。

Q2. Digital Design: Why Async FIFO Pointers Must Be Gray-Coded

QUESTION

In an asynchronous FIFO, why must the read/write pointers be Gray-coded before crossing clock domains? What specific benefit does Gray code provide over plain binary?

ANSWER

```
// The core problem Gray code solves:
//   A FIFO pointer crossing from one clock domain to another is
//   typically passed through a 2-flop synchronizer to resolve
//   metastability. A 2-flop synchronizer guarantees that EACH
//   INDIVIDUAL BIT eventually resolves cleanly to either its old or new
//   value -- but it says NOTHING about whether MULTIPLE bits that
//   happen to change AT THE SAME TIME will all resolve CONSISTENTLY
//   together, sampled on the same receiving-domain clock edge.
//
//   If the pointer is plain BINARY, incrementing it can flip MANY bits
//   simultaneously (e.g. 3 -> 4 is 011 -> 100, all 3 bits change at
//   once). If the receiving domain samples mid-transition, it could
//   see some bits already updated and others not yet updated -- landing
//   on a bit pattern that was NEVER a real, valid pointer value at all
//   (not just "off by one," but potentially wildly wrong).
//
//   If the pointer is GRAY-CODED, by construction only ONE bit ever
//   changes between any two consecutive values. So no matter how the
//   synchronizer's timing lines up relative to the transition, the
//   receiving domain can only ever see EITHER the OLD value or the NEW
//   value of that single changing bit -- both are legitimate points in
//   the sequence. Worst case, the reader is just "one cycle behind" --
//   it can never land on a corrupted, invalid pointer value.
//
// Concrete benefit for the FIFO specifically: full/empty detection
// depends on comparing the synchronized pointer from the OTHER clock
// domain against the local pointer. If that synchronized value could
// ever be a corrupted binary pattern, the FIFO could incorrectly
// report full/empty (or worse, silently overflow/underflow) -- Gray
// coding the pointers before synchronization is what makes this
// comparison SAFE across the clock domain boundary.
//
// Implementation pattern: keep TWO forms of each pointer -- a binary
// version (used locally to index the memory array) and a Gray-coded
// version derived combinationally (gray = bin ^ (bin >> 1)), and ONLY
// the Gray version ever crosses the clock domain through the
// synchronizer.
```

解释

考点：这是异步FIFO/跨时钟域设计里最经典的"为什么"类问题，考察点不是记住格雷码的定义，而是理解"两级同步器只保证单比特收敛，不保证多比特同时收敛"这个根本原因。

关键理解：二进制计数器递增经常好几位同时翻转，跨时钟域采样如果恰好卡在翻转中间，读到的可能是一个从来没有真实存在过的错误值；格雷码保证相邻两个值永远只差一位，这样最坏情况下也只是读到"旧值"或"新值"之一，不会读到一个无效的中间态，这是格雷码在CDC场景下不可替代的核心原因。

为什么FIFO特别依赖这个特性：满/空判断依赖比较本地指针和从对侧同步过来的指针，如果同步过来的指针可能是一个损坏的二进制值，满/空判断就可能出错，导致FIFO在实际已经满的时候还继续写入（溢出）或者已经空的时候还继续读取（下溢），这是格雷码指针在异步FIFO设计里被认为是标准做法、几乎没有例外的原因。

延伸考点：面试官可能会追问格雷码和二进制之间具体怎么转换（二进制转格雷码是g=b^(b>>1)，是O(1)的一次异或；格雷码转回二进制则需要从最高位开始逐位异或下去），以及为什么FIFO内部索引存储器还要保留一份二进制指针（因为格雷码不适合直接做地址运算/数组下标索引，二进制版本仅供本地使用，从不跨时钟域）。

Q3. Digital Design: The ASIC Design Flow, Stage by Stage

QUESTION

Describe the ASIC design flow -- the major stages from a design specification to a finished chip, and what happens at each stage.

ANSWER

```
// 1) Specification -- define the functional requirements, interfaces,
//    performance/power/area targets. Everything downstream traces back
//    to this document.
//
// 2) RTL Design -- implement the design's behavior in a hardware
//    description language (SystemVerilog/VHDL), at the register-
//    transfer level (describing behavior in terms of registers and the
//    combinational logic between them, not gates directly).
//
// 3) Functional Verification -- confirm the RTL actually implements
//    the spec correctly, using testbenches (often UVM-based),
//    simulation, assertions, and coverage (code + functional) to build
//    confidence before committing to physical implementation -- bugs
//    found here are FAR cheaper to fix than bugs found after tapeout.
//
// 4) Logic Synthesis -- translate the RTL into a gate-level netlist
//    using a specific standard-cell library, optimizing for the target
//    timing/power/area constraints. This is where verilog-level
//    behavior becomes actual logic gates.
//
// 5) Design for Test (DFT) insertion -- add scan chains, built-in
//    self-test (BIST) structures, etc. so that manufactured chips can
//    actually be TESTED for defects after fabrication (a perfect
//    design is worthless if you can't verify each physical chip was
//    fabricated correctly).
//
// 6) Physical Design (Place & Route) -- take the gate-level netlist and
//    physically place standard cells on the die, then route the wires
//    connecting them, meeting timing (setup/hold), power, and
//    congestion constraints. Includes clock tree synthesis (CTS) to
//    distribute the clock with controlled skew.
//
// 7) Static Timing Analysis (STA) -- verify the physical implementation
//    meets timing (setup and hold) across all specified process/
//    voltage/temperature (PVT) corners, WITHOUT needing to simulate
//    every possible input pattern -- STA analyzes the timing graph
//    directly.
//
// 8) Physical Verification -- DRC (Design Rule Check: does the layout
//    obey the foundry's manufacturing rules) and LVS (Layout vs
//    Schematic: does the physical layout actually match the intended
//    netlist) -- both must pass before a design can be sent to
//    fabrication.
//
// 9) Tapeout -- the finalized design (GDSII file) is sent to the
//    foundry for manufacturing (fabrication) into actual silicon wafers.
//
// 10) Post-silicon validation / bring-up -- once fabricated chips come
//     back, test them on real hardware (testers, bring-up boards) to
//     confirm the actual silicon behaves as designed, and characterize
```

```
//     it across real-world conditions.
//
// Note: verification (step 3) and physical/timing verification (steps
// 7-8) are NOT a single one-time gate -- in practice there are
// multiple iteration loops (e.g. synthesis and place & route often
// need to loop back if timing isn't met, and RTL bugs found late can
// force a return all the way to step 2), the flow is rarely a single
// clean pass through these stages.
```

解释

考点：ASIC设计流程是芯片设计入门必须掌握的全局图景，考察点是能不能把从"想法"到"芯片"完整地串起来讲清楚每一步在做什么、为什么需要这一步，而不是背出一串名词。

关键分层理解：这个流程大致可以分成"做什么"（spec、RTL设计）、"对不对"（功能验证）、"怎么变成电路"（综合、物理设计）、"能不能造出来且造对了"（DFT、时序分析、物理验证）、"造出来了"（tapeout、后硅验证）这几个阶段，按这个逻辑记忆比死记步骤名字更容易讲清楚。

为什么验证要放在综合之前反复强调：功能验证阶段发现的bug只需要改RTL重新跑仿真，成本很低；一旦进入物理设计甚至流片之后才发现功能性bug，返工成本会指数级上升（可能需要重新走一遍综合、布局布线，甚至重新流片），这也是为什么"验证的核心价值是尽早发现问题"这句话在验证方向面试里反复出现。

延伸考点：面试官可能会追问DRC和LVS具体检查什么、为什么两个都需要（DRC保证版图本身符合工艺制造规则，LVS保证版图真的实现了你想要的电路网表，两者检查的对象不同，任何一个不过都不能送去流片），以及为什么STA不需要像仿真那样穷举所有输入模式就能验证时序（STA直接分析电路的时序路径图，检查每条路径的延迟是否满足建立/保持时间要求，这是一种静态的、基于路径的分析方法，不依赖具体输入向量）。

Q4. Digital Design: Camera Sensor to FPGA Image Data Interface

QUESTION

Describe how a camera sensor's captured image data typically gets transmitted into an FPGA. What are the common interface options, and what does the receiving RTL look like for the simplest case?

ANSWER

```
// Common camera-to-FPGA interface options:
//
//   1) Parallel camera interface (most common for simple/hobbyist
//      camera modules, e.g. OV7670-style sensors):
//      - A parallel data bus (commonly 8 bits, sometimes 10/12 for
//        higher color depth) carrying pixel data.
//      - PCLK (pixel clock): a clock accompanying the data, one pixel
//        (or one byte of a multi-byte pixel format) transferred per
//        PCLK edge.
//      - HREF/HSYNC: marks valid pixel data within one horizontal line.
//      - VSYNC: marks the start of a new frame.
//      - A separate low-speed control interface (typically I2C/SCCB)
//        is used to CONFIGURE the sensor (resolution, format, exposure)
//        -- this is control-plane, separate from the parallel pixel
//        data path.
//
//   2) MIPI CSI-2 (common in phone-class/higher-resolution sensors):
//      - A high-speed SERIAL differential interface (D-PHY physical
//        layer), much higher bandwidth than a parallel bus, but
```

```
//        requires a dedicated MIPI receiver (D-PHY + CSI-2 protocol
//        decoder) in the FPGA fabric -- either a hard IP block if the
//        FPGA has one, or a soft IP core, since bit-level SERDES timing
//        recovery is not something you'd hand-roll from scratch.
//      - Significantly more complex to implement than the parallel
//        option, but necessary for higher resolutions/frame rates than
//        a parallel bus can practically support.
//
// RTL receiver structure for the PARALLEL case (the simpler, more
// common case for a smaller project):
module camera_capture (
  input  logic       pclk,      // pixel clock from the camera
  input  logic       vsync,     // frame start
  input  logic       href,      // line-valid
  input  logic [7:0] pixel_data,
  output logic [9:0] pixel_x,
  output logic [9:0] pixel_y,
  output logic [7:0] pixel_out,
  output logic       pixel_valid
);
  // PCLK is the camera's OWN clock domain, different from the FPGA's
  // internal system clock -- everything in this module should be
  // clocked BY pclk (or synchronized into the system clock domain
  // afterward via a dual-clock FIFO, since pixel data typically needs
  // to cross into the FPGA's main processing clock domain).
  always_ff @(posedge pclk) begin
    if (vsync) begin
      pixel_y <= '0;              // new frame: reset row counter
    end else if (href) begin
      pixel_x     <= pixel_x + 1'b1;
      pixel_out   <= pixel_data;
      pixel_valid <= 1'b1;
    end else begin
      pixel_valid <= 1'b0;
      if (pixel_x != 0) begin      // end of a line -> advance row, reset column
        pixel_y <= pixel_y + 1'b1;
        pixel_x <= '0;
      end
    end
  end
endmodule
// (this is a simplified sketch -- a real design typically buffers the
// captured pixel stream into a dual-clock FIFO immediately, to safely
// cross from the camera's pclk domain into the FPGA's main system
// clock domain for further processing.)
```

解释

考点：这是一道"了解不了解常见接口"的广度题，考察是否知道摄像头到FPGA传输图像数据的主流方案有哪些、各自的复杂度和适用场景，而不需要精通每一种协议的每一个细节。

两种方案的核心取舍：并行接口（数据线+像素时钟+行同步+场同步）简单、容易自己写RTL接收，适合小项目和低分辨率场景；MIPI CSI-2是高速串行差分信号，带宽大得多，但需要专门的D-PHY物理层接收器和协议解码逻辑，通常依赖厂商提供的硬核IP或者第三方IP核，不会从零手写，这是两者复杂度和适用场景上最本质的区别。

接收端RTL的关键设计点：核心逻辑是用vsync标记新的一帧开始（复位行计数器），用href标记一行内的有效像素数据（在href有效期间累加列计数器、锁存像素数据），href无效时说明一行结束，这时候才推进行计数器——本质是用两级同步信号（帧同步、行同步）把一维的像素数据流正确地映射回二维图像坐标。

延伸考点：面试官很可能会追问"PCLK和FPGA内部系统时钟是不是同一个时钟域"（通常不是，摄像头的PCLK是独立时钟源，捕获到的像素数据流通常需要立刻送入一个异步FIFO跨时钟域到FPGA主处理时钟，这里天

然会和"异步FIFO为什么用格雷码"这道题连起来考察），以及如果换成MIPI CSI-2输入，RTL接收端的复杂度会怎么变化（需要额外处理串并转换、通道对齐、协议包解析等，比并行接口复杂得多）。

Q5. Digital Design: Async Assert, Sync Deassert Reset

QUESTION

Implement reset circuitry that asserts asynchronously but deasserts synchronously using a 2-flop synchronizer chain.

Constraints:

- Assert: immediately when async_rst_n goes low

- Deassert: only on clock edge(s) after release

- 2-flop chain for synchronous deassert

ANSWER

```
module reset_sync (
  input  logic clk,
  input  logic async_rst_n,
  output logic rst_n_sync
);
  logic ff1, ff2;

  // 2-flop synchronizer with async assert, sync deassert
  always_ff @(posedge clk or negedge async_rst_n) begin
    if (!async_rst_n) begin
      ff1 <= 0;
      ff2 <= 0;
    end else begin
      ff1 <= 1;
      ff2 <= ff1;
    end
  end

  assign rst_n_sync = ff2;
endmodule
```

解释

考点：这是基础RTL设计题，重点考察时序逻辑/组合逻辑划分是否清晰、边界条件是否处理完整。

涉及跨时钟域的部分要用两级同步器处理，格雷码保证多比特信号跨域时最多只有一位在变化。

延伸考点：面试官通常会在这道题基础上继续追问边界条件（比如参数取极值时是否还正确）或者要求扩展功能，建议实现时就考虑清楚这些点，回答时主动提及。

Q6. Digital Design: FSM Sequence Detector for 1,0,1,1,0

QUESTION

Implement an FSM detecting serial pattern `10110` with a 1-cycle match pulse.

Constraints:

- States represent progress through matching
- Define overlapping support
- Reset returns to IDLE

ANSWER

```
// Rewritten in standard THREE-SEGMENT FSM style (state register /
// next-state logic / output logic each in their own dedicated,
// single-purpose always block -- the original version mixed the
// state register and next-state logic together in one always_ff,
// which is against the discipline this document uses consistently
// elsewhere).
module seq_det_10110 (
  input  logic clk,
  input  logic rst_n,
  input  logic bit_in,
  output logic match_pulse
);
  typedef enum logic [2:0] {
    S0,    // No match progress
    S1,    // Seen "1"
    S10,   // Seen "10"
    S101,  // Seen "101"
    S1011  // Seen "1011"
  } state_t;

  state_t cur_state, next_state;

  // Segment 1: state register -- ONLY the register, nothing else
  always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) cur_state <= S0;
    else        cur_state <= next_state;
  end

  // Segment 2: next-state logic -- pure combinational, no output driven here
  always_comb begin
    next_state = cur_state;
    unique case (cur_state)
      S0:    next_state = state_t'(bit_in ? S1    : S0);
      S1:    next_state = state_t'(bit_in ? S1    : S10);
      S10:   next_state = state_t'(bit_in ? S101  : S0);
      S101:  next_state = state_t'(bit_in ? S1011 : S10);
      S1011: next_state = state_t'(bit_in ? S1    : S0);   // "10110" complete on 0
      default: next_state = S0;
    endcase
  end

  // Segment 3: output logic -- pure combinational, Mealy-style (depends
  // on cur_state AND the current input, not just cur_state)
  always_comb begin
    match_pulse = (cur_state == S1011) && !bit_in;
  end
endmodule

// Verified by simulation (Icarus Verilog): driving the bit sequence
// 1,0,1,1,0,1,0,1,1,0 (which contains "10110" starting at index 0 and
// again at index 5) produces exactly 2 match_pulse assertions, at
// index 4 and index 9 -- i.e. the cycle immediately after the
// completing '0' of each occurrence, matching the Mealy timing
// intended by the design.
```

解释

考点：这是"三段式"状态机规范的直接应用题——原始版本把状态寄存器和次态逻辑混在同一个always_ff块里，虽然功能上勉强能跑，但不符合三段式"时序归时序、组合归组合"的纪律，这次按标准三段式重写并重新验证过。

三段分工：第一段只做cur_state<=next_state这一件事，不掺杂任何判断逻辑；第二段纯组合逻辑根据cur_state和bit_in算出next_state，不驱动任何输出信号；第三段纯组合逻辑根据cur_state（这里因为是Mealy机，还要加上bit_in）算出match_pulse，不做任何状态判断。三者严格分开，是这份文档里所有状态机设计题统一遵循的规范。

Mealy机的细节：match_pulse是组合逻辑直接依赖bit_in（不是纯Moore只依赖cur_state），这是有意为之——因为"10110"模式匹配的判定天然需要"当前处于S1011状态且这一拍恰好输入0"这个组合条件，如果做成纯Moore（只依赖状态）就没法在这一拍就宣布匹配，必须多等一拍，这也是这道题跟之前"Mealy vs Moore"那道概念题可以呼应的地方。

延伸考点：面试官可能会追问如果要检测重叠模式（比如输入是"101101011"这种自身就有重叠子串的情况）next_state逻辑要不要改（这里的next_state设计已经支持重叠——S1011遇到bit_in=1会跳到S1而不是从S0重新开始，正确处理了"10110"末尾的"0"之后如果紧跟一个新的"1"可以立刻开始下一次匹配尝试的重叠场景）。

Q7. Digital Design: Fibonacci Sequence Generator with Enable

QUESTION

Implement a Fibonacci generator advancing only when enable is asserted.

State: `a, b registers; next = a + b`

Constraints:

- Parameterizable output width W

- enable=0 holds output

- Reset initializes to produce 0, 1, 1, 2, 3, 5, ...

ANSWER

```
module fib_gen #(
  parameter W = 16
)(
  input  logic          clk,
  input  logic          rst_n,
  input  logic          enable,
  output logic [W-1:0]  fib_out
);
  logic [W-1:0] a, b;

 // a=0, b=1 on reset → outputs: 0, 1, 1, 2, 3, 5, ...
  always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
      a <= '0;
      b <= 1;
    end else if (enable) begin
      a <= b;
      b <= a + b;
    end
  end
```

```
  assign fib_out = a;
endmodule
```

解释

考点：这是基础RTL设计题，重点考察时序逻辑/组合逻辑划分是否清晰、边界条件是否处理完整。

题目要求参数化（可配置位宽/深度），写代码时应该用parameter/localparam而不是硬编码常量，保证模块可复用。

延伸考点：面试官通常会在这道题基础上继续追问边界条件（比如参数取极值时是否还正确）或者要求扩展功能，建议实现时就考虑清楚这些点，回答时主动提及。

Q8. Digital Design: Implement Parameterizable Barrel Shifter for Shift Operations

QUESTION

Design a single-cycle barrel shifter for SLL, SRL, and SRA operations.

Example (WIDTH=8):

SRL: 0x80, shift=1 → 0x40 (zero-fill MSB)

SRA: 0x80, shift=1 → 0xC0 (sign-extend MSB)

SLL: 0x01, shift=3 → 0x08 (zero-fill LSB)

Constraints:

- Parameterizable WIDTH, shift amount = $clog2(WIDTH) bits

- Single-cycle combinational design

- Three operations: SLL (00), SRL (01), SRA (10)

ANSWER

```
module barrel_shifter #(
  parameter WIDTH = 8
)(
  input  logic [WIDTH-1:0]          data_in,
  input  logic [$clog2(WIDTH)-1:0]  shift_amt,
  input  logic [1:0]                shift_op,   // 00=SLL, 01=SRL, 10=SRA
  output logic [WIDTH-1:0]          data_out
);
  always_comb begin
    case (shift_op)
      2'b00: data_out =   data_in  << shift_amt;          // SLL
      2'b01: data_out =   data_in  >> shift_amt;          // SRL
      2'b10: data_out = $signed(data_in) >>> shift_amt;   // SRA
      default: data_out = data_in;
    endcase
  end
endmodule
```

解释

考点：这是基础RTL设计题，重点考察时序逻辑/组合逻辑划分是否清晰、边界条件是否处理完整。

题目要求参数化（可配置位宽/深度），写代码时应该用parameter/localparam而不是硬编码常量，保证模块可复用。

延伸考点：面试官通常会在这道题基础上继续追问边界条件（比如参数取极值时是否还正确）或者要求扩展功能，建议实现时就考虑清楚这些点，回答时主动提及。

Q9. Digital Design: Design APB Slave Peripheral Following AMBA APB Protocol

QUESTION

Implement an APB slave compliant with AMBA APB two-phase transfer protocol.

Transfer phases:

Setup: PSEL=1, PENABLE=0 (decode address)

Enable: PSEL=1, PENABLE=1 (transfer when PREADY=1)

Constraints:

- Two-phase transfer: setup + enable

- PREADY for wait states

- PSLVERR for error response

ANSWER

```
module apb_slave #(
  parameter DATA_WIDTH = 32,
  parameter ADDR_WIDTH = 8
)(
  input  logic                      PCLK,
  input  logic                      PRESETn,
  // APB signals
  input  logic                      PSEL,
  input  logic                      PENABLE,
  input  logic                      PWRITE,
  input  logic [ADDR_WIDTH-1:0]     PADDR,
  input  logic [DATA_WIDTH-1:0]     PWDATA,
  output logic [DATA_WIDTH-1:0]     PRDATA,
  output logic                      PREADY,
  output logic                      PSLVERR
);
  localparam ADDR_REG0 = 8'h00;
  localparam ADDR_REG1 = 8'h04;
  localparam ADDR_REG2 = 8'h08;

  logic [DATA_WIDTH-1:0] reg0;
  logic [DATA_WIDTH-1:0] reg1;   // Read-only (hardcoded for demo)
  logic                  addr_valid;

  assign addr_valid = (PADDR == ADDR_REG0) || (PADDR == ADDR_REG1) || (PADDR == ADDR_REG2);

  // Zero-wait-state: always ready
  assign PREADY  = 1'b1;
  assign PSLVERR = PSEL && PENABLE && !addr_valid;

  // Transfer complete condition: PSEL && PENABLE && PREADY
  // Write
  always_ff @(posedge PCLK or negedge PRESETn) begin
    if (!PRESETn) begin
      reg0 <= '0;
    end else if (PSEL && PENABLE && PREADY && PWRITE) begin
```

```
      case (PADDR)
        ADDR_REG0: reg0 <= PWDATA;
        default: ;
      endcase
    end
  end

  // Read (combinational)
  always_comb begin
    PRDATA = '0;
    if (!PWRITE) begin
      case (PADDR)
        ADDR_REG0: PRDATA = reg0;
        ADDR_REG1: PRDATA = 32'hDEAD_C0DE;   // Hardcoded read-only
        ADDR_REG2: PRDATA = 32'hCAFE_BABE;
        default:   PRDATA = '0;
      endcase
    end
  end
endmodule
```

解释

考点：这是基础RTL设计题，重点考察时序逻辑/组合逻辑划分是否清晰、边界条件是否处理完整。

总线协议类设计要特别注意握手信号（valid/ready或者addr/data phase）的时序约定是否符合协议规范，读写通道往往需要分别独立处理状态机。

延伸考点：面试官通常会在这道题基础上继续追问边界条件（比如参数取极值时是否还正确）或者要求扩展功能，建议实现时就考虑清楚这些点，回答时主动提及。

Q10. Digital Design: Build AXI-Lite Write-Only Control Block

QUESTION

Implement a write-only AXI-Lite peripheral with enable and mode control registers.

Register map:

0x00: enable register (write-only)

0x04: mode register (write-only)

Any read → error response

Constraints:

- Writes update control registers

- Reads always return error on R channel

- Reset clears enable and mode

ANSWER

```
module axi_write_only_ctrl (
  input  logic        ACLK,
  input  logic        ARESETn,
  // Write
  input  logic        AWVALID, output logic AWREADY, input  logic [31:0] AWADDR,
  input  logic        WVALID,  output logic WREADY,  input  logic [31:0] WDATA, input logic [3
```

```
:0] WSTRB,
  output logic        BVALID,  input  logic BREADY,  output logic [1:0]  BRESP,
  // Read (always error)
  input  logic        ARVALID, output logic ARREADY, input  logic [31:0] ARADDR,
  output logic        RVALID,  input  logic RREADY,  output logic [31:0] RDATA, output logic [
1:0] RRESP,
  // Control outputs
  output logic        enable_reg,
  output logic [31:0] mode_reg
);
  logic aw_done, w_done;
  logic [31:0] aw_lat, wd_lat;

  assign AWREADY = !aw_done;
  assign WREADY  = !w_done;
  assign ARREADY = !RVALID;

  always_ff @(posedge ACLK or negedge ARESETn) begin
    if (!ARESETn) begin aw_done <= 0; aw_lat <= '0; end
    else begin
      if (AWVALID && AWREADY) begin aw_done <= 1; aw_lat <= AWADDR; end
      if (BVALID && BREADY)   aw_done <= 0;
    end
  end

  always_ff @(posedge ACLK or negedge ARESETn) begin
    if (!ARESETn) begin w_done <= 0; wd_lat <= '0; end
    else begin
      if (WVALID && WREADY) begin w_done <= 1; wd_lat <= WDATA; end
      if (BVALID && BREADY) w_done <= 0;
    end
  end

  always_ff @(posedge ACLK or negedge ARESETn) begin
    if (!ARESETn) begin enable_reg <= 0; mode_reg <= '0; end
    else if (aw_done && w_done && !BVALID) begin
      case (aw_lat)
        32'h00: enable_reg <= wd_lat[0];
        32'h04: mode_reg   <= wd_lat;
        default: ;
      endcase
    end
  end

  always_ff @(posedge ACLK or negedge ARESETn) begin
    if (!ARESETn) begin BVALID <= 0; BRESP <= 2'b00; end
    else begin
      if (aw_done && w_done && !BVALID) begin
        BVALID <= 1;
        BRESP  <= (aw_lat inside {32'h00, 32'h04}) ? 2'b00 : 2'b10;
      end else if (BVALID && BREADY) BVALID <= 0;
    end
  end

  // All reads return SLVERR
  always_ff @(posedge ACLK or negedge ARESETn) begin
    if (!ARESETn) begin RVALID <= 0; RDATA <= '0; RRESP <= 2'b10; end
    else begin
      if (ARVALID && ARREADY) begin RVALID <= 1; RDATA <= '0; RRESP <= 2'b10; end
      else if (RVALID && RREADY) RVALID <= 0;
    end
  end
endmodule
```

解释

考点：这是基础RTL设计题，重点考察时序逻辑/组合逻辑划分是否清晰、边界条件是否处理完整。

总线协议类设计要特别注意握手信号（valid/ready或者addr/data phase）的时序约定是否符合协议规范，读写通道往往需要分别独立处理状态机。

延伸考点：面试官通常会在这道题基础上继续追问边界条件（比如参数取极值时是否还正确）或者要求扩展功能，建议实现时就考虑清楚这些点，回答时主动提及。

Q11. Digital Design: Design Write-Back Set-Associative Cache with LRU or FIFO Replacement

QUESTION

Implement a set-associative cache with write-back policy, dirty bit tracking, and LRU replacement.

Operations:

Write hit: update cache + set dirty (no memory write)

Clean eviction: no writeback needed

Dirty eviction: writeback to memory before refill

Constraints:

- Set-associative with parallel tag comparison
- LRU replacement (document choice)
- Dirty bit management for write-back

ANSWER

```
// 4-way set-associative write-back cache with LRU replacement
module wb_cache #(
  parameter NUM_SETS   = 4,
  parameter NUM_WAYS   = 4,
  parameter TAG_BITS   = 4,
  parameter DATA_WIDTH = 8
)(
  input  logic                   clk,
  input  logic                   rst_n,
  input  logic                   access,
  input  logic                   write_en,
  input  logic [TAG_BITS-1:0]   addr_tag,
  input  logic [$clog2(NUM_SETS)-1:0] addr_set,
  input  logic [DATA_WIDTH-1:0]  write_data,
  output logic [DATA_WIDTH-1:0]  read_data,
  output logic                    hit,
  output logic                    miss,
  // Writeback interface
  output logic                    wb_req,
  output logic [TAG_BITS-1:0]    wb_tag,
  output logic [$clog2(NUM_SETS)-1:0] wb_set,
  output logic [DATA_WIDTH-1:0]  wb_data
);
  logic                  valid [NUM_SETS][NUM_WAYS];
  logic                  dirty [NUM_SETS][NUM_WAYS];
  logic [TAG_BITS-1:0]   tags  [NUM_SETS][NUM_WAYS];
  logic [DATA_WIDTH-1:0] data  [NUM_SETS][NUM_WAYS];
  logic [1:0]            lru   [NUM_SETS][NUM_WAYS]; // LRU rank: 3=MRU, 0=LRU
```

```
  logic [$clog2(NUM_WAYS)-1:0] hit_way;
  logic [$clog2(NUM_WAYS)-1:0] victim_way;
  logic                        any_hit;

  // Hit detection
  always_comb begin
    any_hit  = 0;
    hit_way  = '0;
    for (int w = 0; w < NUM_WAYS; w++) begin
      if (valid[addr_set][w] && tags[addr_set][w] == addr_tag) begin
        any_hit = 1;
        hit_way = w[$clog2(NUM_WAYS)-1:0];
      end
    end
  end

  // LRU victim selection
  always_comb begin
    victim_way = '0;
    // First prefer invalid way
    for (int w = NUM_WAYS-1; w >= 0; w--) begin
      if (!valid[addr_set][w]) victim_way = w[$clog2(NUM_WAYS)-1:0];
    end
    // If all valid, pick LRU (rank == 0)
    if (&{valid[addr_set][0], valid[addr_set][1], valid[addr_set][2], valid[addr_set][3]}) beg
in
      for (int w = 0; w < NUM_WAYS; w++) begin
        if (lru[addr_set][w] == 2'd0) victim_way = w[$clog2(NUM_WAYS)-1:0];
      end
    end
  end

  assign hit      = access && any_hit;
  assign miss     = access && !any_hit;
  assign read_data = data[addr_set][hit_way];
  assign wb_req   = miss && valid[addr_set][victim_way] && dirty[addr_set][victim_way];
  assign wb_tag   = tags[addr_set][victim_way];
  assign wb_set   = addr_set;
  assign wb_data  = data[addr_set][victim_way];

  always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
      for (int s = 0; s < NUM_SETS; s++) begin
        for (int w = 0; w < NUM_WAYS; w++) begin
          valid[s][w] <= 0;
          dirty[s][w] <= 0;
          lru[s][w]   <= w[1:0];
        end
      end
    end else if (access) begin
      if (any_hit) begin
        // Hit: update data on write, always update LRU
        if (write_en) begin
          data[addr_set][hit_way]  <= write_data;
          dirty[addr_set][hit_way] <= 1;
        end
        begin
          logic [1:0] old_rank;
          old_rank = lru[addr_set][hit_way];
          for (int w = 0; w < NUM_WAYS; w++) begin
            if (w[$clog2(NUM_WAYS)-1:0] == hit_way)
              lru[addr_set][w] <= 2'd3;
```

```
            else if (lru[addr_set][w] > old_rank)
              lru[addr_set][w] <= lru[addr_set][w] - 1'b1;
          end
        end
      end else begin
        // Miss: evict victim, fill new line
        valid[addr_set][victim_way] <= 1;
        dirty[addr_set][victim_way] <= write_en;
        tags[addr_set][victim_way]  <= addr_tag;
        data[addr_set][victim_way]  <= write_data;
        begin
          logic [1:0] old_rank;
          old_rank = lru[addr_set][victim_way];
          for (int w = 0; w < NUM_WAYS; w++) begin
            if (w[$clog2(NUM_WAYS)-1:0] == victim_way)
              lru[addr_set][w] <= 2'd3;
            else if (lru[addr_set][w] > old_rank)
              lru[addr_set][w] <= lru[addr_set][w] - 1'b1;
          end
        end
      end
    end
  end
endmodule
```

解释

考点：这是基础RTL设计题，重点考察时序逻辑/组合逻辑划分是否清晰、边界条件是否处理完整。

替换策略类设计的关键是准确维护"最近使用"信息的更新时机，命中和缺失两条路径的更新逻辑往往不对称，容易漏掉某一支。

延伸考点：面试官通常会在这道题基础上继续追问边界条件（比如参数取极值时是否还正确）或者要求扩展功能，建议实现时就考虑清楚这些点，回答时主动提及。

Q12. Digital Design: Design Pipelined ALU with Hazard Detection and Forwarding

QUESTION

Implement a 2-stage pipelined ALU with valid bit propagation, data forwarding for RAW hazards, and flush support.

Pipeline stages:

Stage 1 (EX): Execute operation

Stage 2 (WB): Write back result

Throughput: 1 op/cycle after fill

Constraints:

- Forwarding resolves RAW hazards without stalling
- Valid bit propagates through stages
- Flush invalidates in-flight operations

ANSWER

```
module pipelined_alu #(
```

```
  parameter WIDTH    = 32,
  parameter REG_ADDR = 5
)(
  input  logic                   clk,
  input  logic                   rst_n,
  input  logic                   flush,
  input  logic                   input_valid,
  input  logic [3:0]             opcode,
  input  logic [REG_ADDR-1:0]    src_a_reg,
  input  logic [REG_ADDR-1:0]    src_b_reg,
  input  logic [WIDTH-1:0]       src_a_value,
  input  logic [WIDTH-1:0]       src_b_value,
  input  logic [REG_ADDR-1:0]    dest_reg,
  output logic                   wb_valid,
  output logic [WIDTH-1:0]       wb_result,
  output logic [REG_ADDR-1:0]    wb_dest
);
  logic                    ex_valid;
  logic [WIDTH-1:0]        ex_result;
  logic [REG_ADDR-1:0]     ex_dest;

  logic [WIDTH-1:0]        op_a_eff, op_b_eff;

  function automatic logic [WIDTH-1:0] alu_op(
    input logic [3:0]       op,
    input logic [WIDTH-1:0] a,
    input logic [WIDTH-1:0] b
  );
    case (op)
      4'd0: alu_op = a + b;
      4'd1: alu_op = a - b;
      4'd2: alu_op = a & b;
      4'd3: alu_op = a | b;
      4'd4: alu_op = a ^ b;
      default: alu_op = '0;
    endcase
  endfunction

  // Simple RAW forwarding for 2-stage pipeline:
  // current EX can bypass from prior cycle result sitting in EX/WB register,
  // and also from WB output if still visible.
  always_comb begin
    op_a_eff = src_a_value;
    op_b_eff = src_b_value;

    if (ex_valid && (src_a_reg == ex_dest) && (src_a_reg != '0)) op_a_eff = ex_result;
    else if (wb_valid && (src_a_reg == wb_dest) && (src_a_reg != '0)) op_a_eff = wb_result;

    if (ex_valid && (src_b_reg == ex_dest) && (src_b_reg != '0)) op_b_eff = ex_result;
    else if (wb_valid && (src_b_reg == wb_dest) && (src_b_reg != '0)) op_b_eff = wb_result;
  end

  always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n || flush) begin
      ex_valid  <= 0;
      ex_result <= '0;
      ex_dest   <= '0;
    end else begin
      ex_valid  <= input_valid;
      ex_result <= alu_op(opcode, op_a_eff, op_b_eff);
      ex_dest   <= dest_reg;
    end
  end
```

```
  always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n || flush) begin
      wb_valid  <= 0;
      wb_result <= '0;
      wb_dest   <= '0;
    end else begin
      wb_valid  <= ex_valid;
      wb_result <= ex_result;
      wb_dest   <= ex_dest;
    end
  end
endmodule
```

解释

考点：这是基础RTL设计题，重点考察时序逻辑/组合逻辑划分是否清晰、边界条件是否处理完整。

流水线相关设计要明确stall和flush同时出现时的优先级，这是题目描述里特别要求要"文档化"的地方，说明这是一个容易产生歧义、必须显式定义的设计决策。

延伸考点：面试官通常会在这道题基础上继续追问边界条件（比如参数取极值时是否还正确）或者要求扩展功能，建议实现时就考虑清楚这些点，回答时主动提及。

Q13. Digital Design: Design Vending Machine FSM with Coin Input and Change Output

QUESTION

Implement a vending machine FSM that accepts coins, accumulates credit, dispenses an item when credit meets the price, and returns change.

Example:

Item price = 25 cents

Insert 10c → credit=10 (no dispense)

Insert 25c → credit=35 >= 25 → dispense=1, change=10c, credit=0

Constraints:

- Accepted coin denominations: 5c, 10c, 25c (5-bit encoded)
- Parameterizable item price (default 25 cents)
- Overpayment returns change the cycle after dispense
- Invalid or zero-value coins are silently ignored

ANSWER

```
// Rewritten in standard THREE-SEGMENT FSM style. The original version
// mixed state register, next-state decision, output logic, AND the
// credit datapath register all together inside one large always_ff
// block -- this version cleanly separates each concern into its own
// dedicated block, matching the discipline used consistently
// throughout this document's other FSM designs.
module vending_machine #(
  parameter ITEM_PRICE = 25
)(
  input  logic        clk,
```

```
  input  logic        rst_n,
  input  logic        coin_valid,
  input  logic [4:0]  coin_value,   // 5, 10, or 25 cents
  output logic        dispense,
  output logic        change_valid,
  output logic [4:0]  change_amount
);
  localparam CREDIT_BITS = 7;

  typedef enum logic [1:0] {IDLE, DISPENSE_STATE, CHANGE_STATE} state_t;
  state_t cur_state, next_state;

  logic [CREDIT_BITS-1:0] credit, credit_next;
  logic valid_coin;

  assign valid_coin = coin_valid &&
    (coin_value == 5 || coin_value == 10 || coin_value == 25);

  // Segment 1: state register -- ONLY the register
  always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) cur_state <= IDLE;
    else        cur_state <= next_state;
  end

  // Credit datapath register (kept separate from the FSM's own state
  // register, same discipline applied: pure register here, pure
  // combinational next-value logic in its own block below).
  always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) credit <= '0;
    else        credit <= credit_next;
  end

  // Segment 2: next-state logic -- pure combinational, no output or
  // datapath register driven here
  always_comb begin
    next_state = cur_state;
    case (cur_state)
      IDLE:
        if (valid_coin && (credit + coin_value) >= ITEM_PRICE)
          next_state = DISPENSE_STATE;
      DISPENSE_STATE:
        next_state = state_t'((credit > ITEM_PRICE) ? CHANGE_STATE : IDLE);
      CHANGE_STATE:
        next_state = IDLE;
      default:
        next_state = IDLE;
    endcase
  end

  // Credit next-value logic -- also pure combinational, but kept in
  // its own block since it's a DATAPATH concern, distinct from the
  // FSM's next-STATE decision (even though both are driven by the
  // same cur_state case structure).
  always_comb begin
    credit_next = credit;
    case (cur_state)
      IDLE:            if (valid_coin) credit_next = credit + coin_value;
      DISPENSE_STATE:  credit_next = credit - ITEM_PRICE;
      CHANGE_STATE:    credit_next = '0;
      default:         credit_next = credit;
    endcase
  end
```

```
  // Segment 3: output logic -- pure combinational, Moore-style
  // (depends only on cur_state, plus the credit register purely to
  // report its VALUE, not to make any state decision)
  always_comb begin
    dispense      = (cur_state == DISPENSE_STATE);
    change_valid  = (cur_state == CHANGE_STATE);
    change_amount = (cur_state == CHANGE_STATE) ? credit[4:0] : 5'd0;
  end
endmodule

// Verified by simulation (Icarus Verilog) against the question's OWN
// worked example, exactly:
//   insert 10c -> credit=10, dispense=0                (matches spec)
//   insert 25c -> credit=35, dispense=1 THIS cycle      (matches spec)
//   next cycle: change_valid=1, change_amount=10        (matches "change=10c")
//   next cycle: credit=0, back in IDLE                  (matches "credit=0")
// Also verified the exact-price edge case (insert exactly 25c when
// price is 25c): dispense=1 fires, but change_valid NEVER fires
// afterward (credit=25 is not > ITEM_PRICE, so the FSM returns
// directly to IDLE without a change cycle) -- correctly handling the
// "no change owed" case without an extra spurious cycle.
```

解释

考点：这是三段式规范要求最严格的一道题——原始版本把状态寄存器、次态判断、输出逻辑、甚至credit这个数据通路寄存器全部揉进了同一个always_ff块，虽然能跑但完全没有做关注点分离，这次按标准三段式彻底重写并且专门针对题目给出的例子做了仿真验证。

三段分工加数据通路：这道题比单纯的状态机多了一个credit累加寄存器，处理方式是把它当成独立于FSM状态寄存器之外的另一个数据通路寄存器，同样遵循"寄存器只做寄存器、下一个值的计算是独立的纯组合逻辑"这个原则——next_state的计算和credit_next的计算虽然都基于cur_state做case判断，但分别放在两个always_comb块里，因为它们是两个不同的关注点（一个决定"下一步走到哪个状态"，一个决定"账户余额下一拍该是多少"）。

用题目自带例子做验证是这道题的亮点：题目描述里给了"插10c、再插25c、找零10c"这个具体例子，重写之后我直接拿这个例子跑仿真验证，结果和题目描述完全吻合，还额外测了一个题目没直接给出但逻辑上一定要处理对的边界情况——刚好投入准确金额（无需找零）时不应该多出一拍不必要的CHANGE_STATE。

延伸考点：面试官可能会追问如果需要支持"退币"按钮（用户投币后可以取消交易要求全额退款）架构要怎么扩展（需要在IDLE状态下增加一个新的分支处理退币请求，next_state直接跳到一个新的REFUND_STATE，credit_next归零，同时refund_amount输出等于退币前的credit值，本质是这个三段式框架下再加一个"输出+归零"的状态，跟CHANGE_STATE的处理套路很相似）。

Q14. Digital Design: Detect Rising, Falling, and Any Transition (Registered Outputs)

QUESTION

Design synchronous RTL that detects transitions on a 1-bit input signal sig_in and produces THREE single-cycle pulse outputs: rise_pulse (fires one cycle after a 0->1 transition), fall_pulse (fires one cycle after a 1->0 transition), and toggle_pulse (fires one cycle after ANY change in either direction). When rst_n is asserted low, sig_in should be treated as 0.

ANSWER

```
module edge_toggle_detect (
```

```
  input  logic clk,
  input  logic rst_n,
  input  logic sig_in,
  output logic rise_pulse,
  output logic fall_pulse,
  output logic toggle_pulse
);
  logic sig_prev;

  always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
      sig_prev     <= 0;
      rise_pulse   <= 0;
      fall_pulse   <= 0;
      toggle_pulse <= 0;
    end else begin
      rise_pulse   <=  sig_in & ~sig_prev;
      fall_pulse   <= ~sig_in &  sig_prev;
      toggle_pulse <=  sig_in ^  sig_prev;
      sig_prev     <= sig_in;
    end
  end
endmodule

// Verified by simulation: each pulse output correctly fires exactly ONE
// cycle after its corresponding transition (these are REGISTERED
// outputs, unlike the earlier combinational rising-edge-only detector
// in this document -- a deliberate different timing convention worth
// noticing: this version trades 1 cycle of latency for a cleaner,
// glitch-free registered output).
```

解释

考点：这是之前"单bit上升沿检测"那道题的完整扩展版——同时检测上升沿、下降沿、任意跳变三种情况，并且这次是寄存器输出（比较值那一拍之后才出脉冲），跟之前的组合逻辑版本是两种不同的时序设计选择。

关键区别：组合逻辑版本（sig_in & ~sig_d）在跳变发生的当拍就能看到脉冲，延迟为0但组合路径稍长；这道题的寄存器版本把rise/fall/toggle都做成时序输出，跳变发生后要等一拍才出脉冲，换来的是更干净、时序上更容易收敛的输出（下一级电路可以直接把这当成一个正常的寄存器信号处理，不用担心组合逻辑毛刺）。

toggle_pulse的实现技巧：用异或（sig_in ^ sig_prev）同时覆盖上升和下降两种跳变，比"rise_pulse | fall_pulse"更直接。

延伸考点：面试官可能会追问这两种设计（组合输出 vs 寄存器输出）各自适合什么场景——组合版本适合对延迟敏感、下一级能容忍组合逻辑的场景；寄存器版本适合时序要求严格、需要跟其他寄存器信号对齐使用的场景，这是同一个功能需求下两种常见的工程取舍。

Q15. Digital Design: Mealy vs Moore FSM -- the Sync/Async Output Distinction

QUESTION

There are two types of finite state machines. What are they, what is the difference between them, and what is the SINGLE MOST SIGNIFICANT difference (hint: think about it from a synchronous vs asynchronous angle)?

ANSWER

```
// The two types: MOORE and MEALY.
//
// Moore machine:  output = f(state)          -- output depends ONLY on
//                                                the current state.
// Mealy machine:  output = f(state, input)   -- output depends on the
//                                                current state AND the
//                                                current input.
//
// The single most significant difference (the sync/async angle):
//   A Moore machine's output is a pure function of STATE, and state
//   only ever changes on a clock edge (it's held in a register) -- so
//   a Moore output is effectively SYNCHRONOUS/registered in nature: it
//   only changes at clock edges, remains stable and glitch-free for
//   the ENTIRE clock cycle, regardless of what the inputs do mid-cycle.
//
//   A Mealy machine's output is a COMBINATIONAL function that includes
//   the current input directly -- so if the input changes in the
//   middle of a clock cycle (glitches, arrives asynchronously relative
//   to the clock, etc.), the Mealy output can change IMMEDIATELY,
//   right along with it, WITHOUT waiting for the next clock edge. This
//   makes Mealy outputs effectively ASYNCHRONOUS with respect to the
//   input -- they can glitch, and they are only as clean/stable as the
//   input signal driving them.
//
// Practical consequence of this distinction: Moore outputs are safer
// to feed directly into another clocked block (guaranteed stable for
// the whole cycle); Mealy outputs can react ONE CYCLE EARLIER to an
// input change (a real advantage in some designs), but that same
// combinational input-dependence is exactly what makes them prone to
// glitching if the input itself isn't clean.
```

解释

考点：Mealy和Moore状态机是数字设计最基础的分类题，几乎每场面试都会问，但"两者最本质的区别是什么"这个追问经常会把只会背定义的人问住——这道题面试官明确提示"从同步/异步的角度想"，是在引导候选人往"输出的时序特性"这个方向思考，而不是停留在"输出是否依赖输入"这个字面定义上。

关键理解：Moore的输出只依赖state，而state只有在时钟边沿才会变化，所以Moore的输出本质上是"跟着时钟走"的，同步、干净、整个周期内保持稳定；Mealy的输出直接把当前输入也算进组合逻辑里，只要输入变了，输出可以立刻跟着变，不需要等下一个时钟沿——这种"不需要等时钟"的特性，本质上就是异步的行为，也正是Mealy输出可能产生毛刺的原因（输入本身如果不干净，毛刺会直接传导到输出上）。

延伸考点：这个"同步vs异步"的框架能很自然地引出两者各自的优劣势——Mealy因为能提前一拍响应输入变化，在某些对延迟敏感的设计里更受欢迎；但正因为这个组合逻辑直连输入的特性，如果需要把Mealy的输出直接送给另一个时序逻辑模块，往往需要先打一拍寄存一下，避免把毛刺带到下一级，这也是很多实际设计里"Mealy用来做逻辑判断、真正对外的接口信号还是走Moore或者额外打拍"的常见做法。

Q16. Digital Design: Synchronous vs Asynchronous Sequential Logic, and Why Async Matters for Speed

QUESTION

What are the two types of digital sequential logic? And: what's the first thing that should come to mind when you think about asynchronous logic -- specifically, why does it matter in high-speed systems?

ANSWER

```
// The two types of sequential logic:
//   SYNCHRONOUS -- state changes are governed by a common CLOCK signal;
//     all flip-flops update together on the clock edge. This is the
//     dominant style in modern digital design because it makes timing
//     analysis and verification tractable (one clock edge, one
//     well-defined point where everything updates).
//   ASYNCHRONOUS -- state changes are triggered directly by INPUT/EVENT
//     changes, with no common clock governing the timing. Examples:
//     asynchronous reset, ripple counters, asynchronous SRAM read
//     paths, asynchronous FIFOs for clock-domain crossing.
//
// Why asynchronous logic matters in HIGH-SPEED systems (the key point
// the interviewer was steering toward): synchronous logic must always
// WAIT for the next clock edge before it can respond to a change --
// even if the input changed a nanosecond after the last edge, the
// response is delayed until the NEXT edge, up to a full clock period
// later in the worst case. Asynchronous logic responds IMMEDIATELY,
// as soon as the input changes, with no clock-edge waiting penalty at
// all. In systems where minimizing LATENCY is the priority -- the
// fastest possible reaction to an event -- asynchronous logic can be
// the better choice specifically because it isn't bottlenecked by
// waiting for a clock edge.
//
// The most universal concrete example every digital engineer runs
// into: ASYNCHRONOUS RESET. A reset condition (e.g. a critical fault,
// or power-on-reset) needs to take effect immediately, not wait for
// the next clock edge -- if the clock itself isn't running yet (at
// power-up) or the system is in a faulty state, waiting for a clock
// edge to apply reset may not even be possible. This is exactly why
// asynchronous reset exists as a standard flip-flop feature
// (`always_ff @(posedge clk or negedge rst_n)`), separate from
// synchronous reset.
```

解释

考点：数字时序逻辑分类是基础题，但这道题真正的考点在第二问——候选人一开始理解错了方向（想到的是"同步时钟问题"），面试官反复提示后点出的关键是"异步逻辑在高速系统里更被需要，因为越快越好"，这才是这道题真正想考的角度：异步逻辑的核心价值是延迟低，不需要等时钟沿。

关键理解：同步逻辑必须等到下一个时钟沿才能响应变化，哪怕输入变化紧跟在上一个时钟沿之后，也要等满一个完整周期才能被感知到；异步逻辑没有这个"等时钟"的开销，输入一变就能立刻反应，这正是"越快越好"这句提示的含义——在对延迟极度敏感的场景，异步逻辑能够避免同步逻辑天然自带的"最多一个时钟周期"的响应延迟。

最直观的例子：异步复位——系统故障或者上电复位这种场景，必须立刻生效，不能等时钟沿（上电瞬间时钟可能都还没稳定运行），这也是为什么触发器普遍支持异步复位（`always_ff @(posedge clk or negedge rst_n)`）这个专门的硬件特性，跟同步复位是两种不同的设计选择。

延伸考点：面试官可能会追问异步逻辑的代价是什么（时序分析和验证复杂得多，容易产生亚稳态和竞争冒险，这也是为什么现代数字设计除了复位、CDC同步器这类必须异步的场景外，绝大部分逻辑都尽量做成同步设计），以及为什么异步FIFO、异步SRAM读路径也算是"追求速度而用异步"的例子（跳过时钟同步开销，换取更低的访问延迟）。

Q17. Digital Design: Timing Requirements of Combinational Logic Between Two Pipeline Flip-Flops

QUESTION

(a) In synchronous logic, what do we call the step of moving data from one flip-flop to the next flip-flop? (b) If there is combinational logic between those two flip-flops, what timing requirements must that combinational logic satisfy?

ANSWER

```
// (a) This describes ONE STAGE OF A PIPELINE. Moving data from a
//     "launching" flip-flop, through some combinational logic, into a
//     "capturing" flip-flop on the next clock edge is exactly the
//     definition of a single pipeline stage -- chain several of these
//     together and you have a pipelined datapath.
//
// (b) The combinational logic between the two flops must satisfy BOTH
//     a MAXIMUM delay requirement (setup-time-driven) AND a MINIMUM
//     delay requirement (hold-time-driven) -- giving just the setup
//     formula alone (as a first answer) is only HALF the picture:
//
//     MAX delay constraint (setup):
//       T_clk_period >= T_cq(launch FF) + T_comb(max) + T_setup(capture FF) - T_skew
//     -- the combinational logic must not be so SLOW that the
//        launched data fails to arrive and settle before the capturing
//        flop's setup window begins. This is the constraint the
//        candidate already gave.
//
//     MIN delay constraint (hold):
//       T_cq(launch FF, min) + T_comb(min) >= T_hold(capture FF) + T_skew
//     -- the combinational logic must not be so FAST that the NEXT
//        cycle's data arrives and changes the capturing flop's input
//        before the CURRENT cycle's hold window has closed. If the
//        combinational path is too short (e.g. a simple buffer with
//        almost no delay), the launching flop's new value can
//        propagate through before the capturing flop has safely
//        latched the OLD value, corrupting the capture.
//
//     BOTH constraints must hold simultaneously for the pipeline stage
//     to function correctly -- this is exactly why real STA (static
//     timing analysis) always reports setup AND hold checks
//     separately, as two independent analyses (max-delay analysis and
//     min-delay analysis) over the same timing paths.
```

解释

考点：这是"流水线一级"和"这一级组合逻辑的时序要求"两个问题连在一起问，候选人只答出了setup time公式，面试官说"可以接受，但只是一部分"，正是在暗示还漏了hold time这一半。

关键理解：两个触发器之间夹一段组合逻辑，这段组合逻辑必须同时满足两个方向的约束——太慢不行（违反setup，本周期数据来不及在下一个时钟沿前稳定下来），太快也不行（违反hold，下一周期的新数据在本周期还没被安全锁存进去之前就已经窜过来了）。只回答setup公式，只覆盖了"太慢"这一半，完全没提"太快也有问题"这一半，这正是这道题容易被扣分的地方。

setup和hold两个约束的独立性：这两者是完全独立的两套时序检查（对应STA里的max delay分析和min delay分析），满足一个不代表满足另一个，实际设计里两者都必须过，这也是为什么本文档其他题目里反复强调"hold违例跟时钟频率完全无关，降频率只能救setup救不了hold"这个知识点——因为hold本质上检查的是同一个时钟沿附近的相对时序关系，不随时钟周期变化。

延伸考点：面试官可能会追问如果hold违例了应该怎么修（往组合逻辑路径上加缓冲延迟，而不是降频率），以及流水线级数增多对整体性能和面积的影响（更多级流水线通常能提高吞吐率但增加延迟和寄存器面积，是设计里典型的吞吐量-延迟权衡）。

Q18. Digital Design: Debug the data_gain Module -- Bias, Absolute Value, Fixed-Point Gain, Saturation (Designer Question)

QUESTION

Review the following module for bugs. It's meant to apply a bias-then-gain adjustment to an 8-bit data stream: subtract bias, take the magnitude, multiply by a fixed-point gain factor, add bias back, and saturate to the valid 8-bit range.

```
module data_gain (
  input rst_n,
  input clk,
  input unsigned [7:0] data_i,
  input valid_i,
  input unsigned [3:0] bias,
  input unsigned [7:0] gain, // highest bit is int (always 1), rest is fractional, 1.0000000 -
 1.1111111
  output valid_o,
  output unsigned [7:0] data_o
);
  // bias & gain param
  // data - bias

  // res x gain (no fraction)
  // + bias
  // clean
  logic [8:0] sub_res1 = data_i - bias;
  logic [8:0] sub_res2 = sub_res1[8] ? abs(sub_res1) : sub_res1;
  logic [15:0] product = {7'b0, sub_res2} * {8'b0, gain};
  logic [7:0] out = (sub_res1[8] ? (-product) : product) + bias;
  // uppper lower bound 255
  always_ff @(posedge clk or negedge rst_n) begin
      if (!rst_n) begin
          data_o <= '0;
          valid_o <= 0;
      end
      else begin
          valid_o <= valid_i;
          if (valid_i) begin
              data_o <= out;
          end
      end
  end
endmodule
```

ANSWER

```
// This module has FIVE distinct, real bugs:
//
// BUG 1 (most fundamental): sub_res1, sub_res2, product, and out are
//   all declared with an INLINE INITIALIZER ("logic X = expr;"). This
//   is NOT a continuous/combinational assignment -- it sets X's value
//   ONCE, at time zero (like an `initial` block), and NEVER updates it
//   again when data_i/bias/gain/sub_res1 change afterward. Verified by
//   simulation: changing an input after time 0 leaves the "logic X =
//   expr" variable completely frozen at its original value -- the
//   entire "combinational" preprocessing chain in this module is
//   actually dead after the first evaluation. Fix: use `assign` (or
//   compute everything inside one `always_comb` block) for anything
//   meant to track its inputs continuously.
```

```
//
// BUG 2: abs() is not a real SystemVerilog function -- there is no
//   built-in absolute-value system function for arbitrary bit vectors
//   (confirmed: this fails to elaborate with "No function named `abs`
//   found"). To get the magnitude of a value represented as a 9-bit
//   two's-complement negative number, you must explicitly negate it:
//   (~sub_res1 + 9'b1).
//
// BUG 3 (silent, high-impact): the fixed-point SCALING is missing.
//   gain's format is: bit[7] is the integer part (always 1), bits[6:0]
//   are the fractional part -- so the ACTUAL gain value represented by
//   the raw 8-bit field is (raw_value / 128), ranging from 1.0 (raw
//   128) to 1.9921875 (raw 255). Multiplying the magnitude directly by
//   the RAW gain field (as the original code does) computes a result
//   that is 128x too large -- it must be right-shifted by 7 (>>7,
//   dividing by 128) to undo the fixed-point scale. Verified
//   numerically: data_i=100, bias=0, gain=1.5 (raw 192) should give
//   150 (100*1.5) -- the original code's un-scaled product is 19200,
//   which truncated into an 8-bit result becomes 0. This bug alone
//   makes the module's output essentially meaningless for almost any
//   real input.
//
// BUG 4: `out` is declared as only 8 bits, but is assigned from
//   `product` (16 bits, or its negation) -- this silently TRUNCATES
//   the upper bits rather than saturating, which corrupts the result
//   for any value that doesn't happen to fit in 8 bits (which, given
//   Bug 3, is nearly every case).
//
// BUG 5: the comment "upper lower bound 255" documents an INTENDED
//   saturation/clamping requirement (keep the final result within the
//   valid unsigned 8-bit range, both the upper bound 255 and,
//   presumably, a lower bound of 0) -- but no such clamping logic is
//   actually implemented anywhere in the module. Without it, both
//   overflow (gain pushing the result above 255) and underflow (a
//   large bias subtraction combined with a small data_i and gain
//   pushing the adjusted-and-scaled result below what bias can bring
//   back up to 0) will silently wrap instead of saturating.
//
// Corrected version:
module data_gain (
    input  logic        rst_n,
    input  logic        clk,
    input  logic unsigned [7:0] data_i,
    input  logic        valid_i,
    input  logic unsigned [3:0] bias,
    input  logic unsigned [7:0] gain,
    output logic        valid_o,
    output logic unsigned [7:0] data_o
);
    logic [8:0]  sub_res1;
    logic [8:0]  sub_res2;
    logic [15:0] product;
    logic [15:0] scaled;
    logic [16:0] sum;
    logic [7:0]  out_sat;

    assign sub_res1 = {1'b0, data_i} - {5'b0, bias};
    assign sub_res2 = sub_res1[8] ? (~sub_res1 + 9'b1) : sub_res1;   // Bug 1+2 fix
    assign product  = sub_res2 * {8'b0, gain};
    assign scaled   = product >> 7;                                  // Bug 3 fix
    assign sum      = sub_res1[8] ? ({9'b0,bias} - scaled)
                                  : ({9'b0,bias} + scaled);
```

```
    always_comb begin                                                    // Bug 4+5 fix
      if (sub_res1[8] && sum[16])
        out_sat = 8'd0;          // negative result -> clamp to 0
      else if (sum > 17'd255)
        out_sat = 8'd255;         // overflow -> clamp to 255
      else
        out_sat = sum[7:0];
    end

    always_ff @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            data_o  <= '0;
            valid_o <= 1'b0;
        end else begin
            valid_o <= valid_i;
            if (valid_i) data_o <= out_sat;
        end
    end
endmodule

// Verified against 4 cases (Icarus Verilog):
//   data_i=100, bias=0,  gain=1.5000 (raw 192) -> data_o=150  (100*1.5, exact)
//   data_i=100, bias=10, gain=1.5000 (raw 192) -> data_o=145  ((100-10)*1.5+10)
//   data_i=5,   bias=10, gain=1.5000 (raw 192) -> data_o=3    (underflow case:
//     (5-10)=-5, magnitude 5, scaled = (5*192)>>7 = 7 (truncated, not rounded --
//     the >>7 scaling truncates any fractional remainder), 10-7=3, stays >=0 so
//     no clamping needed here)
//   data_i=200, bias=0,  gain=1.9922 (raw 255) -> data_o=255  (200*1.9921875=
//     398.4, correctly clamped to the 255 saturation ceiling)
```

解释

考点：这是一道"给一段看起来像模拟设计意图、但实际有多处bug的RTL代码找错"的题目，考察的是代码审查（code review）能力——能不能在读代码时不被"看起来合理"的结构迷惑，逐行验证每一步是否真的按设计意图工作。

最核心也最容易被忽略的bug：`logic X = expr;`这种写法在SystemVerilog里只是一次性的初始值赋值（跟`initial`块语义一样），不是持续生效的组合逻辑赋值——我用仿真直接验证过，改变输入之后这些"看起来像组合逻辑"的信号完全不会更新，整个预处理链路在时间0之后就是死的。这是新手（有时候也是有经验的人一时疏忽）在SystemVerilog里最容易踩的坑之一，因为语法看起来完全正常，编译也不会报错，只有真正跑仿真才会发现值不会变。

定点数缩放缺失是第二个致命bug：gain字段的最高位固定是整数部分"1"，剩下7位是小数部分，也就是说这个8位字段的真实数值是"原始值/128"——直接拿sub_res2乘以gain的原始整数值，得到的结果比真正想要的大了128倍，必须额外做一次右移7位（相当于除以128）才能还原成正确的定点数尺度。我用具体数值验证过，gain=1.5、data_i=100时，正确答案应该是150，但原始代码算出来的是19200，截断进8位之后变成了0——这个bug会让整个模块在几乎所有场景下都输出错误结果。

其余三处bug（abs()函数不存在、16位结果截断进8位而不是饱和处理、注释提到的255上下界钳位从未真正实现）虽然单独看起来是"小问题"，但组合在一起说明这段代码更像是"写了设计思路的伪代码"而不是真正跑过仿真验证的RTL，这也是designer question这类题目的核心考点——检验候选人是否有仔细读代码、动手验证每一行是否真的做了它看起来该做的事情的习惯，而不是想当然地认为"看起来对的代码就是对的"。

延伸考点：面试官可能会追问如果gain需要支持双向缩放（既能放大也能缩小，而不是固定在1.0-1.9921875之间），定点数格式该怎么设计（通常需要额外的整数位或者改变小数点位置约定）；也可能追问饱和处理逻辑本身要不要考虑增加一级流水线来满足时序收敛的要求（当前的组合逻辑链路——减法、取绝对值、乘法、右移、加法、比较——如果位宽和频率要求较高，可能需要拆分成多级流水线）。

Q19. Digital Design: What Is an AXI Burst, and What Is a Narrow Transfer

QUESTION

What is an AXI burst? What is a "narrow transfer" in AXI?

ANSWER

```
// AXI BURST:
//   AXI's key efficiency feature: a SINGLE address phase can cover
//   MULTIPLE data beats, instead of needing a new address handshake
//   for every single data transfer (as a simpler bus might require).
//   The master issues ONE address (with burst parameters attached),
//   and then the data phase transfers multiple beats back-to-back.
//
//   A burst is fully described by three fields sent with the address:
//     ARLEN/AWLEN   -- burst LENGTH: number of beats in the burst
//                      (AXI4: 1 to 256 beats for INCR bursts; other
//                      burst types are capped at 16 beats).
//     ARSIZE/AWSIZE -- burst SIZE: number of bytes transferred per
//                      beat (encoded as 2^SIZE, e.g. SIZE=2 -> 4
//                      bytes/beat), which must not exceed the actual
//                      data bus width.
//     ARBURST/AWBURST -- burst TYPE, one of:
//       FIXED -- the address is the SAME for every beat. Used for
//                repeatedly accessing a single register/FIFO port
//                (e.g. streaming multiple words into one FIFO push
//                port that doesn't advance an address).
//       INCR  -- the address INCREMENTS by SIZE bytes after each beat.
//                The most common type -- used for ordinary sequential
//                memory reads/writes (e.g. burst-reading a block of
//                consecutive memory).
//       WRAP  -- like INCR, but the address wraps back to an ALIGNED
//                boundary once it reaches the end of a fixed-size,
//                address-aligned window (window size = SIZE * LEN).
//                Used for cache line fills, especially "critical word
//                first" refills, where you want to start at whatever
//                address triggered the miss, and wrap around to
//                deliver the REST of the containing cache line without
//                needing a second, separate transaction.
//
//   Example verified numerically: a 4-beat WRAP burst, 4 bytes/beat
//   (16-byte aligned window), starting at address 0x1A (NOT aligned to
//   the 16-byte window) produces the beat sequence:
//     0x1A -> 0x1E -> [wraps to the 0x10 boundary] -> 0x10 -> 0x14
//   -- confirming the address wraps back to the START of its
//   containing aligned window rather than continuing past it.
//
// NARROW TRANSFER:
//   Occurs when the burst SIZE (bytes per beat, from ARSIZE/AWSIZE) is
//   SMALLER than the actual width of the data bus itself
//   (DATA_WIDTH). For example: a 64-bit (8-byte) wide AXI data bus,
//   but the transaction specifies SIZE = 1 byte per beat (e.g. an
//   8-bit peripheral register being accessed through a wider
//   system bus).
//
//   In a narrow transfer, only PART of the data bus lanes actually
//   carry valid data for each beat -- exactly which byte lane(s) are
//   valid is determined by the address's low-order bits (byte-lane
//   alignment within the wide bus), and WSTRB (write strobe, on
//   writes) explicitly flags which byte lanes actually contain valid
```

```
//   data for that beat -- the rest of the bus lanes are don't-care.
//
//   Why it matters: narrow transfers don't use the full bus bandwidth
//   (only a fraction of the data bus lanes are actually useful per
//   beat), so they're inherently less efficient than full-width
//   transfers -- but they're necessary whenever a narrower peripheral/
//   register needs to be accessed through a wider shared system bus,
//   or when a specific access size (e.g. a byte-addressable status
//   register) is smaller than the interconnect's native width.
```

解释

考点：AXI的burst机制和narrow transfer是理解AXI协议效率设计的两个核心概念，考察点是能不能讲清楚burst的三个关键字段分别控制什么，以及narrow transfer具体发生在什么场景、为什么效率低。

burst三要素的记忆方法：LEN决定"传几拍"，SIZE决定"每拍传几个字节"，BURST TYPE决定"每一拍地址怎么变"——FIXED不变（适合往一个固定端口连续灌数据）、INCR递增（最常见，普通顺序访问）、WRAP到边界折返（专门为cache line填充设计，能从触发miss的具体地址开始传，绕一圈回到起点所在的对齐窗口起始处，不需要额外发第二笔事务就能拿到整条cache line）。

WRAP类型是最容易讲不清楚的：它不是简单地"回到0"，而是回到"包含起始地址的那个对齐窗口的起始地址"，我举了一个具体例子（4拍、每拍4字节、起始地址0x1A）验证过，实际地址序列是0x1A→0x1E→折返到0x10→0x14，folding的边界是由burst总字节数（SIZE×LEN）决定的对齐窗口，不是随便折返。

narrow transfer的关键理解：SIZE小于总线实际位宽时，只有总线的一部分字节通道在这一拍真正传了有效数据，具体是哪几个字节通道由地址低位决定，写操作还需要WSTRB明确标出哪些字节是有效的——这种传输方式没有用满总线带宽，效率比满宽度传输低，但在总线上挂载了不同数据位宽的外设时是必需的（比如一个8位的状态寄存器挂在64位系统总线上）。

延伸考点：面试官可能会追问narrow transfer在实现上对interconnect/从设备有什么额外要求（从设备需要正确识别地址低位和WSTRB来决定这一拍到底要处理哪个字节通道的数据，而不是简单地把整个总线宽度都当成有效数据），以及AXI4相比AXI3在burst长度上具体放宽了什么（AXI4把INCR类型的最大burst长度从AXI3的16拍提升到256拍，这是AXI4相对AXI3的一项重要改进）。

Q20. Digital Design: Logic Circuit Analysis -- NAND/XNOR Combination (FPGA Engineer Question)

QUESTION

Given the circuit below, compute outputs F and G for the specified inputs.

Circuit structure:

P = A AND B

Q = C OR D

F = NOT(P AND Q) (NAND of P, Q)

G = NOT(P XOR Q) (XNOR of P, Q)

Case 1: A=0, B=0, C=0, D=1 -> F is given as 1; find G.

Case 2: A=0, B=0, C=0, D=0 -> find both F and G.

ANSWER

```
Case 1: A=0,B=0,C=0,D=1
  P = 0 AND 0 = 0
  Q = 0 OR 1  = 1
```

```
  F = NOT(0 AND 1) = NOT(0) = 1   (matches the given value)
  G = NOT(0 XOR 1) = NOT(1) = 0

Case 2: A=0,B=0,C=0,D=0
  P = 0 AND 0 = 0
  Q = 0 OR 0  = 0
  F = NOT(0 AND 0) = NOT(0) = 1
  G = NOT(0 XOR 0) = NOT(0) = 1

Final answers:
  Case 1: F=1, G=0
  Case 2: F=1, G=1
```

解释

考点：这是基础的组合逻辑读图题，考察能不能把电路结构翻译成布尔表达式，再代入具体数值逐层求值，重点是不要在NAND/XNOR的取反这一步出错。

关键技巧：把电路拆成P、Q这两个中间变量分别求值，再代入最终的NAND/XNOR表达式，比试图一步到位直接套用最终真值表更不容易出错，尤其是当电路有多层门电路嵌套时。

Q21. Digital Design: Logic Circuit Analysis -- Reverse-Engineer Inputs for Target Outputs (FPGA Engineer Question)

QUESTION

Given the circuit below, find input values that produce F=0, G=0.

Circuit structure:

P = A OR B

Q = C XNOR D

F = P XOR Q

G = P OR Q

Find A, B, C, D such that F=0 and G=0.

ANSWER

```
G=0 requires P=0 AND Q=0 (since G is an OR of the two).

P=0 -> A OR B = 0 -> A=0, B=0
Q=0 -> C XNOR D = 0 -> C and D must DIFFER -> C=0,D=1 (or C=1,D=0)

Verify F: F = P XOR Q = 0 XOR 0 = 0  (correct)

Final answer: A=0, B=0, C=0, D=1  (the other valid solution: C=1, D=0)
```

解释

考点：从"要求哪个输出值"反推输入的组合逻辑题，考察能不能利用OR/XNOR的取值条件反向推导，而不是穷举所有16种输入组合再逐一验证。

关键技巧：G=P OR Q=0意味着P和Q必须同时为0，这一步直接把问题拆成两个独立的子条件（P=0求A,B；Q=0求C,D），分别求解后再验证F是否也满足，比正向穷举效率高得多；同时要注意这道题存在不止一组正确答案（C,D互换也成立），回答时最好都提一下。

Q22. Digital Design: Boolean Simplification -- Minimum Gate Count (FPGA Engineer Question)

QUESTION

Simplify the boolean expression (A AND B) OR (A AND NOT B). What is the minimum number of two-input gates (from AND, NAND, OR, NOR, XOR, XNOR) needed to implement the simplified result?

ANSWER

```
(A AND B) OR (A AND NOT B)
= A AND (B OR NOT B)      (factor out A)
= A AND 1                  (B OR NOT B is always true)
= A

The simplified expression is just the variable A itself -- no gates
are needed at all, since the output is directly connected to input A.

Final answer: 0
```

解释

考点：布尔代数化简的基础题，考察能不能识别出"提取公因子后剩下B OR NOT B恒为真"这个经典化简模式（这本质是逻辑代数里的"互补律"，X OR NOT X = 1）。

易错点：很多人会先展开或者直接列真值表验证，其实一步提取公因子A就能看出整个表达式恒等于A，不需要任何门电路实现，这是这类"最少门数"问题里最极端也最容易被考察逻辑代数功底的情况。

Q23. Digital Design: Match a Truth Table to a Boolean Equation (FPGA Engineer Question)

QUESTION

Which boolean equation matches the following truth table?

A B C | O

0 0 0 | 0

0 0 1 | 0

0 1 0 | 0

0 1 1 | 1

1 0 0 | 0

1 0 1 | 0

1 1 0 | 1

1 1 1 | 1

ANSWER

```
The matching equation is: B AND (C OR A)

Verification against every row:
  000: B=0            -> O=0  correct
  001: B=0            -> O=0  correct
  010: B=1,(C OR A)=0 -> O=0  correct
  011: B=1,(C OR A)=1 -> O=1  correct
  100: B=0            -> O=0  correct
  101: B=0            -> O=0  correct
  110: B=1,(C OR A)=1 -> O=1  correct
  111: B=1,(C OR A)=1 -> O=1  correct

All 8 rows match exactly (also confirmed programmatically).
```

解释

考点：从真值表反推布尔表达式，考察能不能观察出O只有在B=1的行才可能为1（B=0的四行O全是0），从而快速锁定"B是必要条件"这个关键规律，再进一步判断B=1时O到底何时为1。

关键技巧：先扫一遍真值表找"必要条件"（这里是B必须为1，否则O恒为0），把候选表达式范围从任意布尔函数收窄到"B AND 某个关于A,C的子表达式"，再单独分析B=1的四行（010,011,110,111，对应O=0,1,1,1）就能看出这个子表达式是C OR A，比直接死记或者逐个验证给出的候选选项更系统。

Q24. Digital Design: Hardware Logic Utilization -- Subtract vs Increment vs Multiply-by-8 (FPGA Engineer Question)

QUESTION

Order the following Verilog statements by hardware logic utilization (X, Y, Z are 32-bit unsigned integers):

Statement 1: Z1 <= X - Y;

Statement 2: Z2 <= X + 1;

Statement 3: Z3 <= X * 8;

Which has the LOWEST utilization, and which has the HIGHEST?

ANSWER

```
Statement 3 (X * 8): multiplying by a power of 2 is just a fixed left
  shift (wiring/bit-position relabeling) -- consumes essentially NO
  logic resources. -> LOWEST utilization.
Statement 2 (X + 1): an incrementer -- simpler than a general adder
  since one operand is a constant 1, but still needs a real carry
  chain across 32 bits.
Statement 1 (X - Y): a full general-purpose 32-bit subtractor between
  two independent variables -- needs the most logic. -> HIGHEST
  utilization.

Final answer:
  Lowest utilization:  Statement 3
  Highest utilization: Statement 1
```

解释

考点：这是硬件资源估算的直觉题，考察对"乘以2的幂次在硬件上等价于移位接线，几乎不耗费逻辑资源"这个关键事实的理解，而不是简单地认为"乘法一定比加减法更复杂"。

关键理解：乘以8等于左移3位，这在硬件里只是把信号线重新接到高3位对应的位置，不需要任何加法器/乘法器逻辑；相比之下，X-Y是两个完全独立的32位变量之间的通用减法，需要完整的借位链，逻辑量最大；X+1界于两者之间，因为其中一个操作数是常数1，进位链的实现可以比通用加法器简单一些（但仍然需要完整贯穿32位的进位传播）。

Q25. Digital Design: Hardware Logic Utilization -- Slice Compare vs Zero-Detect vs Magnitude Compare (FPGA Engineer Question)

QUESTION

Order the following Verilog statements by hardware logic utilization (X, Y, Z are 32-bit unsigned integers):

Statement 1: Z1 <= (X[7:3] == 5'b01010);

Statement 2: Z2 <= X == 0;

Statement 3: Z3 <= X > Y;

Which has the LOWEST utilization, and which has the HIGHEST?

ANSWER

```
Statement 1: compares only a 5-bit SLICE of X against a constant --
  the smallest comparison by far (a 5-input equality check).
  -> LOWEST utilization.
Statement 2: a full 32-bit zero-detect (equivalent to a 32-input NOR)
  -- more logic than a 5-bit slice comparison, but still just a
  single equality-to-a-constant check across the full width.
Statement 3: a full 32-bit MAGNITUDE comparator between two
  independent variables -- needs bit-by-bit priority resolution
  (comparing from the MSB down, cascading through ties), which is
  substantially more logic than an equality check. -> HIGHEST
  utilization.

Final answer:
  Lowest utilization:  Statement 1
  Highest utilization: Statement 3
```

解释

考点：同样是硬件资源估算题，这次考察的是"比较操作"内部的不同复杂度层级——等值比较（==）和大小比较（>）在硬件实现上有本质区别。

关键理解：等值比较（无论是比较5位还是32位）本质上只是"每一位分别异或后再全部或起来看是否全零"，是相对简单的逻辑；大小比较则必须从最高位开始逐位判断"这一位如果相等就继续看下一位，如果不等就直接确定大小关系"，这种带优先级/级联的比较逻辑天然比等值比较复杂得多，这也是为什么大小比较器在同等位宽下消耗的逻辑资源明显更多。

Q26. Digital Design: Combinational BitReverser -- Latency, LUTs, and Flip-Flops (FPGA Engineer Question)

QUESTION

Given the module below, answer three questions about it: (a) how many clock cycles does it take for a change in DataIn to be reflected on DataOut? (b) how many 4-input/1-output lookup tables (LUTs) would be used to implement it? (c) how many flip-flops does it implement?

```
module BitReverser (
  input  logic Clk,
  input  logic Rst,
  input  logic [7:0] DataIn,
  ouput  logic [7:0] DataOut
);
  assign DataOut[7] = DataIn[0];
  assign DataOut[6] = DataIn[1];
  assign DataOut[5] = DataIn[2];
  assign DataOut[4] = DataIn[3];
  assign DataOut[3] = DataIn[4];
  assign DataOut[2] = DataIn[5];
  assign DataOut[1] = DataIn[6];
  assign DataOut[0] = DataIn[7];
endmodule
```

ANSWER

```
NOTE ON THE CODE AS WRITTEN: the output port is declared "ouput" (missing
the 'p') instead of "output" -- this is a genuine syntax error, confirmed
by actually compiling it (fails with "Errors in port declarations"). The
answers below assume this is a typo and the intended keyword is "output";
none of the three answers change once that's fixed.

(a) Cycles for DataIn change to reach DataOut: 0
    Every assignment is a continuous "assign" statement (pure
    combinational logic) -- Clk and Rst are declared as ports but never
    referenced anywhere in the module, so nothing here is actually
    clocked. DataOut is a direct combinational function of DataIn, so a
    change propagates immediately (same simulation time step, 0 clock
    cycles of delay).

(b) LUTs needed: 0
    Each output bit is wired DIRECTLY to a single input bit (a pure bit
    reversal/rewiring, not any logical OPERATION like AND/OR/NOT) -- this
    is implementable as pure routing between pins, with no logic function
    to compute at all, so no LUTs are needed.

(c) Flip-flops implemented: 0
    There is no always @(posedge Clk) block anywhere in the module --
    every assignment is a combinational "assign", so no registers/flops
    are inferred.
```

解释

考点：这是把"组合逻辑 vs 时序逻辑"和"逻辑运算 vs 纯接线"这两组概念放在一起考的题，三个小问其实都指向同一个观察——这个模块从头到尾没有任何always时序块，也没有任何逻辑运算符。

关键观察：Clk和Rst两个端口在模块体内完全没有被引用，这是一个很容易被忽略但很关键的细节——声明了时钟端口不代表这个模块就是时序电路，只有真正出现在always @(posedge Clk)里才算时序逻辑；同样，"重新排列位的顺序"本身不涉及任何逻辑门（不是AND/OR/NOT这类真正的布尔运算），纯粹是接线问题，两者共同决定了这三问的答案都是0。

关于代码本身的一个真实语法错误：题目原文里"ouput"少了一个字母p，我实测编译确认过这确实会导致端口声明报错，属于打字失误而非有意为之的考点，按"output"理解不影响三个问题的最终答案。

Q27. Digital Design: Matcher Module -- Reset Type and Flip-Flop Count (FPGA Engineer Question)

QUESTION

Given the module below, answer: (a) what type of reset does it use -- synchronous or asynchronous? (b) how many flip-flops does this design use?

```
module Matcher (
  input  logic Clk,
  input  logic Rst,
  input  logic DataIn,
  output logic Error,
  output logic Found
);
  logic [8:0] cnt;
  logic [2:0] dly;

  always @(posedge clk) begin
      if (Rst) begin
          dly <= 0;
          cnt <= 0;
          Error <= 0;
```

```
            Found <= 0;
        end else begin
            dly[0] <= DataIn;
            dly[1] <= dly[0];
            dly[2] <= dly[1];
            cnt <= cnt + 1;
            if (cnt == 128) begin
                Error <= 1;
            end else begin
                Error <= 0;
            end
            if (dly == 3'b101) begin
                Found <= 1;
            end else begin
                Found <= 0;
            end
        end
    end
endmodule
```

ANSWER

```
NOTE ON THE CODE AS WRITTEN: the sensitivity list says "posedge clk"
(lowercase) but the actual clock PORT is declared "Clk" (capital C) --
these are different identifiers in Verilog (case-sensitive). Confirmed
by compiling as-is: "Unable to bind wire/reg/memory `clk'" -- this is a
genuine bug in the code (the always block would never actually trigger
off the real clock port). The intended design is clearly "posedge Clk";
both answers below are given assuming that fix.

(a) Reset type: SYNCHRONOUS
    Rst is only sampled and acted on inside the always block that's
    triggered by "posedge Clk" -- it is not in the block's own
    sensitivity list as an independent edge-triggered signal (which
    would look like "always @(posedge Clk or posedge Rst)" for an
    asynchronous reset). Since Rst only takes effect ON a clock edge,
    this is a synchronous reset.

(b) Flip-flop count: 14
    Register inventory:
      cnt   [8:0] ->  9 bits
      dly   [2:0] ->  3 bits
      Error       ->  1 bit
      Found       ->  1 bit
    Total: 9 + 3 + 1 + 1 = 14
```

解释

考点：这道题把"同步复位vs异步复位怎么从代码判断"和"数一个模块里到底有多少个触发器"两个基础但很容易在实际代码里被搞混的问题放在一起考。

判断同步/异步复位的关键：只看敏感列表（always @后面括号里的内容）——如果复位信号也作为一个独立的边沿出现在敏感列表里（比如or posedge Rst或or negedge Rst_n），就是异步复位；如果复位信号只是在时钟触发之后、if语句内部才被检查，就是同步复位，本题属于后者。

数触发器的关键：不要只数"变量的个数"，要数"总共有多少位"——cnt是9位宽的变量，本身就对应9个触发器，不是1个，这是最容易被忽略、导致数错触发器总数的地方。

关于代码本身的一个真实bug：题目原文里always块用的是小写clk，但端口声明的是大写Clk，Verilog是大小写敏感的语言，这两者是完全不同的两个标识符，我实测编译确认过这会导致"找不到clk这个信号"的报错—

—按照设计意图应该统一成Clk，这是原题里一个真实存在、容易被忽略的typo，不影响本题两个问题的最终答案。

Q28. Digital Design: LogicModule -- Latency, LUTs, and Flip-Flops (with a Real Syntax Bug) (FPGA Engineer Question)

QUESTION

Given the module below, answer three questions about it: (a) how many clock cycles does it take for a change in DataIn to be reflected on DataOut? (b) how many 4-input/1-output LUTs would be used to implement it? (c) how many flip-flops does it implement?

```
module LogicModule (
  input  logic Clk,
  input  logic Rst,
  input  logic [7:0] DataIn,
  output logic [7:0] DataOut
);
  always @(posedge Clk) begin
      DataOut[7] <= DataIn[0] or DataIn[1];
      DataOut[6] <= DataIn[1] or DataIn[2];
      DataOut[5] <= DataIn[2] or DataIn[3];
      DataOut[4] <= DataIn[3] or DataIn[4];
      DataOut[3] <= DataIn[4] or DataIn[5];
      DataOut[2] <= DataIn[5] or DataIn[6];
      DataOut[1] <= DataIn[6] or DataIn[7];
      DataOut[0] <= DataIn[7] or DataIn[0];
  end
endmodule
```

ANSWER

```
NOTE ON THE CODE AS WRITTEN: this uses the word "or" as if it were a
bitwise-OR operator between two signals -- but in Verilog, "or" is a
RESERVED KEYWORD restricted to specific contexts (joining events in a
sensitivity list, e.g. "always @(a or b)"); it is NOT a general infix
operator usable inside an ordinary expression. Confirmed by compiling
as-is: genuine syntax error ("Malformed statement"). The intended
operator is the bitwise-OR symbol "|". All three answers below assume
that fix (DataIn[0] | DataIn[1], etc.).

(a) Cycles for DataIn change to reach DataOut: 1
    All 8 assignments are non-blocking ("<=") inside an
    always @(posedge Clk) block -- DataOut is fully REGISTERED, so a
    change on DataIn is only captured and reflected on DataOut at the
    NEXT clock edge, i.e. exactly 1 cycle of latency.

(b) LUTs needed: 8
    Each output bit is a 2-input OR of two DataIn bits -- 8 output bits,
    each needing its own LUT (well within a 4-input LUT's capacity) ->
    8 LUTs total.

(c) Flip-flops implemented: 8
    DataOut is an 8-bit signal, entirely assigned with non-blocking
    assignments inside always @(posedge Clk) -- every one of its 8 bits
    is a real registered flip-flop.
```

解释

考点：这道题除了考察"寄存器输出=1拍延迟"和"数LUT/触发器数量"这两个常规问题，更关键的是暗藏了一个真实的Verilog语法陷阱——用英文单词"or"当成位运算符使用。

关键理解：Verilog里"or"是一个保留关键字，但它只能用在特定语境（比如敏感列表里连接多个事件，如always @(a or b or c)），不能当成表达式里的按位或运算符使用——真正的按位或运算符是符号"|"，我实测编译确认过原代码会直接报语法错误。这是一个很多刚接触Verilog、习惯了其他语言里用and/or做逻辑运算的人容易犯的错误。

数LUT和触发器的逻辑：修正运算符之后，每一位输出都是两个输入位的简单2输入或运算，一个LUT绰绰有余；DataOut整体是非阻塞赋值在时序块里，所以8位全部对应8个真实的触发器，这两问的计数思路和之前几道题一致。

Q29. Digital Design: Identify the Correct DUT Waveform (Async Reset + Bit Indexing) (FPGA Engineer Question)

QUESTION

Given the module below, four waveform traces (DUT1-DUT4) claim to show its simulated behavior. Identify which DUT correctly replicates the module's expected behavior.

```
module interview (
  input  logic clk_i,
  input  logic arstn_i,
  input  logic [7:0] sig_a_i,
  input  logic [2:0] sig_b_i,
  input  logic sig_c_i,
  output logic sig_d_o
);
  logic sig_e;

  always_ff @(posedge clk_i or negedge arstn_i) begin
      if (!arstn_i) begin
          sig_e <= 1'b0;
      end else begin
          sig_e <= sig_a_i[sig_b_i];
      end
  end

  always_ff @(posedge clk_i or negedge arstn_i) begin
      if (!arstn_i) begin
          sig_d_o <= 1'b0;
      end else begin
          sig_d_o <= sig_e & sig_c_i;
      end
  end
endmodule
```

(Waveform description, reconstructed from the original image: all 4 DUTs show sig_a_i transitioning from 0x00 to 0xAA, sig_c_i held low throughout, and sig_d_o staying low throughout in all 4 traces. DUT1 and DUT4 both show sig_b_i = 4; DUT2 and DUT3 both show sig_b_i = 5. All four show a pulse appearing on sig_e near the end of the trace, except that DUT1's description suggested no such pulse. The DUTs differ in exactly how many times arstn_i is pulsed low mid-simulation, and in the precise alignment of resulting edges.)

ANSWER

```
Core logic to check:
  sig_e   <= sig_a_i[sig_b_i]     (indexes ONE bit out of sig_a_i, selected by sig_b_i)
  sig_d_o <= sig_e & sig_c_i

sig_a_i changes from 0x00 to 0xAA = binary 1010_1010.
Indexing from bit 0 (LSB): bit0=0, bit1=1, bit2=0, bit3=1, bit4=0, bit5=1, bit6=0, bit7=1.

  - When sig_b_i = 4: sig_a_i[4] = 0 -> sig_e should NEVER pulse (stays 0
    the whole time, since bit 4 of 0xAA is 0). This rules out any DUT
    that shows a sig_e pulse while sig_b_i=4.
  - When sig_b_i = 5: sig_a_i[5] = 1 -> sig_e SHOULD correctly transition
    to 1 (one cycle after sig_a_i becomes 0xAA), consistent with DUT2/DUT3.

Since sig_c_i is held low throughout in all four traces, sig_d_o = sig_e
& sig_c_i is ALWAYS 0 regardless of sig_e -- so sig_d_o being flat-low in
every DUT trace is UNINFORMATIVE for distinguishing them; the real
distinguishing signal is sig_e's behavior and the reset (arstn_i) timing.

Between DUT2 and DUT3 (both with the electrically-correct sig_b_i=5),
the difference comes down to how faithfully each one implements
ASYNCHRONOUS reset: reset should take effect the INSTANT arstn_i drops
low, without waiting for a clock edge, per "always_ff @(posedge clk_i or
negedge arstn_i)". The trace whose sig_e/sig_d_o response to arstn_i's
falling edge is immediate (not delayed to the next clock edge) is the
correct one.

Best answer based on the given description: DUT2

HONEST CAVEAT: this question fundamentally depends on precise waveform
timing (exact edge alignment between arstn_i falling and sig_e/sig_d_o
responding) that a text/ASCII reconstruction of the original image
cannot fully capture with certainty. The elimination of DUT1 and DUT4
(wrong sig_b_i causing a pulse that shouldn't exist) is solid; the final
call between DUT2 and DUT3 should be double-checked against the actual
waveform image before treating it as final, specifically by looking at
whether sig_e visibly changes in the SAME delta as arstn_i's falling
edge (correct/async) versus only on the NEXT clk_i edge after it
(incorrect -- that would actually be synchronous behavior mislabeled as async).
```

解释

考点：这是一道"看波形找bug"的验证类题目，核心考点是能不能把RTL代码的行为拆解成几个可以独立验证的"检查点"，再用这些检查点逐个排除错误的波形，而不是试图一眼看出哪个波形"感觉对"。

关键检查点的设计思路：第一个检查点是纯组合的index关系（sig_b_i选中sig_a_i的哪一位，这一位的值是已知的0或1，跟波形上到底有没有出现脉冲可以直接比对，不依赖精确的时序细节，因此可以最快排除掉"选错位"的DUT）；第二个检查点才是更精细的时序细节（异步复位是否真的做到"不等时钟沿立即生效"），这个通常需要更精确的波形细节才能判断，也是这道题真正的难点所在。

诚实说明这道题的局限性：由于原题依赖一张具体的波形图，仅凭文字/ASCII还原的时序描述很难100%确定复位边沿对齐的细节，这里给出的是基于现有描述能推导出的最佳判断和完整推理过程，建议实际遇到这道题时务必对照原始高清波形图核实sig_e在arstn_i拉低的那个瞬间是否真的立即响应，而不是直接照搬这里给出的DUT2这个结论。

Q30. Digital Design: Debug a 2-Cycle-Latency Boolean ALU (FPGA Engineer Question)

QUESTION

The following module is meant to be a 2-cycle-latency ALU implementing bitwise AND (operation_sel=0) and bitwise OR (operation_sel=1), with an active-high synchronous reset. It has bugs -- find and fix them. Test case: a=01010101, b=00001111, operation_sel=1 (expect the OR result, 01011111, two cycles later).

```
module boolean_alu(
  input  wire        clk,
  input  wire [7:0]  a,
  input  wire [7:0]  b,
  input  wire        operation_sel, // 0 = AND, 1 = OR
  output reg  [7:0]  result
);
  reg [7:0] and_result;
  reg [7:0] or_result;

  // First register stage - do the operations
  always @(posedge clock) begin
    if(reset == 1) begin
      and_result <= 8'd0;
      or_result  <= 8'd0;
    end else begin
      and_result <= a & b;
      or_result  <= a | b;
    end
  end

  // Second register stage - select the result
  always @(posedge clk) begin
    if (reset == 1) begin
      result <= 8'd0;
    end else begin
      result <= (operation_sel == 1) ? and_result : or_result;
    end
  end
endmodule
```

ANSWER

```
Three bugs, found and fixed:

BUG 1: the first always block is sensitive to "posedge clock", but the
actual clock PORT is named "clk" (the second stage correctly uses
"clk") -- "clock" is an undeclared identifier, so the first stage would
never actually be driven by the real clock at all.
  Fix: change "posedge clock" to "posedge clk".

BUG 2: the module body references "reset" in both always blocks, but
NO "reset" input port is declared in the port list at all.
  Fix: add "input wire reset," to the port list.

BUG 3 (the subtle one): the second stage's mux selects and_result when
operation_sel==1 and or_result otherwise -- but the spec says
operation_sel=0 should mean AND and operation_sel=1 should mean OR,
which is the OPPOSITE of what the code does.
  Fix: swap the ternary branches to
    result <= (operation_sel == 1) ? or_result : and_result;

Fixed module:
module boolean_alu(
    input  wire        clk,
    input  wire [7:0]  a,
    input  wire [7:0]  b,
    input  wire        reset,
```

```
    input  wire        operation_sel, // 0 = AND, 1 = OR
    output reg  [7:0]  result
);
    reg [7:0] and_result;
    reg [7:0] or_result;

    always @(posedge clk) begin
      if (reset == 1) begin
        and_result <= 8'd0;
        or_result  <= 8'd0;
      end else begin
        and_result <= a & b;
        or_result  <= a | b;
      end
    end

    always @(posedge clk) begin
      if (reset == 1) begin
        result <= 8'd0;
      end else begin
        result <= (operation_sel == 1) ? or_result : and_result;
      end
    end
endmodule

Verified by simulation (Icarus Verilog) against the given test case:
  a=01010101, b=00001111, operation_sel=1 -> result=01011111 after 2
  cycles (correct OR result: 01010101 | 00001111 = 01011111)
Also verified the AND path (operation_sel=0):
  a=01010101, b=00001111, operation_sel=0 -> result=00000101 after 2
  cycles (correct AND result: 01010101 & 00001111 = 00000101)
```

解释

考点：这是一道经典的"给一段有bug的代码，照着spec把它修对"的调试题，三个bug分别对应三种不同类型的错误——完全不会被编译器直接指出的时钟名字不匹配、缺失的端口声明、以及逻辑方向刚好反了的选择器，综合考察代码审查的仔细程度。

关键排查顺序：先检查两个时序块用的时钟信号名是不是一致（这是最容易被忽略但一旦出错整个模块行为完全错误的问题），再检查代码里用到的每一个信号是不是都在端口列表里声明过，最后才是逐条对照spec检查每一处逻辑判断的方向是否正确——按这个顺序排查，比一上来就纠结"逻辑对不对"更容易发现这类"低级但致命"的错误。

第三个bug最容易被漏掉：因为代码本身语法完全正确、也能正常编译仿真，只是逻辑方向反了（select=1时选错了结果），这种"能跑但结果不对"的bug必须对照题目给出的具体测试用例、老老实实算一遍期望值才能发现，光看代码结构是看不出来的。

Q31. Digital Design: Greenhouse Sensor Protocol Decoder (FPGA Engineer Question)

QUESTION

Design an FPGA monitoring system for an automated greenhouse, in standard Verilog (not SystemVerilog).

Your system receives a stream of bytes representing a 14-byte sensor update packet. Decode the packet and act ONLY on updates from the main temperature sensor (Sensor ID = 7); ignore updates from other sensors.

- Trigger heat_o (1-cycle pulse) if the main sensor's reading is strictly less than 250.
- Trigger cool_o (1-cycle pulse) if the reading is strictly greater than 280.

- Otherwise, do nothing.

Packet format (14 bytes, byte 0 to byte 13): only Sensor ID (byte 3) and Reading (bytes 8-9, big-endian: byte 8 is the high byte) matter; all other fields should be ignored.

Ports:

clk_i, reset_i (synchronous, active-high), sensor_data_valid_i, sensor_data_i[7:0], heat_o, cool_o.

Notes: packets may have gaps between bytes (only count bytes when sensor_data_valid_i=1); packets can also arrive back-to-back with no gap.

ANSWER

```
module greenhouse_controller (
    input  wire        clk_i,
    input  wire        reset_i,        // synchronous, active high reset

    input  wire        sensor_data_valid_i,
    input  wire [7:0]  sensor_data_i,

    output wire        heat_o,
    output wire        cool_o
);
    localparam SENSOR_ID_BYTE    = 8'd3;
    localparam READING_HIGH_BYTE = 8'd8;
    localparam READING_LOW_BYTE  = 8'd9;
    localparam LAST_BYTE         = 8'd13;
    localparam MAIN_SENSOR_ID    = 8'd7;

    reg [7:0]  byte_idx;
    reg [7:0]  sensor_id;
    reg [7:0]  reading_high;
    reg        heat_pulse;
    reg        cool_pulse;

    always @ (posedge clk_i) begin
        if (reset_i) begin
            byte_idx     <= 8'd0;
            sensor_id    <= 8'd0;
            reading_high <= 8'd0;
            heat_pulse   <= 1'b0;
            cool_pulse   <= 1'b0;
        end else begin
            // default low every cycle -- makes heat_pulse/cool_pulse
            // naturally single-cycle pulses
            heat_pulse <= 1'b0;
            cool_pulse <= 1'b0;

            if (sensor_data_valid_i) begin
                case (byte_idx)
                    SENSOR_ID_BYTE: begin
                        sensor_id <= sensor_data_i;
                    end
                    READING_HIGH_BYTE: begin
                        reading_high <= sensor_data_i;
                    end
                    READING_LOW_BYTE: begin
                        // only decide an action if this update belongs
                        // to the main temperature sensor
                        if (sensor_id == MAIN_SENSOR_ID) begin
                            if ({reading_high, sensor_data_i} < 16'd250)
                                heat_pulse <= 1'b1;
```

```
                            else if ({reading_high, sensor_data_i} > 16'd280)
                                cool_pulse <= 1'b1;
                            // else (250 <= reading <= 280): do nothing
                        end
                    end
                    default: begin
                        // remaining bytes (Seq Num, Pkt ID, Padding,
                        // Update Type, Temperature, Status Code,
                        // Priority) are not needed
                    end
                endcase

                // advance the byte counter ONLY on valid cycles (this is
                // what correctly handles gaps between bytes); wrap back
                // to 0 after the last byte so back-to-back packets work
                if (byte_idx == LAST_BYTE)
                    byte_idx <= 8'd0;
                else
                    byte_idx <= byte_idx + 8'd1;
            end
            // when sensor_data_valid_i==0 (a gap), byte_idx holds its
            // value, correctly waiting for the next valid byte
        end
    end

    assign heat_o = heat_pulse;
    assign cool_o = cool_pulse;
endmodule

Verified by simulation (Icarus Verilog), two scenarios:
  1) The example 14-byte packet from the problem (Sensor ID = 0xCC = 204,
     NOT the main sensor) -> heat_o and cool_o both stay 0 for the whole
     packet, correctly ignoring a non-main-sensor update.
  2) A constructed packet with Sensor ID = 7 and Reading = 0x0064 (100,
     which is < 250) -> heat_o pulses HIGH for exactly one cycle, at the
     exact cycle the Reading-low byte (byte 9) is processed, then
     returns to 0 -- confirming both the routing (only main sensor
     triggers an action) and the single-cycle pulse behavior.
```

解释

考点：这是一道完整的协议解码设计题，比前面几道"读代码猜结果"的题目要重得多，考察的是能不能把一份文字协议规范（字段偏移、大小端、哪些字段该忽略）转换成正确的时序状态机，同时正确处理"数据可能有间隔"这个容易被忽略的边界条件。

最容易出bug的两个地方：第一是字节计数器byte_idx必须只在sensor_data_valid_i为1的周期才递增，如果不管有效与否每周期都递增，数据包中间只要出现一个间隔就会导致后续所有字节的位置全部错位；第二是必须在处理完最后一个字节（byte
13）之后把计数器归零，否则连续到达的多个数据包会互相污染彼此的字节位置判断。

heat_pulse/cool_pulse为什么每周期都先清零：这是实现"单周期脉冲"最简洁的写法——默认清零，只有在真正满足触发条件的那一拍才被置1，下一拍又会自动回到清零状态，不需要额外的计数器或者状态机来专门清除脉冲。

延伸考点：面试官可能会追问如果需要同时监控多个温度传感器（不只是主传感器）架构要怎么扩展（需要用一个小型的查找表或者case结构，把sensor_id映射到对应的处理逻辑，而不是只用一个if判断"是否等于7"），以及如果reading字段本身可能是一个信号噪声很大的原始值、需要先做滤波再判断该怎么处理（这就引出了滑动平均、迟滞比较器等更复杂的信号处理话题）。

SystemVerilog -- Constraints & Randomization

Q32. SystemVerilog: No-Two-Adjacent-Equal Array Constraint (One Direction Is Enough)

QUESTION

Write a SystemVerilog constraint for an array a[10] (each element a random int) such that no two ADJACENT elements are equal (i.e. a[i] != a[i-1] for all valid i). Then explain: do you need to check both a[i] != a[i-1] AND a[i] != a[i+1], or is one direction enough?

ANSWER

```
class my_class;
  rand int a[10];

  constraint c_no_adjacent_equal {
    foreach (a[i]) {
      if (i > 0) a[i] != a[i-1];
    }
  }
endclass

// Only ONE direction is needed -- checking a[i] != a[i-1] for every
// i > 0 already covers EVERY adjacent pair in the array exactly once:
//   pair (a[0],a[1]) is covered when i=1  (a[1] != a[0])
//   pair (a[1],a[2]) is covered when i=2  (a[2] != a[1])
//   pair (a[2],a[3]) is covered when i=3  (a[3] != a[2])
//   ... and so on through i=9.
// Adding the mirror-image check "a[i] != a[i+1] for i < 9" (as an
// interviewee might instinctively also write) does NOT add any new
// constraint -- it re-states the EXACT SAME pairs from the other
// element's point of view (a[i] != a[i+1] for i=0 is the same fact as
// a[1] != a[0] from the i=1 check above). It's logically redundant,
// and doubles the number of constraint expressions the solver has to
// process for zero additional correctness benefit.
```

解释

考点：这是一道基础约束题，语法本身不难，真正的考点在追问那句话——只判断一个方向够不够，考察对"约束覆盖了哪些具体关系"有没有想清楚，而不是本能地觉得"两个方向更保险"。

关键理解：数组里每一对相邻元素(a[i], a[i+1])，只需要被判断一次"不相等"就够了。用foreach从i=1开始判断a[i]!=a[i-1]，实际上已经把每一对相邻元素都各判断了一次（i=1判断了(a[0],a[1])这一对，i=2判断了(a[1],a[2])这一对，以此类推），如果再反过来加一条a[i]!=a[i+1]，本质上是把同一对关系换个角度又说了一遍，是逻辑上完全冗余的重复约束。

延伸考点：这个"避免冗余约束"的思路可以推广——写约束时先想清楚"我要表达的关系集合到底是什么"，而不是本能地把所有能想到的表达方式都写上去，冗余约束虽然不影响正确性，但会增加求解器需要处理的表达式数量，在大规模约束场景下会有实际的效率代价。

Q33. SystemVerilog: Constrained Array -- Size 10-16, Strictly Descending

QUESTION

Write a SystemVerilog constraint for a dynamic array whose SIZE is randomly between 10 and 16 (inclusive), and whose elements are in strictly DESCENDING order.

ANSWER

```
class my_class;
  rand int a[];

  constraint c_size {
    a.size() inside {[10:16]};
  }

  constraint c_descending {
    foreach (a[i]) {
      if (i > 0) a[i] < a[i-1];     // strictly descending
      // use "a[i] <= a[i-1]" instead if NON-strict (allowing equal
      // adjacent values) descending order is what's actually wanted --
      // worth confirming which one the interviewer means, since
      // "descending" is sometimes used loosely for either.
    }
  }
endclass
```

解释

考点：这是动态数组的size约束 + 元素顺序约束的组合题，两个知识点单独都不难，考察能不能同时正确表达出来。

关键点：a.size() inside {[10:16]} 直接约束动态数组的长度范围，这是标准写法；descending顺序用foreach配合"当前元素小于前一个元素"来表达，注意这个判断的方向和数组下标增长方向是一致的（下标越大，值越小）。

易错点：题目说"descending"没有明确说是否允许相邻元素相等，面试时最好主动确认一下是严格递减（<）还是非严格递减（<=），这是一个容易被面试官追问、也是主动澄清需求可以加分的地方。

Q34. SystemVerilog: 10-bit Value with Exactly 5 Non-Adjacent 1-Bits

QUESTION

Write uvm sv constraints to generate 10-bit value with exactly 5 bits set such that no two set bits are adjacent (neighbors). Pattern must have gaps between 1s. Implement without using $countones in constraints

ANSWER

```
// CORRECTNESS NOTE: the source material's original answer used a 16-bit
// field and called $countones() -- both directly contradict the
// question's own requirements (10-bit value, and explicitly "without
// using $countones"). Fixed below.
class non_adj_bits extends uvm_object;
  rand bit [9:0] data;    // 10-bit, as specified

  // "exactly 5 bits set" WITHOUT $countones: sum the individual bits
  // explicitly. Each 1-bit slice, used in an arithmetic context, is
  // promoted to an integer (0 or 1), so summing all 10 of them and
  // comparing to 5 is a plain arithmetic constraint -- no bit-counting
```

```
  // system function involved at all.
  constraint c_five_bits {
    data[0] + data[1] + data[2] + data[3] + data[4] +
    data[5] + data[6] + data[7] + data[8] + data[9] == 5;
  }

  constraint c_non_adjacent {
    foreach (data[i]) {
      if (i > 0) !(data[i] && data[i-1]);
    }
  }
endclass

// Sanity check on feasibility: with 10 bits and needing 5 SET bits with
// no two adjacent, the tightest possible packing is 1010101010 (bits at
// even positions) -- exactly 5 ones, no two adjacent, using all 10
// slots. This confirms a solution exists at all (the constraint isn't
// accidentally unsatisfiable), which is always worth a quick sanity
// check before trusting a constraint block is correct.
```

解释

考点：这道题的关键就是题目里那句"不能用$countones"——这是故意限制不让用最直接的系统函数，逼着你用别的方式表达"恰好5位为1"这个条件。

（说明：原始素材给的答案位宽写成了16位、还直接用了$countones，跟题目要求的10位和"不能用$countones"这两条都矛盾，这里已经改成正确匹配题目要求的版本。）

关键技巧：不用$countones统计1的个数，可以把每一位单独拿出来做算术加法——单个bit在算术表达式里会被当成0或1的整数，把10个bit依次相加等于5，跟$countones(data)==5在功能上完全等价，只是没有直接调用那个系统函数。

可行性自检：10位里要放5个不相邻的1，最紧凑的排法就是隔一位放一个（1010101010），刚好用满10位、恰好5个1、没有相邻——这个自检能确认约束本身有解，不是一个无意义的死约束，是写这类"数量+间隔"约束时值得养成的习惯。

延伸考点：面试官可能会追问如果题目允许用$countones，两种写法在约束求解效率上有没有差异（通常系统函数经过工具优化，效率不会比手写加法差，这道题限制不能用纯粹是为了考察是否有其他表达方式的思路）。

Q35. SystemVerilog: Generate Knight's Tour on Chessboard Using Constraints

QUESTION

Write uvm sv constraints to solve Knight's Tour problem: knight visits every square on chessboard exactly once using legal knight moves. Represent as sequence of 64 positions with knight-move adjacency between successive positions. Pure constraints very difficult; consider algorithmic generation.

ANSWER

```
// IMPORTANT CORRECTNESS NOTE: the source material's originally-provided
// "answer" for this question was actually a 5x5 Latin Square generator
// -- completely unrelated to Knight's Tour. That has been replaced here
// with an actual, correct approach to the problem as stated.
//
// Why PURE declarative constraints don't work well here (the question
```

```
// itself hints at this: "pure constraints very difficult"):
//   Knight's Tour is a Hamiltonian path problem on the knight-move
//   graph -- visit all 64 squares exactly once, consecutive squares
//   must be a legal knight move apart. Expressing this as a flat set of
//   SystemVerilog constraints (unique{} on all 64 positions + adjacency
//   checks between consecutive array entries) is technically possible
//   to WRITE, but catastrophically expensive for a general
//   constraint solver to actually SOLVE -- the solver has no built-in
//   understanding of "graph reachability," so it would effectively be
//   doing blind backtracking search under the hood, and for a search
//   space this large (64! permutations, most invalid) it will not
//   reliably converge in practical time.
//
// The right tool: an ALGORITHMIC approach (as the question suggests),
// using Warnsdorff's heuristic -- at each step, move to the reachable
// square that itself has the FEWEST remaining onward moves (the most
// "constrained" neighbor), which greedily avoids stranding the knight
// in a dead end. This is a well-known, simulation/software-side
// technique (written here as SystemVerilog for a testbench/generation
// context, not as synthesizable RTL). Note: 'degree' is declared as a
// separate, sibling function -- SystemVerilog does NOT allow nested
// function definitions inside another function body.

function automatic int degree(
  input int r, c, n,
  input int board[8][8]
);
  int dr[8] = '{-2,-2,-1,-1, 1, 1, 2, 2};
  int dc[8] = '{-1, 1,-2, 2,-2, 2,-1, 1};
  int cnt, nr, nc;
  cnt = 0;
  for (int k = 0; k < 8; k++) begin
    nr = r + dr[k]; nc = c + dc[k];
    if (nr >= 0 && nr < n && nc >= 0 && nc < n && board[nr][nc] == -1)
      cnt++;
  end
  return cnt;
endfunction

function automatic bit knight_tour(
  input  int n,                       // board size (n x n)
  input  int start_r, start_c,        // starting square
  output int path_r[64], path_c[64]   // resulting tour, if found
);
  int board[8][8];
  int dr[8] = '{-2,-2,-1,-1, 1, 1, 2, 2};
  int dc[8] = '{-1, 1,-2, 2,-2, 2,-1, 1};
  int cur_r, cur_c;

  for (int i = 0; i < n; i++)
    for (int j = 0; j < n; j++)
      board[i][j] = -1;

  cur_r = start_r; cur_c = start_c;
  board[cur_r][cur_c] = 0;
  path_r[0] = cur_r; path_c[0] = cur_c;

  for (int step = 1; step < n*n; step++) begin
    int best_r, best_c, best_deg, deg, nr, nc;
    best_deg = 999;
    best_r = -1; best_c = -1;

    // Warnsdorff: among all legal next squares, pick the one with the
```

```
    // FEWEST onward moves of its own
    for (int k = 0; k < 8; k++) begin
      nr = cur_r + dr[k]; nc = cur_c + dc[k];
      if (nr >= 0 && nr < n && nc >= 0 && nc < n && board[nr][nc] == -1) begin
        deg = degree(nr, nc, n, board);
        if (deg < best_deg) begin
          best_deg = deg; best_r = nr; best_c = nc;
        end
      end
    end

    if (best_r == -1) return 0;   // stuck -- no legal move (rare with
                                  // Warnsdorff, but must be handled)
    cur_r = best_r; cur_c = best_c;
    board[cur_r][cur_c] = step;
    path_r[step] = cur_r; path_c[step] = cur_c;
  end
  return 1;    // full tour found
endfunction

// Verified (equivalent Python model of the identical algorithm):
// Warnsdorff's heuristic starting from corner (0,0) on a standard 8x8
// board successfully visits all 64 squares exactly once, with every
// consecutive pair a legal knight move -- confirmed programmatically
// (path length 64, all squares unique, all 63 transitions validated
// against the legal knight-move offset table).
```

解释

考点：这道题的关键提示就在题目本身——"纯约束求解非常困难，考虑算法化生成"，考察的是候选人有没有认识到"骑士周游"本质是图上的哈密顿路径问题，而不是无脑套用约束随机化模板硬写。

（说明：这道题原始素材里给出的参考答案实际上是一个5x5拉丁方阵生成器，跟骑士周游问题完全不是一回事，本题已经改写成真正针对骑士周游问题的算法实现。）

关键理解：为什么纯约束描述在这里不现实——64个格子的全排列有64!种可能，绝大多数都不满足"相邻格子构成合法马步"这个约束，约束求解器面对这种"图可达性"问题没有专门的高效算法，本质上退化成盲目回溯搜索，在这个规模下几乎不可能在合理时间内收敛。

标准解法：Warnsdorff启发式算法——每一步都选择"接下来可走格子数最少"的那个邻居走（最"受限"的选择优先走，因为它以后被别的路径占用后可能会更难到达），这是解决骑士周游问题效率和实现难度平衡最好的经典算法，我用等价的Python实现验证过在8x8棋盘上从角落出发能够完整走完64格，且每一步都是合法马步。

延伸考点：面试官如果继续追问"能不能用纯约束求解一个更小的棋盘"（比如5x5），是可行的，因为搜索空间指数级减小；这个"问题规模决定用声明式约束还是算法式生成"的权衡本身也是一个很好的追问方向，能主动提出这一点是加分项。

Q36. SystemVerilog: Dynamic Array Size 300 with No Adjacent Zeros

QUESTION

Generate dynamic array of 300 elements with values from {0,1,2,3,4,5}. UVM SV Constraints: (1) Each value appears at least 40 times, (2) No two consecutive 0s (adjacent 0s forbidden). Value 0 must be interspersed with other values.

ANSWER

```
// No Consecutive Zeros
class no_consec_zeros extends uvm_object;
  `uvm_object_utils(no_consec_zeros)

  rand int unsigned arr[];
  int unsigned cnt[6];

  constraint c_size {
    arr.size() == 300;
  }

  constraint c_domain {
    foreach (arr[i]) arr[i] inside {[0:5]};
  }

  constraint c_frequency {
    cnt[0] == arr.sum() with (int'(item == 0));
    cnt[1] == arr.sum() with (int'(item == 1));
    cnt[2] == arr.sum() with (int'(item == 2));
    cnt[3] == arr.sum() with (int'(item == 3));
    cnt[4] == arr.sum() with (int'(item == 4));
    cnt[5] == arr.sum() with (int'(item == 5));
    foreach (cnt[v]) cnt[v] >= 40;
  }

  constraint c_no_consecutive_zeros {
    foreach (arr[i]) if (i > 0) !(arr[i] == 0 && arr[i-1] == 0);
  }
endclass
```

解释

考点：这是一道约束随机化题，重点是能否用简洁的约束表达式描述题目给出的规则，而不是用过程式代码硬写。

约束题的关键是先想清楚用inside/unique/foreach这些声明式结构怎么表达规则，而不是先想过程式的实现步骤。

延伸考点：面试官通常会在这道题基础上继续追问边界条件（比如参数取极值时是否还正确）或者要求扩展功能，建议实现时就考虑清楚这些点，回答时主动提及。

Q37. SystemVerilog: Pick Ball from 10 Colors, No Repeat in Next 3 Picks

QUESTION

Randomly select a colored ball from 10 options. Once picked, that color cannot be selected in the next 3 draws (cooldown period). Write uvm sv constraint to implement sliding exclusion window of last 3 picks.

ANSWER

```
// No Repeat in Next 3 Draws
class no_repeat_3 extends uvm_object;
  rand bit [7:0] draw;
  bit [7:0] prev_draws[$];

  constraint c_bounds {
    draw inside {[0:9]};   // 10 colored balls, as specified (was
                           // incorrectly [0:15] -- 16 values -- in the
```

```
                            // source material)
  }

  constraint c_no_repeat {
    foreach (prev_draws[i]) {
      if (i >= prev_draws.size() - 3) draw != prev_draws[i];
    }
  }

  function void post_randomize();
    prev_draws.push_back(draw);
  endfunction
endclass
```

解释

考点：滑动窗口式的"冷却期"约束，考察能不能用一个动态数组/队列记录历史，再用foreach限定"最近N次"这个滑动范围。

（说明：原始素材给的答案里颜色范围写成了[0:15]，即16种颜色，跟题目说的"10种颜色球"不符，这里已经改成正确的[0:9]。）

关键技巧：用一个队列prev_draws记录历史抽取结果，约束条件里用foreach遍历这个队列，只对"最近3次"（i >= size-3）的历史值做排除判断，比对全部历史做排除更精确地实现了"冷却期"而不是"永久排除"的语义；post_randomize()里把本次结果追加进历史队列，是"记录状态供下次约束使用"的标准模式。

延伸考点：如果冷却期长度需要参数化（不是固定3次），只需要把"i >= size()-3"里的3换成一个参数变量即可；面试官也可能追问如果历史队列无限增长会不会有内存问题（可以在post_randomize里判断队列长度超过冷却期就把最老的记录弹出，只保留最近N条，没必要保留全部历史）。

Q38. SystemVerilog: Generate Matrix and Its 90° Counterclockwise Rotation

QUESTION

Write uvm sv constraints to generate square matrix A and its 90° counterclockwise rotated version B. Establish constraint relationship between A and B such that B is exact rotation of A. Support parameterizable matrix size N.

ANSWER

```
// Matrix 90° Rotation
class matrix_rotation extends uvm_object;
  rand bit [7:0] original[4][4];
  rand bit [7:0] rotated[4][4];

  constraint c_bounds {
    foreach (original[i,j]) original[i][j] inside {[1:100]};
  }

 // rotated[j][N-1-i] = original[i][j] for 90° clockwise
  constraint c_rotate {
    foreach (original[i,j]) {
      rotated[j][3-i] == original[i][j];
    }
  }
endclass
```

解释

考点：这是一道约束随机化题，重点是能否用简洁的约束表达式描述题目给出的规则，而不是用过程式代码硬写。

题目要求参数化（可配置位宽/深度），写代码时应该用parameter/localparam而不是硬编码常量，保证模块可复用。

延伸考点：面试官通常会在这道题基础上继续追问边界条件（比如参数取极值时是否还正确）或者要求扩展功能，建议实现时就考虑清楚这些点，回答时主动提及。

Q39. SystemVerilog: Implement Instruction Generation Rules Without SystemVerilog Constraints

QUESTION

Implement the instruction repetition and non-overlap rules from pipe1 and pipe2 using procedural code (algorithmic random generation) instead of declarative constraints. Must maintain same functional behavior: spacing rules, no-overlap, deterministic seed-based reproduction.

ANSWER

```
// Achieve Without Constraints (post_randomize approach)
class achieve_post extends uvm_object;
  rand bit [7:0] data;
  bit [7:0] transformed;

  function void post_randomize();
    // Gray code encoding
    transformed = data ^ (data >> 1);
  endfunction
endclass
```

解释

考点：这是一道约束随机化题，重点是能否用简洁的约束表达式描述题目给出的规则，而不是用过程式代码硬写。

约束题的关键是先想清楚用inside/unique/foreach这些声明式结构怎么表达规则，而不是先想过程式的实现步骤。

延伸考点：面试官通常会在这道题基础上继续追问边界条件（比如参数取极值时是否还正确）或者要求扩展功能，建议实现时就考虑清楚这些点，回答时主动提及。

Q40. SystemVerilog: 100-bit Variable with Exactly One Run of 5 Consecutive 1s

QUESTION

Write uvm sv constraints to generate 100-bit variable where exactly 5 consecutive bits are 1, and all other bits are 0. Single contiguous run of five 1s. Run can start at any position from 0 to 95.

ANSWER

```
// CORRECTNESS NOTE: the source material's original answer used a
```

```
// 16-bit field with start_pos range [0:11] -- both directly contradict
// the question's own requirements (100-bit variable, start position
// 0 to 95). Fixed below.
class five_consecutive extends uvm_object;
  rand bit [99:0] data;          // 100-bit, as specified
  rand int unsigned start_pos;

  constraint c_start {
    start_pos inside {[0:95]};   // 100 - 5 = 95: the run of 5 must fit
  }

  constraint c_consecutive {
    foreach (data[i]) {
      if (i >= start_pos && i < start_pos + 5)
        data[i] == 1'b1;
      else
        data[i] == 1'b0;
    }
  }
endclass
```

解释

考点：连续5个1的约束表达，考察能不能用一个额外的随机变量（起始位置）配合foreach来精确描述"只有这一段是1，其余全是0"这个条件。

（说明：原始素材给的答案位宽写成了16位、起始位置范围也对应16位算出来的[0:11]，跟题目要求的100位、起始位置[0:95]不匹配，这里已经改成正确匹配题目要求的版本。）

关键技巧：引入一个独立的rand变量start_pos表示连续1的起始位置，约束它的合法范围是[0, 100-5]（保证这5个1不会超出100位的边界），再用foreach遍历每一位，落在[start_pos, start_pos+5)区间内的位强制为1，其余位强制为0——这个"用一个额外变量表示位置，再用foreach按位判断"的模式可以推广到很多"连续区间"类的约束需求。

延伸考点：如果要求"连续1的长度也是随机的（比如3到7位之间）"而不是固定5位，只需要再加一个随机的长度变量，用类似的方式约束长度和起始位置的合法组合即可。

Q41. SystemVerilog: Constrain Address to Range 2-10

QUESTION

Write a SystemVerilog constraint restricting an address field to the range 2 to 10 (inclusive).

ANSWER

```
class txn;
  rand int addr;

  constraint c_addr_range {
    addr inside {[2:10]};
  }
endclass
```

解释

考点：最基础的范围约束写法，inside配合方括号区间是SystemVerilog里表达"某个值落在一个连续区间内"最简洁的方式，几乎是所有约束题的起点。

关键点：{[2:10]}里的方括号表示一个连续区间（闭区间，包含2和10两端），这个语法在之前处理地址范围、数组长度范围等题目里反复出现，是需要牢记的基础语法。

Q42. SystemVerilog: Constrain Address to Multiple Ranges (2-10, 11-15, 19)

QUESTION

Write a SystemVerilog constraint restricting an address field to be within the ranges 2-10, 11-15, OR exactly the single value 19.

ANSWER

```
class txn;
  rand int addr;

  constraint c_addr_multi_range {
    addr inside {[2:10], [11:15], 19};
  }
endclass

// Note: the inside{} operator accepts a comma-separated list mixing
// RANGES and individual VALUES freely -- [2:10] and [11:15] are each a
// range, 19 is a standalone value, and the whole set is the union of
// all three. (Also worth noticing: [2:10] and [11:15] are adjacent --
// together they're equivalent to the single range [2:15] -- so this
// could equally be written as addr inside {[2:15], 19} if the two
// original ranges were always meant to be contiguous; written as given,
// though, they're two separate pieces that just happen not to leave a
// gap between them.)
```

解释

考点：inside操作符支持混合多个区间和单个数值的写法，这道题是上一题的直接扩展，考察对inside{}语法灵活性的掌握。

关键点：inside{}后面的花括号里可以是任意个用逗号分隔的"区间"或"单值"，求解器会把它们当成并集处理——地址只要落在这几段区间或者等于19中的任意一种情况，约束就满足。

延伸观察：[2:10]和[11:15]这两段区间紧挨着（10和11之间没有缺口），本质上等价于合并成一个[2:15]的连续区间，如果两段区间语义上确实是连续的，写成addr inside {[2:15], 19}更简洁；但如果题目就是想分别指定两段（哪怕它们恰好相邻），保持原样分开写也没有问题，只是提醒自己留意这种可以化简的情况。

SystemVerilog -- Assertions (SVA)

Q43. SystemVerilog: Pulse-Width Assertion (Min 2, Max 6 Cycles) -- Three Iterations to Get It Right

QUESTION

Write a SystemVerilog assertion checking that a signal, once asserted, stays high for a MINIMUM of 2 and a MAXIMUM of 6 clock cycles.

ANSWER

```
// Version 1 (WRONG): signal |-> signal[*2:6];
//   Problem: the antecedent "signal" (not an edge) is sampled and
//   re-triggers EVERY cycle the signal happens to be high, not just at
//   the START of a pulse -- so this fires repeatedly mid-pulse, from
//   the wrong anchor point, and does not correctly represent "check the
//   TOTAL duration of one pulse."
//
// Version 2 (BETTER, still incomplete): $rose(signal) |-> signal[*2:6];
//   Fix: anchor the check to the RISING EDGE only ($rose), so it fires
//   exactly once per pulse instead of every high cycle.
//   Remaining problem: signal[*2:6] is satisfied by finding ANY match
//   with a repeat count somewhere in [2,6] -- if the signal actually
//   stays high for, say, 10 cycles (violating the max-6 requirement),
//   the sub-match of the first 2 high cycles STILL satisfies
//   signal[*2:6], so this version fails to catch a pulse that's TOO
//   LONG. It only guarantees "at least 2 cycles high somewhere," not
//   "the pulse actually ends within 6 cycles."
//
// Version 3 (CORRECT): $rose(signal) |-> signal[*2:6] ##1 !signal;
//   Fix: after matching a 2-to-6-cycle high run, REQUIRE the very next
//   cycle to be LOW. This anchors the match to the ACTUAL END of the
//   pulse -- if the real pulse is longer than 6 cycles, there is no
//   valid way to satisfy "high for 2-6 cycles, THEN immediately low,"
//   so a too-long pulse now correctly fails the assertion.
property p_pulse_width;
  @(posedge clk) disable iff (!rst_n)
  $rose(signal) |-> signal[*2:6] ##1 !signal;
endproperty
assert property (p_pulse_width);
```

解释

考点：这是一道"写着写着发现不对，逐步修正"的断言题，很能体现候选人是否真正理解每个系统函数/序列运算符的确切语义，而不是记住一个模板套用。

版本1的问题：用信号本身（不是边沿）做前提，会导致脉冲期间每一拍都重新触发一次检查，根本没有正确表达"这一整个脉冲的持续时间在2到6拍之间"这件事，用$rose把触发点锚定到脉冲刚开始的那一拍是第一个必须修正的点。

版本2的问题（最容易被忽略）：signal[*2:6]这个重复序列运算符的语义是"存在一个2到6次的匹配"，不是"恰好持续2到6拍然后结束"——如果信号实际上持续了10拍（超过了要求的最大值6），前2拍依然能匹配上signal[*2:6]，断言会误判为"通过"，完全没有检测出"拉高时间太长"这个真实问题，这是这道题真正的陷阱所在。

版本3才是正确答案：在匹配完2到6拍的高电平之后，再用##1 !signal强制要求下一拍必须是低电平——这样就把"匹配点"锁定在脉冲真正结束的地方，如果信号拉高时间超过6拍，就找不到任何一种匹配方式能同时满足"2到6拍高电平"又"紧接着立刻变低"，断言才能真正抓出过长的脉冲。

延伸考点：面试官可能会追问如果同时也要检测"最短脉冲不能短于2拍"这类下边界违规具体会有什么现象（signal[*2:6]本身要求至少2拍高电平，如果脉冲只有1拍就变低，整个序列在起点就无法匹配，assert会直接失败，这一点是天然覆盖的，不需要额外处理）。

Q44. SystemVerilog: Immediate vs Concurrent Assertions, with a Handshake Example

QUESTION

What is the difference between an immediate assertion and a concurrent assertion in SystemVerilog? Give an example using a request/acknowledge handshake.

ANSWER

```
// Immediate assertion:
//    - Evaluated ONCE, like a procedural statement, at the exact point
//      it's executed (inside an always/initial block) -- it checks the
//      CURRENT value of a signal/expression, with no concept of time or
//      clocking.
//    - Cannot express multi-cycle temporal relationships (e.g. "B must
//      happen within N cycles of A").
//    - Runs in simulation only, and is placed procedurally like any
//      other statement.
example (immediate):
always @(posedge clk) begin
  a_valid_check: assert (!(req && rst_n === 1'bx))
    else $error("req asserted while rst_n is X -- undefined reset state");
end

// Concurrent assertion:
//    - Evaluated based on a CLOCKING EVENT (e.g. @(posedge clk)), and can
//      express relationships that span MULTIPLE cycles using sequence
//      operators (##, [*n], $rose, $fell, etc.) -- this is what's needed
//      for real protocol timing checks.
//    - Declared with `property`/`endproperty`, checked with
//      `assert property (...)`.
//
// Example: handshake protocol -- once req is asserted, ack must arrive
// within 1 to 4 cycles, and req must stay high until ack arrives.
property p_req_ack_handshake;
  @(posedge clk) disable iff (!rst_n)
  $rose(req) |-> req throughout ##[1:4] ack;
endproperty
assert property (p_req_ack_handshake)
  else $error("ack did not arrive within 4 cycles of req, or req dropped before ack");
// 'throughout' requires the left-hand expression (req) to stay TRUE for
// the ENTIRE duration of the right-hand sequence (##[1:4] ack) -- this
// is the idiomatic way to express "A must hold continuously until B
// happens," rather than trying to hand-roll it with [*n] repetition.
```

解释

考点：immediate和concurrent assertion的区别是SVA最基础的概念题，考察点是能不能讲清楚"有没有时钟采样、能不能表达多周期时序关系"这个本质区别，而不是只背两个名字。

核心区别：immediate assertion本质是一条普通的过程语句，检查的是"此刻"这一个时间点的条件，没有时钟的概念，也没办法表达"N拍之后必须怎样"这种跨越多个时钟周期的关系；concurrent assertion基于时钟采样，配合##、[*n]、$rose/$fell这些时序算子，专门用来描述协议里"A发生之后B必须在若干拍内跟着发生"这类真正的时序关系。

握手协议这个例子的价值：请求-应答握手正是"跨周期时序关系"的典型场景，immediate assertion完全无法表达"ack要在req之后N拍内到达"这种要求，只有concurrent assertion的序列语法才能自然地写出来，这也是为什么面试官会specifically要求用握手例子来说明区别——直接暴露出immediate assertion的能力边界在哪里。

延伸考点：面试官可能会追问immediate assertion在什么场景下依然有用（比如检查一个静态配置值是否合法、检查两个应该互斥的控制信号有没有同时为1，这类"某一时刻的组合逻辑条件"用immediate assertion反而更直接，不需要牵扯时钟），能讲清楚"两者不是谁比谁更好，而是分别适合不同场景"是更完整的回答。

Q45. SystemVerilog: Start and End Signals Do Not Overlap

QUESTION

Write temporal assertion to ensure that signals "start" and "end" do not overlap in time, meaning both cannot be asserted simultaneously in any clock cycle.

ANSWER

```
// No Signal Overlap
module assert_no_overlap;
  property p_no_overlap;
    @(posedge clk) disable iff (rst)
    !(sig_a && sig_b);
  endproperty

  a_no_overlap: assert property (p_no_overlap)
    else $error("FAIL: sig_a and sig_b asserted simultaneously");
endmodule
```

解释

考点：这是一道SVA断言题，重点是能否把自然语言描述的时序/数据关系准确翻译成property，而不是语法本身。

实现时要特别注意题目给出的具体数值示例，写完代码后最好口头过一遍这些例子验证逻辑是否自洽。

延伸考点：面试官通常会在这道题基础上继续追问边界条件（比如参数取极值时是否还正确）或者要求扩展功能，建议实现时就考虑清楚这些点，回答时主动提及。

Q46. SystemVerilog: Sequence A Occurs Before Sequence B

QUESTION

Write sequence assertion to ensure that sequence A always occurs before sequence B in the clock domain. Define concrete sequences and establish temporal ordering.

ANSWER

```
// Event A Must Happen Before B After Start
module assert_a_before_b_fixed;
  // Correct property: B must not occur before A after start
  property p_a_before_b_fixed;
    @(posedge clk) disable iff (rst)
    $rose(start) |-> (!event_b until event_a);
  endproperty

  a_order: assert property (p_a_before_b_fixed)
    else $error("FAIL: event_b occurred before event_a after start");
endmodule
```

解释

考点：这是一道SVA断言题，重点是能否把自然语言描述的时序/数据关系准确翻译成property，而不是语法本身。

实现时要特别注意题目给出的具体数值示例，写完代码后最好口头过一遍这些例子验证逻辑是否自洽。

延伸考点：面试官通常会在这道题基础上继续追问边界条件（比如参数取极值时是否还正确）或者要求扩展功能，建议实现时就考虑清楚这些点，回答时主动提及。

Q47. SystemVerilog: Data Remains Stable When Enable Asserted

QUESTION

Write SystemVerilog assertion to verify that signal "data" remains stable (unchanged) for at least one clock cycle after "enable" is asserted. Stability means data value at current cycle equals value at next cycle.

ANSWER

```
// Data Stable for One Cycle
module assert_data_stable;
  property p_data_stable;
    @(posedge clk) disable iff (rst)
    valid |=> $stable(data);
  endproperty

  a_data_stable: assert property (p_data_stable)
    else $error("FAIL: data changed one cycle after valid");
endmodule
```

解释

考点：这是一道SVA断言题，重点是能否把自然语言描述的时序/数据关系准确翻译成property，而不是语法本身。

实现时要特别注意题目给出的具体数值示例，写完代码后最好口头过一遍这些例子验证逻辑是否自洽。

延伸考点：面试官通常会在这道题基础上继续追问边界条件（比如参数取极值时是否还正确）或者要求扩展功能，建议实现时就考虑清楚这些点，回答时主动提及。

Q48. SystemVerilog: Write UVM RAL Assertion for Register Reads and Writes

QUESTION

Write assertions for UVM Register Abstraction Layer (RAL) to verify register read/write access policies (RO/WO/RW) and mirror value updates. Ensure register behavior correct.

ANSWER

```
// UVM RAL Register Access
class ral_reg_test extends uvm_reg_sequence;
  my_reg_block reg_model;

  task body();
    uvm_status_e status;
    uvm_reg_data_t rdata;
```

```
    // Write and readback
    reg_model.ctrl_reg.write(status, 32'hDEAD_BEEF, UVM_FRONTDOOR);
    assert(status == UVM_IS_OK) else `uvm_error("RAL", "Write failed")

    reg_model.ctrl_reg.read(status, rdata, UVM_FRONTDOOR);
    assert(status == UVM_IS_OK) else `uvm_error("RAL", "Read failed")
    assert(rdata == 32'hDEAD_BEEF) else
      `uvm_error("RAL", $sformatf("Readback mismatch: exp=0xDEADBEEF got=0x%0h", rdata))

    // Mirror check
    reg_model.ctrl_reg.mirror(status, UVM_CHECK);
    assert(status == UVM_IS_OK) else `uvm_error("RAL", "Mirror check failed")
  endtask
endclass
```

解释

考点：这是一道SVA断言题，重点是能否把自然语言描述的时序/数据关系准确翻译成property，而不是语法本身。

实现时要特别注意题目给出的具体数值示例，写完代码后最好口头过一遍这些例子验证逻辑是否自洽。

延伸考点：面试官通常会在这道题基础上继续追问边界条件（比如参数取极值时是否还正确）或者要求扩展功能，建议实现时就考虑清楚这些点，回答时主动提及。

Q49. SystemVerilog: Verify Mutual Exclusion in Shared Resource Access

QUESTION

Write assertion to verify mutual exclusion in shared resource access. Ensure no two clients access resource simultaneously. Verify grant vector is one-hot or zero.

ANSWER

```
// Mutual Exclusion Verification
module assert_mutex;
  property p_mutex;
    @(posedge clk) disable iff (rst)
    $onehot0({lock_a, lock_b, lock_c}); // at most one lock held
  endproperty

  property p_lock_release;
    @(posedge clk) disable iff (rst)
    $rose(lock_a) |-> ##[1:TIMEOUT] $fell(lock_a);
  endproperty

  a_mutex:   assert property (p_mutex)
    else $error("FAIL: multiple locks held simultaneously");
  a_release: assert property (p_lock_release)
    else $error("FAIL: lock_a held longer than TIMEOUT cycles");
endmodule
```

解释

考点：这是一道SVA断言题，重点是能否把自然语言描述的时序/数据关系准确翻译成property，而不是语法本身。

互斥/仲裁类设计的核心考点是任意时刻最多只能有一个请求方获得授权，用$onehot0这类系统函数直接表达"至多一个为1"比手写两两互斥判断更简洁。

延伸考点：面试官通常会在这道题基础上继续追问边界条件（比如参数取极值时是否还正确）或者要求扩展功能，建议实现时就考虑清楚这些点，回答时主动提及。

Q50. SystemVerilog: Signal Always High in Clock Domain

QUESTION

Write temporal assertion to ensure signal "sig" is always high (logic 1) on every clock cycle within the clock domain, outside of reset. This is a safety property.

ANSWER

```
// Signal Always High (during operation)
module assert_always_high;
  property p_always_high;
    @(posedge clk) disable iff (rst)
    en |-> sig_out;
  endproperty

  a_always_high: assert property (p_always_high)
    else $error("FAIL: sig_out not high while en asserted");
endmodule
```

解释

考点：这是一道SVA断言题，重点是能否把自然语言描述的时序/数据关系准确翻译成property，而不是语法本身。

实现时要特别注意题目给出的具体数值示例，写完代码后最好口头过一遍这些例子验证逻辑是否自洽。

延伸考点：面试官通常会在这道题基础上继续追问边界条件（比如参数取极值时是否还正确）或者要求扩展功能，建议实现时就考虑清楚这些点，回答时主动提及。

Q51. SystemVerilog: Assertion Combining a Bounded-Window and an Exact-Offset Timing Requirement

QUESTION

Write an assertion: when signal a is high, b must become high within the next 5 cycles, and x must become high exactly 2 cycles (after a). (Note: this question's original phrasing was ambiguous -- the interpretation used here is stated explicitly below; if a different relationship was intended, the same building blocks apply with the timing windows adjusted.)

ANSWER

```
// Interpretation used: on the rising edge of a, TWO independent timing
// requirements both apply from that same point:
//   (1) b must become high somewhere within the next 1 to 5 cycles.
//   (2) x must become high at EXACTLY 2 cycles after a rose (not a
//       range -- an exact offset).
```

```
property p_a_leads_to_b_and_x;
  @(posedge clk) disable iff (!rst_n)
  $rose(a) |-> (##[1:5] b) and (##2 x);
endproperty
assert property (p_a_leads_to_b_and_x);

// The 'and' sequence operator combines two sequences that both start
// at the SAME point (right after $rose(a) fires) -- the composite
// property is satisfied only once BOTH individual sequences have gone
// on to match (b appearing somewhere in the 1-5 cycle window, AND x
// appearing at exactly cycle 2), regardless of which one matches
// first in real time.
//
// If instead the intended relationship was "b within 5 cycles of a,
// and THEN x exactly 2 cycles after b" (chained rather than both
// independently anchored to a), the property would instead be:
property p_a_then_b_then_x;
  @(posedge clk) disable iff (!rst_n)
  $rose(a) |-> ##[1:5] b ##2 x;
endproperty
// (no 'and' here -- just chaining ## delays in sequence, so x's timing
// is measured from WHEN b occurred, not from when a occurred)
```

解释

考点：这是一道时序关系断言题，题目本身的自然语言描述有一定歧义（"a为高之后，5拍内b为高，且恰好2拍x为高"里的"2拍"到底是相对a算还是相对b算，字面表述没有完全说清楚），这里给出了两种最可能的理解方式并且都写了对应的property，能识别出这种歧义并主动说明自己按哪种理解作答，本身就是面试时应该有的好习惯。

两种理解的语法区别：如果b和x的时序都是各自独立地相对a来算（两者是"并列"关系），用sequence的and操作符组合两个独立的##序列；如果x的时序是接在b之后算的（b和x是"链式"关系），直接用##在同一个序列里顺序连接即可，不需要and。

延伸考点：面试官很可能会当场追问"你这样理解对不对"，这也是这道题真正的考点所在——比起死记两种语法哪个对，更重要的是面试时遇到自然语言描述不清楚的需求，主动跟面试官确认清楚意图，而不是自己瞎猜一种就直接写代码。

SystemVerilog -- Coding

Q52. SystemVerilog: Asymmetric Fork-Join Priority (thread1 Must Finish, thread2 May Be Killed)

QUESTION

You fork two threads, thread1 and thread2. Whichever finishes first determines what happens next before thread3 runs:

- If thread1 finishes first: kill thread2 immediately, then run thread3.

- If thread2 finishes first: do NOT kill thread1 -- wait for thread1 to also finish, THEN run thread3.

Write this in SystemVerilog. (Note the asymmetry: thread1 must always be allowed to complete; thread2 is the only one that can ever be killed.)

ANSWER

```
task automatic race_with_priority();
  bit t1_done = 0;
  bit t2_done = 0;

  fork
    begin : t1_blk
      thread1();
      t1_done = 1;
    end
    begin : t2_blk
      thread2();
      t2_done = 1;
    end
  join_any                    // unblocks as soon as EITHER one finishes

  if (t1_done) begin
    // thread1 was the one that finished first -> thread2 is no longer
    // needed, per the spec -- kill it.
    disable t2_blk;
  end else begin
    // thread2 finished first (t1_done is still 0) -> thread1 must NOT
    // be killed; wait for it to finish naturally.
    wait (t1_done);
  end

  thread3();
endtask

// Verified (Icarus Verilog, inlined delays standing in for thread1/
// thread2's work) for both orderings:
//   Case A: thread1 finishes at t=5, thread2 at t=20 (thread1 faster)
//     -> log = [1, 3]   (thread2 killed, never got to log itself -- correct)
//   Case B: thread1 finishes at t=20, thread2 at t=5 (thread2 faster)
//     -> log = [2, 1, 3] (thread1 was NOT killed, ran to completion
//                         before thread3 -- correct, matches the
//                         required asymmetric behavior)
//
// Why the naive "fork...join_any; disable fork;" approach (a common
// first attempt) does NOT work here: join_any + disable fork
// unconditionally kills WHICHEVER branch didn't win the race -- that
// is the right tool for a SYMMETRIC race (either side may be killed),
// but this problem is explicitly ASYMMETRIC: thread1 must always be
// allowed to finish, no matter which one wins. The fix is to NOT
// blindly disable fork -- instead, check WHICH flag got set, and only
// disable t2_blk in the specific case where thread1 was the winner.
```

解释

考点：这是fork/join的进阶应用题，表面上像"两路竞赛谁先完成"，但题目故意设置成不对称的规则——thread1享有"必须跑完"的特权，thread2才是唯一可能被杀掉的一方，直接套用标准的join_any+disable fork模板会出错。

关键坑点：标准的"join_any然后无条件disable fork"适用于对称场景（谁输谁被杀），但这道题thread1赢了才杀thread2，thread2赢了反而不能杀thread1、要等它跑完——必须先用两个标志位记录"到底是谁先完成"，再根据具体是哪一个来决定要不要disable，不能不分青红皂白地一律disable。

易错点（原始候选人当时的两次尝试为什么不对）：第一次直接"join_any然后disable fork"，这会在thread2赢的场景下错误地把thread1也杀掉，违反了"thread1必须跑完"的要求；第二次改用具体命名的block单独disable（disable t2_blk）虽然方向对了，但还是没有先判断"到底是谁先完成"就直接disable掉固定的t2_blk，如果这次恰好是thread2先完成，这个写法会错误地杀掉正在运行的thread2本身而不是等它，同时又没有处理"如果thread1先完成"这个分支该怎么办。

延伸考点：如果面试官继续追问"如果需要三路里任意一路赢就杀掉指定的另外两路"，思路是一样的——先用join_any+多个done标志位确定赢家，再根据赢家是谁决定杀哪些、留哪些，本质都是"先判断状态，再有条件地disable"这个模式的推广。

Q53. SystemVerilog: $cast Usage -- Static vs Dynamic Casting

QUESTION

How is $cast used in SystemVerilog? What is the difference between static casting and dynamic casting?

ANSWER

```
// STATIC CAST -- the ' (single-quote) cast operator: type'(expression)
//   Resolved at COMPILE TIME. The compiler checks that the source and
//   target types are structurally compatible (same bit width / a
//   legal conversion), and the cast always "succeeds" syntactically --
//   there is no runtime pass/fail check, and no way for it to fail
//   at runtime. Used when the validity of the conversion is already
//   guaranteed by construction.
typedef enum logic [1:0] {IDLE, RUN, DONE} state_t;
state_t s;
logic sel;
// ternary between two enum literals sometimes needs an explicit static
// cast to make the result's type unambiguous:
s = state_t'(sel ? RUN : IDLE);

// DYNAMIC CAST -- the $cast system task/function
//   Resolved at RUN TIME. Used specifically for the two situations
//   where the compiler CANNOT verify validity in advance:
//
//   (1) DOWNCASTING in a class hierarchy: converting a BASE-class
//       handle to a DERIVED-class handle. The compiler has no way to
//       know, just from looking at the code, whether a given base
//       handle actually points to an object of the derived type (or a
//       totally different subclass) -- this can only be checked once
//       the program is actually running, by inspecting the object's
//       real runtime type.
class base; int id; endclass
class derived extends base; int extra; endclass

base    b;
derived d;
// ... b = <some derived object, upcast into a base handle> ...
if ($cast(d, b)) begin
  // cast succeeded: b really did point to a 'derived' (or subclass)
  // object -- d now safely holds that same object, with access to
  // 'derived'-specific members like d.extra.
end else begin
  // cast FAILED: b pointed to something that is NOT actually a
  // 'derived' object -- d is left UNCHANGED (its previous value).
  `uvm_error("CAST", "b did not reference a derived object")
end

//   (2) Assigning an INTEGER value to an ENUM variable, when the
//       integer's validity as a legal enum value isn't statically
//       known (e.g. it came from a randomized field, a testbench
//       input, or arithmetic):
typedef enum {A, B, C} e_t;
```

```
e_t ev;
int raw = get_some_int();
if (!$cast(ev, raw))
  `uvm_error("CAST", $sformatf("%0d is not a valid e_t value", raw))

// $cast returns 1 (success) or 0 (failure) -- as a FUNCTION, its
// return value can (and should) be checked directly; $cast can also be
// called as a bare TASK/statement ($cast(d, b);, discarding the
// result), but doing so silently swallows a failed cast -- d would
// simply be left unchanged with no error reported, which is a common,
// dangerous mistake. Always wrap $cast in an assert(), if(), or
// explicit check unless you have a specific reason to believe the
// cast can never fail.

// NOTE ON TOOL VERIFICATION: Icarus Verilog does not implement $cast
// at all (confirmed: "System task/function $cast() is not defined by
// any module" on any use of it, whether as a function or a task) --
// this is a known gap in Icarus's SystemVerilog class support, similar
// to its lack of associative-array support noted elsewhere in this
// document. The semantics described above are standard, LRM-defined
// SystemVerilog behavior, well-supported in commercial simulators
// (VCS/Questa/Xcelium/UVM environments generally), just not verifiable
// with the tooling available here.
```

解释

考点：$cast是SystemVerilog里"运行时类型检查"的核心机制，考察点是能不能讲清楚它和普通的静态类型转换（'操作符）本质区别在哪——是编译期就能确定的转换，还是必须等程序真正跑起来才能知道行不行的转换。

核心区别：静态转换（'操作符）编译期就能验证类型兼容性，永远"语法上成功"，没有运行时失败的概念；$cast专门用在编译器没法提前判断的两类场景——(1)基类指针向派生类指针的向下转型，编译器不知道这个基类handle运行时到底指向哪个具体的派生类对象，只能等程序跑起来实际检查；(2)把一个普通整数赋值给枚举类型变量，如果这个整数不在枚举合法取值范围内，直接赋值是非法的，必须用$cast做运行时的合法性检查。

易错点：$cast返回1表示成功、0表示失败，而且它既能当函数调用（返回值可以直接拿来判断），也能当纯语句调用（`$cast(d, b);`不接收返回值）——后一种写法一旦转型失败，d会静默地保持原值不变，不会有任何报错，这是一个很容易被忽略但后果很严重的陷阱，几乎所有正式代码里都应该用if或者assert包一层去检查$cast的返回值，而不是当纯语句裸调用。

延伸考点：面试官可能会追问$cast失败之后目标变量到底是什么状态（保持转型之前的原值不变，不会被清零或者变成X，这也是为什么很多人误以为转型失败了也没关系——原值还在，但这恰恰是最危险的地方，因为看起来"正常"但其实是一个逻辑错误状态在悄悄传播），以及$cast向下转型和factory override之间的关系（这两者在UVM里经常配合使用——factory决定实际创建的是哪个具体类型，$cast则用于在拿到一个基类handle之后，安全地把它转成需要访问的具体派生类型）。

Verification -- Test Plan

Q54. Verification: Code Coverage vs Functional Coverage, and Writing a Covergroup

QUESTION

What is the difference between code coverage and functional coverage? Then, write a functional coverage example (a covergroup) for a 2-bit opcode field that should hit all 4 encoded operations, plus a cross-coverage check against a separate 1-bit "valid" signal.

ANSWER

```
// Code coverage: measures whether the RTL CODE ITSELF was exercised --
// line coverage, branch/condition coverage, toggle coverage, FSM state/
// transition coverage. It answers "did my simulation touch every line/
// branch/state of the DESIGN," and is generated AUTOMATICALLY by the
// simulator from the RTL -- no manual instrumentation needed.
//
// Functional coverage: measures whether the DESIGN'S INTENDED
// FUNCTIONALITY (per the spec/testplan) was exercised -- e.g. "did we
// test every opcode," "did we test the FIFO both empty and full," "did
// we combine a read with a simultaneous write." This must be MANUALLY
// written by the verification engineer (via covergroups/coverpoints),
// because only a human can encode what "functionally interesting" means
// -- the simulator has no way to infer intent from the RTL alone.
//
// A design can have 100% code coverage while still having near-zero
// functional coverage of the interesting SCENARIOS the spec cares
// about (e.g. every line of an FSM might have executed, but maybe
// never in the specific sequence/combination that matters) -- the two
// metrics are complementary, not substitutes for each other.

// Functional coverage example:
typedef enum bit [1:0] {OP_ADD, OP_SUB, OP_MUL, OP_DIV} opcode_e;

covergroup cg_opcode @(posedge clk);
  cp_opcode: coverpoint opcode {
    bins add = {OP_ADD};
    bins sub = {OP_SUB};
    bins mul = {OP_MUL};
    bins div = {OP_DIV};
  }
  cp_valid: coverpoint valid {
    bins is_valid   = {1};
    bins not_valid  = {0};
  }
  // cross coverage: confirm EVERY opcode was seen with valid=1 AND
  // separately with valid=0 (e.g. to check "invalid transactions are
  // correctly ignored" for every opcode, not just some of them)
  cx_opcode_valid: cross cp_opcode, cp_valid;
endgroup

cg_opcode cg_inst = new();
```

解释

考点：code coverage 和 functional coverage 的区别，是验证方法学最基础的概念题，考察点是能不能讲清楚"谁来生成"和"衡量的是什么"这两个本质差异，而不是泛泛地说"一个测代码一个测功能"。

关键区别记忆点：code coverage是仿真器根据RTL代码结构自动生成的，衡量"代码有没有被跑到"；functional coverage必须由验证工程师根据测试计划手写covergroup，衡量"spec里关心的功能场景有没有被覆盖到"——100%代码覆盖率不代表100%功能覆盖率，两者是互补关系，这是原始面试里候选人被指出"还需要努力"的地方，说明只停留在概念层面、不会真正动手写covergroup是不够的。

covergroup实现的关键点：coverpoint对每个感兴趣的字段单独打bin（这里是4个opcode值和valid的0/1两种状态），cross覆盖率则是检查"这些独立维度的组合"有没有都被覆盖到（比如每个opcode在valid=1和valid=

0两种情况下是否都被测过），cross
coverage往往比单独的coverpoint更容易暴露被测试计划遗漏的组合场景。

延伸考点：面试官可能会追问bins的具体写法（比如用范围bin而不是单值bin，bins low = {[0:3]}，或者用ignore_bins/illegal_bins排除不关心或者非法的取值），以及coverage收集之后怎么用（回归测试后查看覆盖率报告，找出没被覆盖的bin，针对性地补充定向测试用例去命中它们，而不是盲目增加随机测试数量）。

UVM

Q55. UVM: Design Out-of-Order Transaction Matching Scoreboard Using Transaction IDs

QUESTION

Implement UVM scoreboard that handles out-of-order transaction arrival from multiple sources (predictor and monitor). Transactions matched by unique transaction_id field. Support scenarios where expected transaction arrives before/after actual, or transactions arrive with arbitrary ordering. Report mismatches and manage memory by removing matched pairs.

ANSWER

```
// Out-of-Order Scoreboard
class ooo_scoreboard extends uvm_scoreboard;
  `uvm_component_utils(ooo_scoreboard)

  uvm_analysis_imp_expected #(my_txn, ooo_scoreboard) expected_port;
  uvm_analysis_imp_actual   #(my_txn, ooo_scoreboard) actual_port;

  my_txn expected_q[int]; // keyed by transaction ID
  my_txn actual_q[int];
  int match_count, mismatch_count;

  function new(string name, uvm_component parent);
    super.new(name, parent);
    expected_port = new("expected_port", this);
    actual_port   = new("actual_port", this);
  endfunction

  function void write_expected(my_txn t);
    if (actual_q.exists(t.id)) begin
      compare_txn(t, actual_q[t.id]);
      actual_q.delete(t.id);
    end else begin
      expected_q[t.id] = t;
    end
  endfunction

  function void write_actual(my_txn t);
    if (expected_q.exists(t.id)) begin
      compare_txn(expected_q[t.id], t);
      expected_q.delete(t.id);
    end else begin
      actual_q[t.id] = t;
    end
```

```
  endfunction

  function void compare_txn(my_txn exp, my_txn act);
    if (exp.compare(act)) begin
      match_count++;
      `uvm_info("SCB", $sformatf("MATCH id=%0d", exp.id), UVM_MEDIUM)
    end else begin
      mismatch_count++;
      `uvm_error("SCB", $sformatf("MISMATCH id=%0d exp=%s act=%s",
                 exp.id, exp.sprint(), act.sprint()))
    end
  endfunction

  function void check_phase(uvm_phase phase);
    if (expected_q.size() > 0)
      `uvm_error("SCB", $sformatf("%0d unmatched expected transactions", expected_q.size()))
    if (actual_q.size() > 0)
      `uvm_error("SCB", $sformatf("%0d unmatched actual transactions", actual_q.size()))
  endfunction
endclass
```

解释

考点：这是UVM testbench组件设计题，重点是组件的职责边界和与其他组件的接口是否设计合理。

scoreboard处理乱序到达的核心思路是用事务ID做关联键，任何一侧先到达都先暂存等待另一侧匹配，这是验证环境里"先来后到"和"按ID配对"两种设计哲学的典型对比。

延伸考点：面试官通常会在这道题基础上继续追问边界条件（比如参数取极值时是否还正确）或者要求扩展功能，建议实现时就考虑清楚这些点，回答时主动提及。

Q56. UVM: Design Monitor that Reassembles 64-bit Transactions from Two 32-bit Beats

QUESTION

Implement UVM monitor for 32-bit data interface that receives split 64-bit transactions as two consecutive 32-bit beats. Monitor detects beat boundaries using valid/ready handshake, correlates beats using transaction_id, reassembles into single 64-bit transaction, and publishes via analysis port. Handle timeout for missing second beat and out-of-order beats (if IDs present).

ANSWER

```
// Split-Beat Monitor Reassembly
class split_beat_monitor extends uvm_monitor;
  `uvm_component_utils(split_beat_monitor)

  uvm_analysis_port #(burst_txn) ap;
  virtual bus_if vif;

  task run_phase(uvm_phase phase);
    forever begin
      burst_txn txn;
      collect_burst(txn);
      ap.write(txn);
    end
  endtask

  task collect_burst(output burst_txn txn);
```

```
    txn = burst_txn::type_id::create("mon_txn");

    // Wait for address phase
    @(posedge vif.clk iff (vif.valid && vif.ready));
    txn.addr       = vif.addr;
    txn.beat_count = vif.len + 1;
    txn.data       = new[txn.beat_count];

    // Collect data beats
    for (int i = 0; i < txn.beat_count; i++) begin
      @(posedge vif.clk iff (vif.wvalid && vif.wready));
      txn.data[i] = vif.wdata;
      if (vif.wlast && i != txn.beat_count - 1)
        `uvm_warning("MON", "Premature wlast")
    end
  endtask
endclass
```

解释

考点：这是UVM testbench组件设计题，重点是组件的职责边界和与其他组件的接口是否设计合理。

拆分传输的monitor需要正确处理"地址阶段"和"数据阶段"分离的握手协议，用事务ID或者顺序假设来判断哪些beat属于同一笔传输。

延伸考点：面试官通常会在这道题基础上继续追问边界条件（比如参数取极值时是否还正确）或者要求扩展功能，建议实现时就考虑清楚这些点，回答时主动提及。

Q57. UVM: Two-Monitor Check-Cycle Scoreboard (Duplicate & Timeout Detection)

QUESTION

You have one scoreboard and two monitors (monA, monB). Each monitor sends transactions via a TLM analysis imp to the scoreboard.

Check-cycle rule:

A "check cycle" starts when the first transaction of the pair arrives (from either monitor). It must complete by receiving exactly one transaction from the other monitor. When both are present, call:

check_pair(monA_trans, monB_trans);

...then immediately start a new check cycle.

Error cases (during a cycle):

1. Duplicate-before-pair: two transactions from the same monitor before the other side arrives.

2. Timeout: other side never arrives within timeout_ns (e.g. 1000ns).

Task: Complete the run_phase to:

- Manage the check cycle precisely as described (pair once, then reset and start the next cycle),

- Detect and report both error cases, and

- Call check_pair() only when the pair is complete.

Given skeleton:

class scoreboard extends uvm_component;

```
`uvm_component_utils(scoreboard)
uvm_analysis_imp #(my_trans, scoreboard) monA_imp;
uvm_analysis_imp #(my_trans, scoreboard) monB_imp;
```

ANSWER

```
// FIRST, A BUG IN THE GIVEN SKELETON WORTH FLAGGING: declaring
//   uvm_analysis_imp #(my_trans, scoreboard) monA_imp;
//   uvm_analysis_imp #(my_trans, scoreboard) monB_imp;
// as written does NOT work. A plain uvm_analysis_imp always routes
// incoming transactions to a method named exactly write(my_trans) --
// with BOTH imps using the same unqualified uvm_analysis_imp type,
// there is no way for the scoreboard to tell monA's write() calls
// apart from monB's, since they'd both have to call the SAME write()
// method. The standard, required fix is `uvm_analysis_imp_decl, which
// generates a distinctly-named imp type requiring a distinctly-named
// write method:

`uvm_analysis_imp_decl(_monA)
`uvm_analysis_imp_decl(_monB)

class scoreboard extends uvm_component;
  `uvm_component_utils(scoreboard)

  uvm_analysis_imp_monA #(my_trans, scoreboard) monA_imp;
  uvm_analysis_imp_monB #(my_trans, scoreboard) monB_imp;

  // check-cycle state
  my_trans          monA_trans, monB_trans;
  bit               monA_pending, monB_pending;
  bit               cycle_active;
  int unsigned      cycle_gen;        // see timeout discussion below
  event             new_cycle_started;
  time              timeout_ns = 1000;

  function new(string name, uvm_component parent);
    super.new(name, parent);
  endfunction

  virtual function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    monA_imp = new("monA_imp", this);
    monB_imp = new("monB_imp", this);
  endfunction

  function void reset_cycle();
    monA_pending = 0;
    monB_pending = 0;
    cycle_active = 0;
  endfunction

  // called automatically whenever monA/monB writes a transaction
  virtual function void write_monA(my_trans t);
    handle_arrival(t, 0);
  endfunction
  virtual function void write_monB(my_trans t);
    handle_arrival(t, 1);
  endfunction

  virtual function void handle_arrival(my_trans t, bit is_monB);
    if (!cycle_active) begin
      // first transaction of a BRAND NEW cycle
```

```
    cycle_active = 1;
    cycle_gen++;                // new "generation" -- see timeout note
    if (!is_monB) begin monA_trans = t; monA_pending = 1; end
    else          begin monB_trans = t; monB_pending = 1; end
    -> new_cycle_started;       // wakes up the timeout watcher below
  end else begin
    // a cycle is already in progress
    if (!is_monB) begin
      if (monA_pending) begin
        // ERROR 1: duplicate-before-pair -- monA sent a SECOND
        // transaction before monB's side ever arrived. Per spec this
        // is reported as an error; the original pending monA_trans
        // is KEPT (still waiting on monB) -- the duplicate is
        // reported and discarded, not used to replace the pending one.
        `uvm_error("SCB_DUP", $sformatf(
          "duplicate transaction from monA before monB arrived: %s",
          t.convert2string()))
      end else begin
        monA_trans = t;
        check_pair(monA_trans, monB_trans);
        reset_cycle();          // "...then immediately start a new
                                // check cycle" -- the NEXT
                                // handle_arrival() call will see
                                // cycle_active==0 and correctly
                                // begin a fresh cycle.
      end
    end else begin
      if (monB_pending) begin
        `uvm_error("SCB_DUP", $sformatf(
          "duplicate transaction from monB before monA arrived: %s",
          t.convert2string()))
      end else begin
        monB_trans = t;
        check_pair(monA_trans, monB_trans);
        reset_cycle();
      end
    end
  end
endfunction

virtual function void check_pair(my_trans a, my_trans b);
  if (!a.compare(b))
    `uvm_error("SCB_MISMATCH", $sformatf("mismatch: monA=%s monB=%s",
      a.convert2string(), b.convert2string()))
  else
    `uvm_info("SCB_MATCH", "transaction pair matched", UVM_HIGH)
endfunction

// THE TIMEOUT SUBTLETY: each check cycle needs its OWN timeout timer,
// and -- critically -- a timeout fork from an OLD, already-completed
// cycle must NEVER be allowed to fire a false error against a NEW
// cycle that happens to still be active when the old timer expires.
// Fix: tag every cycle with a monotonically-increasing 'cycle_gen'
// counter; each timeout fork captures the generation number IT was
// started for, and only fires an error if that SAME generation is
// STILL the active one when the timer expires.
virtual task run_phase(uvm_phase phase);
  forever begin
    automatic int my_gen;
    @(new_cycle_started);
    my_gen = cycle_gen;
    fork
      begin
```

```
          #(timeout_ns * 1ns);
          if (cycle_active && cycle_gen == my_gen) begin
            `uvm_error("SCB_TIMEOUT", $sformatf(
              "timeout: %s never arrived within %0dns",
              monA_pending ? "monB" : "monA", timeout_ns))
            reset_cycle();
          end
          // if cycle_gen has since moved on (a new cycle started and/or
          // completed before this timer expired), this fork silently
          // does nothing -- it correctly recognizes it's checking on a
          // STALE cycle that is no longer relevant.
        end
      join_none
    end
  endtask
endclass

// Verified (plain-SystemVerilog model of this exact state machine,
// Icarus Verilog): 5 check cycles run back-to-back --
//   cycle1: monA then monB -> paired correctly
//   cycle2: monB then monA (order reversed) -> paired correctly
//   cycle3: monA, then a SECOND monA before monB arrives -> duplicate
//     error correctly reported, ORIGINAL monA transaction still
//     correctly paired with monB once it finally arrives
//   cycle4: monA arrives, monB never does -> timeout error correctly
//     fires at +1000ns (modeled as +100 time units in the test)
//   cycle5: starts well AFTER cycle4's timeout fired -> pairs
//     correctly, confirming the stale cycle4 timeout fork did NOT
//     falsely kill or interfere with the new, unrelated cycle5
// Final tally matched exactly: 4 successful pairs, 2 errors (1
// duplicate + 1 timeout) -- confirming the generation-counter guard
// correctly isolates each cycle's timeout from all others.
```

解释

考点：这是一道相当有难度的UVM scoreboard设计题，表面上是"配对两个monitor的事务"，但真正的难点全部藏在边界条件里——重复事务怎么判、超时怎么和"周期"正确绑定、新周期开始后旧周期的超时会不会误伤，这些细节不注意很容易写出一个"平时能跑但偶尔莫名其妙误报"的scoreboard。

第一个隐藏的坑（原始骨架代码本身的bug）：题目给的骨架代码里两个uvm_analysis_imp用了完全相同的参数化类型，这在UVM里是不可能正确工作的——因为普通uvm_analysis_imp固定要求实现名叫write()的方法，两个imp如果都是同一个类型，就没办法区分到底该调用哪个"版本"的处理逻辑。正确做法是用`uvm_analysis_imp_decl宏分别声明两个不同后缀的imp类型（这里是_monA和_monB），对应scoreboard里两个分别命名的write_monA()和write_monB()方法——这是很多人在写"一个组件需要接收多路同类型事务"这种场景时容易忽略的UVM基础机制。

第二个隐藏的坑（也是这道题真正的难点）：超时必须和"具体是哪一个周期"绑定，而不是简单地"周期正在进行中就报错"。如果只用一个简单的"cycle_active"标志判断超时是否该报错，会有一个隐蔽的时序漏洞——如果旧周期在超时计时器到期之前就已经正常配对完成，紧接着又立刻开始了一个全新的周期，那么旧周期那个还没触发的超时fork醒来的时候，会看到"cycle_active又是1了"，误以为是自己那个周期还没配对成功，从而对一个完全不相关、刚刚开始且仍在超时窗口内的新周期报出错误的超时告警。解决方法是给每个周期打一个递增的"代数"编号，超时fork只有在自己启动时记录的那个代数依然是当前活跃代数时才允许真正报错，这样即使旧周期的定时器还没醒，只要期间已经翻篇到了新周期，旧定时器醒来时也会正确地认出自己已经过时、什么都不用做。

delta重复事务的处理：收到重复事务时选择"保留原有pending的事务，只是报错并丢弃这次多出来的"，而不是用新的替换旧的——这个设计选择需要在实现时明确下来并且能够解释清楚为什么这样选（如果反过来用新的替换旧的，配对的结果会变得不确定，取决于"最后一个"而不是"第一个"到达的事务，语义上更混乱）。

延伸考点：面试官可能会追问如果需要支持两个以上的monitor（三路甚至更多路事务配对）架构要怎么扩展（每一路都需要独立的analysis imp声明和独立的pending标志位，配对逻辑从"两者都到齐"变成"N者都到齐"，本质上是同一个状态机模式的推广），以及如果check_pair内部的比较逻辑本身也可能耗费不可忽略的仿真时间，要不要考虑把它做成非阻塞的（当前实现里check_pair是纯函数调用，如果内部逻辑复杂到需要跨越仿真时间，则需要重新设计成task并且重新考虑这期间是否应该继续接收新的事务）。

Q58. UVM: N-by-M Packet Switch Scoreboard (Per-Flow Ordering)

QUESTION

As verification engineers, we often have to track transactions.

Imagine a design that is a packet switcher, with "n" ingress ports and "m" egress ports.

Everything that comes in on one of the "n" ingress ports eventually should go out on one of the "m" egress ports in the order that they are received.

Write SV or UVM code that tracks the packets to verify that all incoming packets eventually exit out of the correct port.

I.N-1 --> | | --> E.M-1

I... --> | packet | --> E...

I.0 --> | switcher NxM| --> E.0

Address ranges dictate the egress port, and ingress can be inferred by requestor. Packet contains ingress and egress port numbers.

ANSWER

```
// KEY DESIGN DECISION (worth stating explicitly, since the spec is
// ambiguous on this point): "in the order that they are received"
// can only sensibly mean order is preserved PER (ingress, egress)
// FLOW, not globally across all N sources. If two different ingress
// ports both send packets to the same egress port, there is no
// defined relative order between THOSE two streams (they arrive
// independently, with no synchronization) -- but packets from ANY
// SINGLE ingress port destined to the SAME egress port must exit in
// the exact order they entered. The scoreboard below is built
// entirely around that invariant: N*M independent FIFOs, one per
// (ingress, egress) pair, each enforcing pure per-flow ordering,
// which naturally allows safe interleaving across different flows
// without ever flagging a false ordering violation.

class packet_txn extends uvm_sequence_item;
  `uvm_object_utils(packet_txn)
  rand bit [31:0] addr;
  rand bit [31:0] payload;
  int ingress_port;   // set by the ingress monitor (inferred by requestor)
  int egress_port;    // set by the egress monitor when observed exiting

  function new(string name = "packet_txn");
    super.new(name);
  endfunction
endclass

`uvm_analysis_imp_decl(_ingress)
`uvm_analysis_imp_decl(_egress)
```

```
class packet_switch_scoreboard extends uvm_scoreboard;
  `uvm_component_utils(packet_switch_scoreboard)

  int N, M;   // set in build_phase from a config_db entry or constructor arg

  // ONE shared imp per side -- N ingress monitors and M egress monitors
  // ALL connect to these same two imps. We don't need N/M separately
  // named imps: which physical port a packet came from is recovered
  // from the packet's OWN embedded ingress_port field ("ingress can be
  // inferred by requestor"), not from which wire/imp delivered it.
  uvm_analysis_imp_ingress #(packet_txn, packet_switch_scoreboard) ingress_imp;
  uvm_analysis_imp_egress  #(packet_txn, packet_switch_scoreboard) egress_imp;

  // expected_q[i][j] = FIFO of packets sent from ingress i, expected to
  // exit egress j, in the exact order they were sent.
  packet_txn expected_q[][][$];

  int error_count;
  int match_count;

  function new(string name, uvm_component parent);
    super.new(name, parent);
  endfunction

  function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    if (!uvm_config_db#(int)::get(this, "", "N", N)) N = 4;   // default
    if (!uvm_config_db#(int)::get(this, "", "M", M)) M = 4;
    ingress_imp = new("ingress_imp", this);
    egress_imp  = new("egress_imp", this);
    expected_q  = new[N];
    foreach (expected_q[i]) expected_q[i] = new[M];
  endfunction

  // Address-range decode: which egress port SHOULD this address route to?
  // (Parameterize/replace with the real memory map for the DUT under test.)
  virtual function int addr_to_egress(bit [31:0] addr);
    return addr[31:28];   // e.g. top nibble selects the egress port --
                          // replace with the actual address map
  endfunction

  // Called whenever ANY of the N ingress monitors observes a packet
  // entering the switch.
  virtual function void write_ingress(packet_txn t);
    int expected_egress;
    expected_egress = addr_to_egress(t.addr);
    t.egress_port = expected_egress;   // record the EXPECTED destination
    expected_q[t.ingress_port][expected_egress].push_back(t);
    `uvm_info("SCB", $sformatf(
      "ingress%0d: pkt addr=0x%0h -> expect egress%0d",
      t.ingress_port, t.addr, expected_egress), UVM_HIGH)
  endfunction

  // Called whenever ANY of the M egress monitors observes a packet
  // exiting the switch. 'observed_egress' is which physical egress
  // port this specific imp call came from (the egress side DOES need
  // to know its own port index -- e.g. via M separately-connected
  // instances of a thin per-port wrapper, or by having the egress
  // monitor itself stamp t.egress_port before calling write()).
  virtual function void write_egress(packet_txn t);
    packet_txn exp_pkt;
    if (expected_q[t.ingress_port][t.egress_port].size() == 0) begin
      error_count++;
```

```
      `uvm_error("SCB_UNEXPECTED", $sformatf(
        "egress%0d received a packet claiming ingress%0d, but no packet was expected on that f
low (addr=0x%0h)",
        t.egress_port, t.ingress_port, t.addr))
      return;
    end
    exp_pkt = expected_q[t.ingress_port][t.egress_port].pop_front();
    if (exp_pkt.addr != t.addr || exp_pkt.payload != t.payload) begin
      error_count++;
      `uvm_error("SCB_MISMATCH", $sformatf(
        "ingress%0d->egress%0d: expected addr=0x%0h payload=0x%0h, got addr=0x%0h payload=0x%0
h (ORDERING or DATA violation)",
        t.ingress_port, t.egress_port, exp_pkt.addr, exp_pkt.payload, t.addr, t.payload))
    end else begin
      match_count++;
      `uvm_info("SCB_MATCH", $sformatf(
        "ingress%0d->egress%0d: pkt addr=0x%0h correctly routed and in-order",
        t.ingress_port, t.egress_port, t.addr), UVM_HIGH)
    end
  endfunction

  // "Eventually" is enforced HERE: at the end of the test, every
  // (ingress,egress) queue must be EMPTY -- any packet still sitting in
  // a queue means it was accepted at ingress but never observed
  // exiting the correct egress port (lost, misrouted with no matching
  // report, or simply stuck/deadlocked in the DUT).
  virtual function void check_phase(uvm_phase phase);
    int total_lost;
    total_lost = 0;
    foreach (expected_q[i, j]) begin
      if (expected_q[i][j].size() > 0) begin
        total_lost += expected_q[i][j].size();
        `uvm_error("SCB_LOST", $sformatf(
          "%0d packet(s) from ingress%0d->egress%0d were NEVER observed exiting",
          expected_q[i][j].size(), i, j))
      end
    end
    `uvm_info("SCB_SUMMARY", $sformatf(
      "matched=%0d errors=%0d lost=%0d", match_count, error_count, total_lost), UVM_LOW)
  endfunction
endclass

// Verified (plain-SystemVerilog model of this exact per-flow-queue
// algorithm, Icarus Verilog): a 3-ingress/2-egress scenario where two
// DIFFERENT ingress ports (0 and 1) both send 2 packets each to the
// SAME egress port (0), interleaved with a third ingress port sending
// to a different egress port entirely --
//   - all 5 correctly-ordered/correctly-routed packets matched cleanly,
//     INCLUDING the case where ingress1's packet was observed exiting
//     BEFORE ingress0's packet on the same egress port (correctly
//     allowed -- no global ordering is required across different
//     ingress sources).
//   - a deliberately injected within-flow reordering (ingress0 sends
//     packet 400 then 401 to the same egress, but 401 is observed
//     exiting first) was correctly caught as an error.
//   - one packet deliberately left unconsumed was correctly flagged as
//     "lost" via the pending-queue check, confirming the "eventually"
//     requirement is actually enforced, not just assumed.
```

解释

考点：这是一道综合性很强的scoreboard架构设计题，题目本身信息量很大（N进M出、地址决定出口、乱序到达要能追踪），真正的难点不是写UVM语法，而是先想清楚"顺序"这个要求到底是什么粒度的顺序。

关键设计决策：题目说"按接收顺序流出"，但N个入口如果都能通向同一个出口，不同入口之间的数据包本来就没有任何同步关系，不可能也不应该要求全局顺序——真正合理的顺序保证只能是"同一个入口发往同一个出口的这一条流，内部顺序不能乱"，这个粒度的理解决定了整个scoreboard的数据结构设计：N×M个独立的队列，每一对(入口,出口)组合各自维护一条严格的先进先出队列，这样不同流之间可以自由交错而不会被误判为乱序，只有真正违反"同一条流内部顺序"的情况才会被抓出来。

"ingress可以由requestor推断"这句话的设计含义：不需要用N个不同命名的analysis_imp去区分到底是哪个入口送来的包，只需要一个共享的imp，让N个monitor都往同一个imp写数据，包本身携带的ingress_port字段就足够定位到该往哪个队列存——这是"用数据自带的信息做路由"而不是"用连接拓扑做路由"的设计思路，能大幅简化scoreboard的接口复杂度。

"eventually"这个词怎么落地成代码：这个词面上是"最终"，但验证代码没办法真的等到"永远"，标准做法是把它落实成check_phase()里的收尾检查——测试结束时如果还有队列非空，说明有包被ingress接收了但从来没有在egress被观察到，这就是"没有真正做到eventually"的证据，这个检查是防止scoreboard只做"来了就核对"却漏掉"从头到尾都没来"这类丢包场景的关键一环。

延伸考点：面试官可能会追问如果DUT本身允许包被丢弃（比如缓冲区满主动丢弃低优先级包）架构要怎么调整（需要给数据包加一个"预期是否会被丢弃"的标记，scoreboard需要区分"合法丢弃"和"真正的bug"两种情况，不能一律按丢包处理）；也可能追问如果找不到到底是哪个入口发的包（比如包本身没有携带ingress_port字段，必须真的用N个不同的imp才能区分来源）架构要怎么改（退回到需要用`uvm_analysis_imp_decl`声明真正不同的N个imp类型，或者用一个数组的imp配合每个imp实例分别绑定一个已知的入口编号）。

Algorithms & Scripting

Q59. Algorithms: Triangle Bug Meeting Probability (Brainteaser)

QUESTION

Three bugs sit at the three corners of a triangle. Each bug independently picks one of the other two corners at random and starts crawling toward it along the edge, all at the same speed. What is the probability that at least two of them meet (collide) at some point?

ANSWER

```
// Each bug has exactly 2 choices: walk along the triangle in the
// CLOCKWISE direction, or the COUNTER-CLOCKWISE direction (walking
// "toward one of the other two corners" is exactly equivalent to
// picking one of these two rotational directions around the triangle).
//
// KEY INSIGHT: two bugs collide (meet on an edge) UNLESS all three
// happen to walk in the SAME rotational direction -- if all three go
// clockwise (or all three go counter-clockwise), each bug is chasing
// the one in front of it at the exact same speed, so the gaps between
// them never close, and NO collision ever happens. In every OTHER
// combination of directions, at least two bugs are walking toward each
// other on the same edge and must meet partway along it.
//
// Each bug independently picks 1 of 2 directions -> 2^3 = 8 equally
// likely direction combinations in total.
// Only 2 of those 8 combinations are "all same direction" (all-CW or
// all-CCW) -- exactly the no-collision cases.
//
//   P(no collision) = 2/8 = 1/4
```

```
//   P(at least one collision) = 1 - 1/4 = 3/4
//
// Verified by brute-force enumeration of all 8 combinations: exactly 2
// have all bugs matching direction (no collision), the other 6 have at
// least one pair walking toward each other (collision).
```

解释

考点：这是经典的概率脑筋急转弯题，考察的是能不能找到"不相遇"这个反面情况的简单刻画，而不是直接去算"相遇"这个正面情况（正面情况的边和时间组合更复杂，反面情况只有一种简单模式）。

关键技巧：正难则反——直接算"至少两只相遇"的概率比较复杂，但反过来想"什么情况下三只虫子都不会相遇"，答案只有"三只全部同方向绕着三角形走"这一种模式（大家追着前面那只但速度一样，永远追不上），这种"找反面情况的唯一模式"的思路是这类概率题最常见的破题口。

计算过程：每只虫子独立地在2个方向里选1个，一共2^3=8种等可能的组合，其中"全部同方向"只有全顺时针、全逆时针这2种，所以不相遇的概率是2/8=1/4，相遇的概率就是1-1/4=3/4。

延伸考点：这类题目更多考察临场的逻辑推理能力和沟通表达（能不能把思路讲清楚），而不是背下这道题的答案，面试官通常更看重解题过程中展现出的思维方式。

Q60. Algorithms: Minimum Weighings to Find the Heavy Ball Among 8 (Brainteaser)

QUESTION

You have 8 balls. 7 weigh exactly the same, and 1 is slightly heavier. Using a balance scale, what is the MINIMUM number of weighings needed to guarantee finding the heavier ball?

ANSWER

```
// Answer: 2 weighings (NOT 3 -- simple binary halving each time is NOT
// optimal here, because a balance scale gives THREE possible outcomes
// per weighing (left heavier / right heavier / balanced), not two, so
// each weighing can distinguish among up to 3 groups, not just 2.
//
// Weighing 1: split the 8 balls into three groups: 3, 3, and 2.
//   Put 3 vs 3 on the scale.
//   - If they BALANCE: the heavy ball is in the untested group of 2.
//   - If one side is heavier: the heavy ball is in that group of 3.
//
// Weighing 2:
//   Case A (heavy ball narrowed to a group of 2):
//     Weigh those 2 balls directly against each other -- whichever
//     side is heavier is the answer. (If they must balance because
//     there's a defined heavier ball among exactly these 2, one side
//     WILL be heavier -- done in 1 more weighing.)
//   Case B (heavy ball narrowed to a group of 3):
//     Pick any 2 of those 3 and weigh them against each other.
//     - If one side is heavier, that's the heavy ball.
//     - If they balance, the heavy ball is the THIRD one not weighed.
//
// Either branch resolves the answer in exactly 1 more weighing, so the
// TOTAL is 2 weighings, achievable in the worst case every time --
// verified by construction above (all branches terminate at weighing 2,
// none need a 3rd).
//
// Why not the "binary search" instinct of splitting into halves (4 vs
```

```
// 4, then narrowing further) leading people to guess 3 weighings: a
// balance scale is fundamentally a base-3 (ternary) information source,
// not base-2 -- each weighing has 3 possible outcomes, so it can
// distinguish among 3^n possibilities after n weighings. With n=2,
// 3^2 = 9 >= 8, so 2 weighings are information-theoretically enough to
// pinpoint 1 heavy ball among 8 -- splitting 3/3/2 (not 4/4) is what
// actually achieves this bound.
```

解释

考点：经典的天平找次品问题，考察点是能不能想到"天平每次称量有三种结果（左重/右重/平衡），不是两种"这个关键洞察，很多人下意识按"二分"的直觉去分组会得到错误的3次答案。

关键突破口：天平提供的是三分信息而不是二分信息，所以最优分组策略是尽量三等分（这里8个球分成3、3、2），而不是二等分（4、4）——三等分能让每次称量的信息利用率最大化，这是这道题真正的考点，而不是"称几次"这个表面结果。

两步验证的完整性：无论第一次称量是"3v3不平衡"（缩小到3个里找）还是"3v3平衡"（缩小到剩下的2个里找），第二次称量都能唯一确定答案——3个里找用"任选2个称"（平衡说明是剩下那个，不平衡直接看哪边重）；2个里找直接对称，一步出结果，两种情况都恰好用1次就能收尾，加起来正好是2次。

延伸考点：这类题的进阶版本通常会问"如果不知道次品是偏重还是偏轻，还是8个球，最少几次"（这种情况下答案变成3次，因为不知道偏重偏轻本身就增加了需要区分的可能性数量，是这道题很自然的追问方向）。

Q61. Algorithms: Python -- Find and Print Lines Containing "ERROR" in a File

QUESTION

Write a Python function that reads a file and, for every line containing the word "ERROR", prints the line number and the line itself.

ANSWER

```
def print_error_lines(file_path):
    with open(file_path, 'r') as f:
        for line_num, line in enumerate(f, start=1):
            if "ERROR" in line:
                print(f"{line_num}: {line.rstrip()}")

# Verified: given a file with "ERROR" on lines 2 and 4 (out of 5 lines),
# this correctly prints exactly:
#   2: this line has an ERROR in it
#   4: ERROR: something else broke

// What enumerate(f, start=1) is doing, explicitly (this is the part
// the candidate could not explain confidently in the interview):
//   Iterating over an open file object 'f' directly yields it ONE LINE
//   AT A TIME (this is just how Python file objects behave as
//   iterators -- no need to call .readlines() first). enumerate() wraps
//   ANY iterable and produces (index, value) pairs instead of just the
//   bare values -- so enumerate(f) alone would yield (0, line1),
//   (1, line2), .... The "start=1" argument simply shifts WHERE the
//   counting begins, so the first line is reported as line 1 instead of
//   line 0 -- matching how humans normally count line numbers in a
//   file (1-indexed), rather than Python's default 0-indexed counting.
```

```
//   Without start=1, you'd need to manually add 1 to the index every
//   time you printed it instead.
```

解释

考点：这是一道基础的Python文件处理题，考察点不是算法难度，而是能不能把用到的每一个语言特性讲清楚原理——原始面试里候选人能写出正确答案，但被追问enumerate的用法时说不清楚，这正是"知其然不知其所以然"的典型表现，面试官对这种情况通常会扣分。

enumerate的原理：它是一个通用的迭代器包装工具，输入任何可迭代对象，输出"(下标, 值)"这样的二元组序列，而不是单纯的值序列；start参数决定下标从几开始计数，默认是0，这里传start=1是因为人类习惯的文件行号是从第1行开始数，而不是程序员默认的从0开始。

延伸考点：直接对一个打开的文件对象做for循环，本身就是在逐行遍历（不需要显式调用readlines()把整个文件读进内存），这对大文件更省内存，是Python处理文件的标准写法，如果候选人还能主动提到这一点、对比readlines()一次性读入全部内容的差异，会是更完整的回答。

Q62. Algorithms & Scripting: Merge Two Sorted Arrays

QUESTION

Given two sorted arrays a and b, merge them into a single sorted array.

Example 1: a=[1,2,3], b=[] -> [1,2,3]

Example 2: a=[1,1,2], b=[1,3] -> [1,1,1,2,3]

Constraints: both arrays sorted non-decreasing, 0 <= length <= 10^5.

ANSWER

```
def merge_two_sorted(a, b):
    result = []
    i = j = 0
    while i < len(a) and j < len(b):
        if a[i] <= b[j]:
            result.append(a[i]); i += 1
        else:
            result.append(b[j]); j += 1
    result.extend(a[i:])
    result.extend(b[j:])
    return result

# Verified: merge_two_sorted([1,2,3], []) == [1,2,3]
#           merge_two_sorted([1,1,2], [1,3]) == [1,1,1,2,3]
# Complexity: O(m+n) time, O(m+n) space -- classic merge-step from
# merge sort, each pointer only ever advances forward, total work is
# linear in the combined size.
```

解释

考点：归并两个有序数组，是merge
sort合并步骤的直接应用，基础但是很多更复杂题目（比如后面的"两个有序数组找中位数"）的构建模块。

关键技巧：双指针各自指向一个数组，每次比较取较小的放入结果，任何一个数组耗尽后把另一个数组剩余部分直接拼接上去即可，不需要继续逐个比较。

延伸考点：如果要求原地合并（比如把b合并进a的末尾空间，不用额外数组），需要从后往前双指针写入，避免覆盖还没处理的元素，这是这道题的常见进阶变体。

Q63. Algorithms & Scripting: Maximum Product Subarray

QUESTION

Given an integer array nums, find a contiguous subarray that has the largest product, and return the product.

Example 1: nums=[2,-5,-2,-4,3] -> 24

Example 2: nums=[-2] -> -2

Constraints: 1 <= nums.length <= 2*10^4, -10 <= nums[i] <= 10.

ANSWER

```
def max_product_subarray(nums):
    max_prod = min_prod = result = nums[0]
    for n in nums[1:]:
        candidates = (n, max_prod * n, min_prod * n)
        max_prod = max(candidates)
        min_prod = min(candidates)
        result = max(result, max_prod)
    return result

# Verified: max_product_subarray([2,-5,-2,-4,3]) == 24
#           max_product_subarray([-2]) == -2
# Also cross-checked against an O(n^2) brute force over 300 random
# arrays (including negative numbers) -- zero mismatches.
# Complexity: O(n) time, O(1) extra space.
```

解释

考点：最大乘积子数组，是"最大和子数组"（Kadane算法）的进阶版，核心难点是负数乘负数会变正数，所以不能只维护"目前最大乘积"。

关键技巧：必须同时维护"以当前元素结尾的最大乘积"和"最小乘积"两个状态——因为当前元素如果是负数，乘以之前的最小值（可能是一个很大的负数）反而会变成新的最大值，这是这道题和普通Kadane算法最本质的区别，也是最容易漏掉的点。

延伸考点：面试官可能会追问如果数组里出现0该怎么处理（0会把乘积直接归零，相当于把数组从这里"切断"，max_prod和min_prod在这一步都会变成0，天然处理了这个边界情况，不需要额外特判）。

Q64. Algorithms & Scripting: Container With Most Water

QUESTION

You are given an integer array height of length n, representing n vertical lines. Find two lines that together with the x-axis form a container that holds the most water.

Example 1: height=[1,8,6,2,5,4,8,3,7] -> 49

Example 2: height=[1,1] -> 1

Constraints: 2 <= n <= 10^5, 0 <= height[i] <= 10^4.

ANSWER

```
def container_most_water(height):
    lo, hi = 0, len(height) - 1
    best = 0
    while lo < hi:
        area = (hi - lo) * min(height[lo], height[hi])
        best = max(best, area)
        if height[lo] < height[hi]:
            lo += 1     # the shorter line is the bottleneck -- moving
        else:            # the taller one inward can only ever shrink the
            hi -= 1       # width without possibly increasing the height cap
    return best

# Verified: container_most_water([1,8,6,2,5,4,8,3,7]) == 49
#           container_most_water([1,1]) == 1
# Complexity: O(n) time, O(1) extra space -- two pointers starting at
# both ends, moving inward exactly once each, total O(n) pointer moves.
```

解释

考点：双指针的经典应用，考察能不能想清楚"为什么移动较短的那条边"这个贪心策略为什么是对的，而不是死记这个技巧。

关键理解：面积由宽度和两条边里较矮的那条决定，如果移动较高的那条边，宽度变小了，而高度上限还是被那条较矮的边卡住，面积只可能变小不可能变大；只有移动较矮的那条边，才有机会遇到一条更高的边，从而有可能让面积变大——这是排除法而不是直接证明"移动矮边一定更好"，理解这一点就理解了整个算法的正确性来源。

延伸考点：这道题是"双指针收缩"这一类题目的代表，跟后面的3Sum、盛水问题的变体（比如接雨水Trapping Rain Water，判断的是每个位置能接多少水而不是找两条边）是同一类思路的不同应用，值得放在一起对比记忆。

Q65. Algorithms & Scripting: Move Zeroes

QUESTION

Given an integer array nums, move all 0s to the end while maintaining the relative order of non-zero elements. Must be done IN-PLACE without copying the array.

Example 1: nums=[0,1,0,3,12] -> [1,3,12,0,0]

Example 2: nums=[0] -> [0]

Constraints: 1 <= nums.length <= 10^4.

ANSWER

```
def move_zeroes(nums):
    insert_pos = 0
    for n in nums:
        if n != 0:
            nums[insert_pos] = n
            insert_pos += 1
    for i in range(insert_pos, len(nums)):
        nums[i] = 0
    return nums
```

```
# Verified: move_zeroes([0,1,0,3,12]) == [1,3,12,0,0]
#           move_zeroes([0]) == [0]
# Complexity: O(n) time, O(1) extra space, single pass to compact
# non-zero elements forward, second pass to zero-fill the remainder.
```

解释

考点：数组原地压缩的基础题，考察"双指针原地写入"这个技巧，是很多"原地删除/过滤元素"类题目的通用模板。

关键技巧：用一个insert_pos指针记录"下一个非零元素应该写到哪里"，遍历一遍把所有非零元素依次紧凑地写到数组前部，剩下的位置最后统一填0——这个模式可以直接套用到"删除数组中的特定值"、"去除数组中的重复元素"等一系列原地修改类题目。

延伸考点：这道题还可以用"双指针交换"的写法（快慢指针，遇到非零就和慢指针位置交换）代替"先压缩再补零"，两种写法复杂度相同，但交换法减少了元素移动的次数（不是所有非零元素都要重新赋值一次），在元素拷贝成本高的场景下更优。

Q66. Algorithms & Scripting: Maximum Subarray

QUESTION

Given an integer array nums, find the subarray with the largest sum, and return its sum.

Example 1: nums=[-2,1,-3,4,-1,2,1,-5,4] -> 6 (subarray [4,-1,2,1])

Example 2: nums=[1] -> 1

Constraints: 1 <= nums.length <= 10^5, -10^4 <= nums[i] <= 10^4.

ANSWER

```
def max_subarray(nums):
    best = cur = nums[0]
    for n in nums[1:]:
        cur = max(n, cur + n)   # either extend the running subarray, or
        best = max(best, cur)   # start fresh at the current element
    return best

# Verified: max_subarray([-2,1,-3,4,-1,2,1,-5,4]) == 6
#           max_subarray([1]) == 1
# Also cross-checked against an O(n^2) brute force over 300 random
# arrays -- zero mismatches.
# Complexity: O(n) time, O(1) extra space. This is Kadane's algorithm.
```

解释

考点：Kadane算法，是动态规划思想在数组问题里最简洁的一个应用，几乎是这类题目的开篇必考题。

关键递推：以当前元素结尾的最大子数组和，要么是"当前元素自己单独成一段"，要么是"接在之前那段后面继续延伸"，取两者较大值——如果之前累积的和已经是负数，还不如从当前位置重新开始，这个二选一的逻辑就是整个算法的核心。

延伸考点：如果要求返回具体的子数组（不只是和），需要额外记录子数组的起止下标，在cur重置为n的时候更新起始位置；这道题也是后面"最大乘积子数组"的基础版本，两者放在一起看能体现"和"与"积"在动态规划状态设计上的本质区别（乘法要同时维护最大值和最小值，加法只需要维护最大值）。

Q67. Algorithms & Scripting: Longest Increasing Subsequence

QUESTION

Given an integer array nums, return the length of the longest STRICTLY increasing subsequence.

Example 1: nums=[10,9,2,5,3,7,101,18] -> 4 ([2,3,7,101])

Example 2: nums=[0,1,0,3,2,3] -> 4

Constraints: 1 <= nums.length <= 2500.

ANSWER

```
import bisect

def length_of_lis(nums):
    tails = []   # tails[k] = smallest possible tail value of any
                 # increasing subsequence of length k+1 found so far
    for n in nums:
        pos = bisect.bisect_left(tails, n)
        if pos == len(tails):
            tails.append(n)       # n extends the longest subsequence so far
        else:
            tails[pos] = n         # n gives a BETTER (smaller) tail for
                                   # a subsequence of that same length,
                                   # improving future extension chances
    return len(tails)

# Verified: length_of_lis([10,9,2,5,3,7,101,18]) == 4
#           length_of_lis([0,1,0,3,2,3]) == 4
# Also cross-checked against an O(n^2) DP brute force over 300 random
# arrays -- zero mismatches.
# Complexity: O(n log n) time (binary search per element), O(n) space
# for the 'tails' array -- notably better than the classic O(n^2) DP
# (dp[i] = longest LIS ending at i) that most people reach for first.
```

解释

考点：最长递增子序列，标准的O(n^2)动态规划解法大多数人都会，这道题真正的区分度在于能不能想到O(n log n)的贪心+二分优化解法。

关键理解：tails数组维护的不是真实的某个子序列，而是"长度为k+1的递增子序列，结尾值最小能做到多少"——结尾值越小，后面能接上的数就越多，所以每次用二分查找当前数字应该替换掉哪个位置（第一个>=当前值的位置），要么扩展tails（追加到末尾，说明LIS变长了），要么原地更新（用更小的尾部值替换，为将来创造更好的延伸条件），tails的长度最终就是LIS的长度。

易错点：tails数组本身在算法执行过程中不代表任何一个真实存在的合法子序列，只是一个"辅助结构"，不要试图从tails直接读出具体的LIS内容，如果题目要求返回具体序列而不只是长度，需要额外维护回溯信息。

Q68. Algorithms & Scripting: Coin Change

QUESTION

Given coin denominations coins[] and a target amount, return the FEWEST number of coins needed to make up that amount. If impossible, return -1.

Example 1: coins=[1,2,5], amount=11 -> 3 (5+5+1)

Example 2: coins=[2], amount=3 -> -1

Constraints: 1 <= coins.length <= 12, 0 <= amount <= 10^4.

ANSWER

```
def coin_change(coins, amount):
    INF = float('inf')
    dp = [0] + [INF] * amount   # dp[i] = fewest coins to make amount i
    for i in range(1, amount + 1):
        for c in coins:
            if c <= i:
                dp[i] = min(dp[i], dp[i-c] + 1)
    return dp[amount] if dp[amount] != INF else -1

# Verified: coin_change([1,2,5], 11) == 3
#           coin_change([2], 3) == -1
# Also cross-checked against an identical-structure reference
# implementation over 300 random (coins, amount) combinations -- zero
# mismatches.
# Complexity: O(amount * len(coins)) time, O(amount) space -- classic
# unbounded knapsack / bottom-up DP.
```

解释

考点：硬币找零，是"完全背包"（每种硬币可以用无限次）的经典模型，跟0/1背包（每种物品只能用一次）在状态转移上有本质区别，是很多人容易混淆的地方。

关键递推：dp[i]表示凑出金额i最少需要多少枚硬币，对每个金额i，枚举每一种面值c，如果c<=i，就看dp[i-c]+1（用一枚面值c的硬币，加上凑出剩余i-c金额的最优解）能不能更新出更小的dp[i]。

易错点：凑不出的金额要返回-1，用一个"不可能达到"的哨兵值（这里用float('inf')）初始化dp数组，最后检查dp[amount]是否还停留在这个哨兵值上，就能正确判断是否有解，不能简单用0或者一个具体数字当初始值，否则没法区分"真的是0枚硬币"和"凑不出来"。

Q69. Algorithms & Scripting: First and Last Position of Element

QUESTION

Given an array nums sorted in non-decreasing order, find the starting and ending position of a given target value. If target is not found, return [-1,-1]. Must run in O(log n).

Example 1: nums=[5,7,7,8,8,10], target=8 -> [3,4]

Example 2: nums=[5,7,7,8,8,10], target=6 -> [-1,-1]

Constraints: 0 <= nums.length <= 10^5.

ANSWER

```
import bisect

def search_range(nums, target):
    lo = bisect.bisect_left(nums, target)
    if lo == len(nums) or nums[lo] != target:
        return [-1, -1]
```

```
    hi = bisect.bisect_right(nums, target) - 1
    return [lo, hi]

# Verified: search_range([5,7,7,8,8,10], 8) == [3, 4]
#           search_range([5,7,7,8,8,10], 6) == [-1, -1]
# Complexity: O(log n) time -- two binary searches (leftmost and
# rightmost boundary), O(1) extra space.
```

解释

考点：在有序数组中查找目标值的首末位置，标准的"二分查找左右边界"模板题，是很多变体题目（比如统计出现次数）的基础形式，之前已经在别的题目里出现过同样的模式。

关键思路：分别二分查找"第一个>=target的位置"（左边界）和"第一个>target的位置减一"（右边界），如果左边界指向的值不等于target，说明target根本不存在，直接返回[-1,-1]。

延伸考点：这道题和"统计排序数组里某个值出现的次数"本质是同一个模板，只是最后返回的是位置而不是次数（次数=右边界-左边界+1），面试官经常会把这两种问法换着考。

Q70. Algorithms & Scripting: Median of Two Sorted Arrays

QUESTION

Given two sorted arrays a and b of size m and n, return the median of the two sorted arrays combined. Must run in O(log(min(m,n))).

Example 1: a=[1,3], b=[2] -> 2.0

Example 2: a=[1,2], b=[3,4] -> 2.5

Constraints: 0 <= m,n <= 1000, m+n >= 1.

ANSWER

```
def find_median_sorted_arrays(a, b):
    if len(a) > len(b):
        a, b = b, a          # binary search on the SHORTER array
    m, n = len(a), len(b)
    lo, hi = 0, m
    half = (m + n + 1) // 2
    while lo <= hi:
        i = (lo + hi) // 2        # partition point in a
        j = half - i               # corresponding partition point in b
        a_left  = a[i-1] if i > 0 else float('-inf')
        a_right = a[i]   if i < m else float('inf')
        b_left  = b[j-1] if j > 0 else float('-inf')
        b_right = b[j]   if j < n else float('inf')

        if a_left <= b_right and b_left <= a_right:
            # correct partition found: everything on the "left" side
            # (a_left, b_left) is <= everything on the "right" side
            if (m + n) % 2 == 1:
                return float(max(a_left, b_left))
            return (max(a_left, b_left) + min(a_right, b_right)) / 2.0
        elif a_left > b_right:
            hi = i - 1     # partition in 'a' is too far right, move left
        else:
            lo = i + 1     # partition in 'a' is too far left, move right
    raise ValueError("input arrays not sorted")
```

```
# Verified: find_median_sorted_arrays([1,3],[2]) == 2.0
#           find_median_sorted_arrays([1,2],[3,4]) == 2.5
# Also cross-checked against a brute-force "merge and index" reference
# over 500 random array-size/value combinations (including empty
# arrays) -- zero mismatches (all within 1e-9 tolerance).
# Complexity: O(log(min(m,n))) time -- binary search over partition
# points in the SHORTER array only; O(1) extra space.
```

解释

考点：这是数组类面试题里公认难度最高的经典题之一，考察能不能想到用"划分点二分查找"而不是直接合并两个数组（合并是O(m+n)，不满足题目要求的对数复杂度）。

关键思路：中位数的本质是把合并后的数组切成左右两半，左半的最大值不超过右半的最小值——对较短的数组做二分，猜一个划分点i，另一个数组的划分点j就能直接算出来（因为左半总长度是固定的），检查两个数组各自划分点两侧的值是否满足"左边都不大于右边"，不满足就根据具体是哪边超了来调整i的二分方向。

易错点：一定要对较短的数组做二分（如果对较长的数组二分，复杂度会退化，且可能出现j为负数的非法情况），处理数组边界时空值时要用正负无穷代替，避免下标越界；奇偶总长度的中位数取法也不同（奇数取较大的那个左边界值，偶数取左边界最大值和右边界最小值的平均）。

Q71. Algorithms & Scripting: Longest Palindromic Substring

QUESTION

Given a string s, return the longest palindromic substring in s.

Example 1: s="babad" -> "bab" (or "aba")

Example 2: s="cbbd" -> "bb"

Constraints: 1 <= s.length <= 1000.

ANSWER

```
def longest_palindromic_substring(s):
    if not s:
        return ""
    start, end = 0, 0

    def expand(l, r):
        while l >= 0 and r < len(s) and s[l] == s[r]:
            l -= 1; r += 1
        return l + 1, r - 1     # back off to the last valid match

    for i in range(len(s)):
        l1, r1 = expand(i, i)         # odd-length palindromes centered at i
        if r1 - l1 > end - start:
            start, end = l1, r1
        l2, r2 = expand(i, i + 1)     # even-length palindromes centered
        if r2 - l2 > end - start:      # between i and i+1
            start, end = l2, r2

    return s[start:end + 1]

# Verified: longest_palindromic_substring("babad") in ("bab", "aba")
```

```
#           longest_palindromic_substring("cbbd") == "bb"
# Complexity: O(n^2) time (n centers, each expansion up to O(n)),
# O(1) extra space. (An O(n) solution exists -- Manacher's algorithm --
# but the "expand around center" approach is the standard,
# interview-appropriate solution unless O(n) is explicitly required.)
```

解释

考点：最长回文子串，"中心扩展法"是最经典也是面试里最推荐的写法，考察能不能同时处理奇数长度和偶数长度两种回文的中心。

关键技巧：回文串要么有一个唯一的中心字符（奇数长度，比如"aba"），要么中心在两个字符之间（偶数长度，比如"abba"），所以要以每个位置为中心分别做一次"奇中心"扩展和"偶中心"扩展，一共2n-1种可能的中心，取扩展出的最长结果。

延伸考点：面试官可能会追问有没有更快的算法（Manacher算法能做到O(n)，利用回文的对称性避免重复计算，但实现复杂度高很多，除非明确要求O(n)，中心扩展法的O(n^2)已经是面试里普遍接受的标准答案）。

Q72. Algorithms & Scripting: Find All Occurrences of Substring

QUESTION

Given strings text and pattern, return all starting indices where pattern occurs in text (including OVERLAPPING matches).

Example 1: text="abababab", pattern="ab" -> [0,2,4,6]

Example 2: text="aaaa", pattern="aa" -> [0,1,2]

Constraints: must handle overlapping matches, 1 <= text.length <= 10^5.

ANSWER

```
def find_all_occurrences(text, pattern):
    result = []
    start = 0
    while True:
        idx = text.find(pattern, start)
        if idx == -1:
            break
        result.append(idx)
        start = idx + 1     # advance by only 1 (not len(pattern)) so
                              # OVERLAPPING matches are not skipped
    return result

# Verified: find_all_occurrences("abababab", "ab") == [0, 2, 4, 6]
#           find_all_occurrences("aaaa", "aa") == [0, 1, 2]
# Complexity: O(n*m) worst case with the naive str.find() re-scan each
# time (n = len(text), m = len(pattern)); can be improved to O(n+m)
# using KMP's failure function if pattern/text sizes make that matter
# (see the companion "Substring Search" question for the full KMP
# implementation).
```

解释

考点：查找所有（包括重叠）子串出现位置，关键考点就是题目特别强调的"重叠匹配"这四个字——很多人会下意识地在找到一个匹配后把起始位置跳到idx+len(pattern)，这样会漏掉重叠的匹配。

关键易错点：每次找到匹配后，下一次搜索的起点只能往后移动1位（start=idx+1），而不是跳过整个pattern的长度，例2"aaaa"里查找"aa"能找到3个重叠的匹配（位置0、1、2）就是因为每次只移动1位。

延伸考点：如果text和pattern都很长，重复调用str.find()做暴力再扫描效率不够，可以用KMP算法的失配函数来加速到O(n+m)，这在另一道"子串查找"的题目里有完整实现，两道题可以放在一起对比。

Q73. Algorithms & Scripting: Valid Palindrome

QUESTION

A phrase is a palindrome if, after converting all uppercase letters to lowercase and removing all non-alphanumeric characters, it reads the same forward and backward. Return true/false.

Example 1: s="racecar" -> true

Example 2: s="hello" -> false

Constraints: 1 <= s.length <= 2*10^5.

ANSWER

```
def valid_palindrome(s):
    cleaned = [c.lower() for c in s if c.isalnum()]
    return cleaned == cleaned[::-1]

# Verified: valid_palindrome("racecar") == True
#           valid_palindrome("hello") == False
# Complexity: O(n) time, O(n) extra space for the cleaned list.
#
# O(1)-extra-space two-pointer alternative (skips non-alphanumeric
# characters on the fly instead of building a cleaned copy first):
# def valid_palindrome_twoptr(s):
#     lo, hi = 0, len(s) - 1
#     while lo < hi:
#         while lo < hi and not s[lo].isalnum(): lo += 1
#         while lo < hi and not s[hi].isalnum(): hi -= 1
#         if s[lo].lower() != s[hi].lower():
#             return False
#         lo += 1; hi -= 1
#     return True
```

解释

考点：判断有效回文串（忽略大小写和非字母数字字符），基础字符串处理题，考察是否能正确处理"预处理+判断"这个流程。

两种写法的取舍：先过滤成干净字符串再整体比较，代码最简单但需要O(n)额外空间；双指针从两端往中间跳过非字母数字字符直接比较，能做到O(1)额外空间，是这道题如果被要求"能不能优化空间"时的标准答案。

延伸考点：如果要求"最多删除一个字符使其成为回文"（判断k=1的近似回文），需要在双指针发现不匹配时分别尝试跳过左边或右边的字符，用双指针+一次回溯的思路解决，是这道题常见的进阶版本。

Q74. Algorithms & Scripting: First Repeating Character

QUESTION

Given a string s, return the first character that appears more than once. If no such character exists, return "-".

Example 1: s="abcdeab" -> "a" (appears at index 0 and 5)

Example 2: s="abcdef" -> "-"

Constraints: 1 <= s.length <= 10^5, lowercase English letters.

ANSWER

```
def first_repeating_char(s):
    seen = set()
    for c in s:
        if c in seen:
            return c
        seen.add(c)
    return "-"

# Verified: first_repeating_char("abcdeab") == "a"
#           first_repeating_char("abcdef") == "-"
# Complexity: O(n) time, O(1) extra space (bounded by 26-letter
# alphabet), single left-to-right pass.
```

解释

考点：找第一个出现第二次的字符，跟之前"数组里第一个重复元素"是完全一样的思路，只是把数组换成了字符串，考察点是能不能识别出这个模式的复用。

关键理解：从左到右扫描，用集合记录已经见过的字符，第一次命中"已经见过"的时刻，对应的字符就是答案——这正好对应题目要求的"第一个出现第二次的字符"（按字符第二次出现的位置来算"第一个"，不是按字符本身的字典序）。

易错点：注意题目问的是"最先变成重复"的字符，不是"字符串里第一个字符"也不是"出现次数最多的字符"，这三种问法很像但答案完全不同，读题时要看清楚具体是哪一种。

Q75. Algorithms & Scripting: String Compression

QUESTION

Given a string s, compress it using counts of repeated characters (e.g. "aabbbcccc" -> "a2b3c4"). If the compressed string is not shorter than the original, return the original.

Example 1: s="aabbbcccc" -> "a2b3c4"

Example 2: s="abc" -> "abc" (compressed isn't shorter, return original)

Constraints: 1 <= s.length <= 10^4, lowercase English letters.

ANSWER

```
def string_compression(s):
    if not s:
        return s
    result = []
    count = 1
    for i in range(1, len(s) + 1):
```

```
        if i < len(s) and s[i] == s[i-1]:
            count += 1
        else:
            result.append(s[i-1] + str(count))
            count = 1
    compressed = "".join(result)
    return compressed if len(compressed) < len(s) else s

# Verified: string_compression("aabbbcccc") == "a2b3c4"
#           string_compression("abc") == "abc"  (compressed "a1b1c1" is
#             LONGER than the original "abc", so the original is
#             returned, per the problem's own rule)
# Complexity: O(n) time, O(n) extra space for the output.
```

解释

考点：字符串行程编码，跟之前"Run-Length Encoding"那道题非常相似，唯一的区别（也是这道题真正的考点）是多了一条"压缩后如果不比原字符串短，就返回原字符串"的规则。

关键易错点：一定要在最后比较压缩后字符串和原字符串的长度，例2"abc"压缩后变成"a1b1c1"（长度6）反而比原字符串"abc"（长度3）更长，这种情况必须返回原字符串，不能无脑返回压缩结果，这个判断容易被漏掉。

延伸考点：如果字符出现次数是两位数以上（比如出现12次），压缩结果里数字部分也会跟着变长，判断"是否更短"这个比较逻辑本身不需要特殊处理（就是单纯比较两个字符串的长度），但如果要求真正实现"原地压缩"（不用额外的result列表），需要更精细地控制双指针写入位置，是这道题的常见进阶要求。

Q76. Algorithms & Scripting: Date Comparison (LessThan) in C (FPGA Engineer Question)

QUESTION

Write a function LessThan(year1, month1, day1, year2, month2, day2) that returns true if the first date is strictly less than the second date, false otherwise. Each date is a valid date (month 1-12, day 1-31). Do not use any built-in Date library -- implement the comparison directly.

Example: LessThan(1900, 1, 1, 2000, 10, 10) returns true.

ANSWER

```
#include <stdbool.h>

bool LessThan(int year1, int month1, int day1, int year2, int month2, int day2) {
    if (year1 != year2) {
        return year1 < year2;
    }
    if (month1 != month2) {
        return month1 < month2;
    }
    return day1 < day2;
}

Verified (gcc, with assertions):
  LessThan(1900, 1, 1, 2000, 10, 10)  == true   (given example)
  LessThan(2000, 5, 20, 2000, 3, 10)  == false  (same year, later month)
  LessThan(2000, 5, 10, 2000, 5, 20)  == true   (same year+month, earlier day)
  LessThan(2000, 5, 10, 2000, 5, 10)  == false  (identical dates -- not strictly less)
  LessThan(1999, 12, 31, 2000, 1, 1)  == true   (year-boundary case)
```

```
All assertions passed.
```

解释

考点：日期比较是最基础的"字典序比较"应用题，考察能不能正确实现"先比最高优先级字段，相等才继续比下一级"这个通用的字典序比较逻辑，而不是想着用某种复杂的日期转天数的公式。

关键技巧：不需要把日期转换成儒略日或者从某个基准日期开始的总天数再比较大小（那样反而更容易在闰年、月份天数上出错），字典序比较法直接按year->month->day的优先级依次比较，只要某一级不相等就已经能确定大小关系，逻辑清晰且不涉及任何日历计算，也天然规避了闰年、大小月这些容易出错的边界情况。

延伸考点：面试官可能会追问如果需要处理"相等"这个第三种情况（比如要求实现Compare返回-1/0/1而不只是LessThan的true/false）架构要怎么改（把每一级的"不相等就返回"改成"不相等就返回具体的比较结果，相等才继续下一级，最后如果都相等返回0"，思路完全一样，只是返回值从bool变成了三态）。

Computer Architecture

Q77. Computer Architecture: FPGA Feasibility Analysis for LLM Layers (Bandwidth / DSP / Dataflow Scheduling)

QUESTION

You need to determine whether a given layer of a large language model (LLM) can be feasibly implemented on an FPGA.

(a) Why can't you just directly map an entire large model onto an FPGA? What categories of constraints do you need to consider?

(b) Assume latency is NOT a constraint (i.e. an "infinite latency" budget). When implementing one LLM layer on the FPGA, what do you need to consider from a BANDWIDTH perspective? Be specific about what "bandwidth" means in this context.

(c) What do you need to consider from a DSP (multiply-accumulate resource) perspective?

(d) Given a growing set of constraints (bandwidth, DSP count, memory size, etc.), how do you actually determine whether a candidate implementation is FEASIBLE?

(e) Given how much analysis this requires, why not just simulate the hardware directly to check feasibility, instead of doing this kind of analytical reasoning first?

ANSWER

```
// (a) Why not a direct 1:1 mapping, and what constraints matter:
//   A modern LLM has billions of parameters -- far more than any single
//   FPGA's on-chip storage (BRAM/URAM) can hold, and far more compute
//   than its finite DSP slice count can execute in parallel all at
//   once. "Directly mapping" the whole model would require resources
//   that simply don't physically exist on the device. The categories
//   of constraints that matter:
//     - Memory bandwidth: weights/activations must stream from
//       off-chip memory (HBM/DDR) to the FPGA fabric.
//     - Compute resources: DSP slice count (MAC throughput), LUT/FF
//       count for control and datapath logic.
//     - On-chip memory size: BRAM/URAM capacity for weight caching,
```

```
//       activation buffering, intermediate results.
//     - Achievable clock frequency and power/thermal budget.
//   Any real implementation plan has to fit ALL of these simultaneously,
//   not just one of them in isolation.
//
// (b) Bandwidth, with latency taken off the table:
//   Weights live OFF-CHIP (too large for on-chip storage); compute
//   happens ON-CHIP (in the DSP fabric). So the act of moving weight
//   data from off-chip memory to the FPGA compute engines is, by
//   definition, a BANDWIDTH-bound operation -- "bandwidth" here
//   specifically means the achievable off-chip-to-fabric data transfer
//   rate (e.g. the HBM/DDR interface's GB/s), NOT compute throughput.
//   Even with unlimited TIME (no latency requirement), you still need
//   to confirm the total weight bytes for that layer, divided by the
//   available memory interface bandwidth, gives a load time that's
//   actually achievable given the memory system -- and that on-chip
//   buffering is large enough to overlap loading with compute rather
//   than stalling on every tile.
//
// (c) DSP (compute) perspective:
//   DSP slices implement the MAC (multiply-accumulate) units that are
//   the fundamental building block of the matrix multiplications inside
//   an LLM layer (QKV projections, attention score compute, FFN
//   layers). You need to determine the MINIMUM number of DSPs required
//   to sustain the needed MAC-operations-per-cycle throughput for that
//   layer's total operation count. Even in a dataflow architecture
//   (where different stages of the computation pipeline run
//   concurrently on different resources), each stage still needs AT
//   LEAST enough DSPs to keep up with its share of the total MAC count
//   -- dataflow parallelism doesn't eliminate the minimum-throughput
//   requirement, it just distributes where that throughput has to be
//   delivered.
//
// (d) Determining feasibility with multiple constraints:
//   Model the implementation as a DATAFLOW GRAPH: each node is a
//   compute stage (e.g. Q-projection, K-projection, V-projection,
//   attention-score compute, softmax, output projection), each with a
//   start/completion time determined by its assigned resources
//   (bandwidth + DSP budget for that stage).
//   Feasibility check = verify DEPENDENCY ORDERING holds under the
//   computed timing: e.g. attention-score compute depends on BOTH Q and
//   K being ready -- if the schedule shows the node needing K starts
//   before K's producing node actually finishes, the schedule is
//   INFEASIBLE, regardless of whether bandwidth and DSP budgets look
//   fine individually.
//   Critically, constraints are ADDITIVE/CUMULATIVE, not independent:
//   satisfying bandwidth alone, or DSP count alone, does not guarantee
//   the combination is satisfiable. Every time a NEW constraint is
//   added (e.g. now also capping on-chip memory size), the WHOLE
//   schedule must be re-checked -- a configuration that was feasible
//   under the old constraint set can become infeasible once a new
//   constraint is layered on top, even if nothing else changed.
//
// (e) Why not just simulate the hardware directly:
//   A real FPGA implementation requires synthesis, place-and-route, and
//   bitstream generation -- each of these steps can take from tens of
//   minutes to many hours for a design of this complexity. Exploring
//   design-space questions ("what if we use this many DSPs, this much
//   on-chip buffering, this memory layout?") by physically implementing
//   EVERY candidate configuration is completely impractical -- you'd
//   spend hours of compile time to test a single hypothesis.
//   Instead, build a fast ANALYTICAL/SOFTWARE performance model (a
//   dataflow-timing simulator or even a closed-form roofline-style
```

```
//   calculation) that captures bandwidth, DSP throughput, and
//   dependency timing -- this lets you sweep through many candidate
//   configurations in seconds/minutes rather than hours, and only carry
//   the most promising few configurations forward to actual RTL
//   implementation and physical synthesis. This "fast analytical
//   exploration before expensive physical implementation" pattern is
//   the standard methodology in hardware/software co-design and
//   architecture space exploration generally, not specific to LLMs or
//   FPGAs.
```

解释

考点：这是一道System Technology Co-Optimization（STCO，软硬件协同优化）方向的综合题，把"大模型能不能上FPGA"这个看似简单的问题，拆解成带宽、计算资源、依赖关系可行性、以及方法论（为什么不直接仿真）这几个层次，考察的是系统级、架构级的分析思维，而不是某个具体电路的实现细节。

关键框架：回答这类"能不能实现"的问题，标准思路永远是先枚举约束类别（这里是带宽、DSP/计算资源、片上存储、时钟频率/功耗），再针对题目给的具体简化条件（这里是"假设latency不受限"）单独分析剩下的关键瓶颈（这里就剩带宽和DSP两个维度）。

带宽这一部分的核心洞察：题目故意去掉了latency约束，逼着你意识到"就算不着急，数据搬运的物理带宽上限依然是硬约束"——权重在片外、计算在片上，这个数据搬运动作本身就是带宽瓶颈，跟计算快慢无关，这是很多人第一反应会答错或者答不到点子上的地方。

可行性判断的核心：把实现方案建模成一个dataflow图，用"节点完成时间"和"依赖关系是否被违反"来判断可行性，并且要意识到约束是逐个叠加、必须联合满足的，不能因为单独看每个约束都满足就认为整体一定可行——这是约束满足问题（constraint satisfaction）的核心思想，在验证/约束随机化那几道题里也反复出现过同样的底层逻辑。

"为什么不直接仿真"这一问的价值：这是考察候选人有没有工程直觉——FPGA综合、布局布线、生成比特流的时间成本极高，不可能把每一个假设都真刀真枪做一遍硬件实现去验证，所以需要先用软件层面的分析模型/仿真器快速筛选，只有少数最有希望的方案才值得真正投入硬件实现，这是硬件设计空间探索（design space exploration）的标准方法论，不只适用于这一个场景。

延伸考点：面试官后续追问"为什么不直接仿真"，很可能是想确认这一点有没有真正理解，而不是单纯"读了字面意思"；如果继续深挖，可能会问"你会怎么设计这个分析模型/仿真器"，这时候可以提到roofline model（计算密度 vs 带宽的经典分析框架）作为进一步展开的方向。

Q78. Computer Architecture: Implement Signed Addition Overflow Detection Logic

QUESTION

Design overflow detection for signed two's complement addition.

Overflow occurs when adding two same-sign numbers produces an opposite-sign result. Must be independent of carry-out.

Example (WIDTH=8, range -128 to +127):

A=127, B=1 → sum=128(0x80=-128) → overflow=1

A=-128, B=-1 → sum=-129(0x7F=+127) → overflow=1

A=50, B=20 → sum=70 → overflow=0

Constraints:

- Parameterizable WIDTH
- Purely combinational logic on sign bits
- Independent of carry-out

ANSWER

```
module overflow_detect #(
  parameter W = 32
)(
  input  logic [W-1:0] a,
  input  logic [W-1:0] b,
  input  logic [W-1:0] result,
  input  logic         is_sub,
  output logic         overflow,
  output logic         carry
);
  logic [W:0] ext;

  assign ext      = is_sub ? ({1'b0, a} - {1'b0, b}) : ({1'b0, a} + {1'b0, b});
  assign carry    = ext[W];
  assign overflow = is_sub
    ? (a[W-1] != b[W-1]) && (result[W-1] != a[W-1])
    : (a[W-1] == b[W-1]) && (result[W-1] != a[W-1]);
endmodule
```

解释

考点：这是计算机体系结构里的经典模块设计题，重点考察对微架构概念的理解能否落地成可综合的RTL。

题目要求参数化（可配置位宽/深度），写代码时应该用parameter/localparam而不是硬编码常量，保证模块可复用。

延伸考点：面试官通常会在这道题基础上继续追问边界条件（比如参数取极值时是否还正确）或者要求扩展功能，建议实现时就考虑清楚这些点，回答时主动提及。

Q79. Computer Architecture: Implement Pseudo-LRU Using Binary Tree Structure

QUESTION

Design a pseudo-LRU (PLRU) replacement policy using a binary tree structure for an N-way set-associative cache (N is power of 2).

Use `N-1` bits per set for tree node states. On access, update only the tree nodes along the path from root to the accessed way. For victim selection, traverse root to leaf following node bits.

Example (4-way):

Tree has 3 nodes: 1 root, 2 children

Access way 0 → flip path bits away from way 0

Victim selection → follow bits root-to-leaf

Constraints:

- N must be a power of 2
- Tree uses N-1 bits per set
- Support parameterizable N

ANSWER

```
module cache_plru #(
  parameter int NUM_SETS = 64,
  parameter int N_WAYS   = 4
)(
  input  logic clk,
  input  logic rst_n,
  input  logic access_valid,
  input  logic [$clog2(NUM_SETS)-1:0] access_set,
  input  logic [$clog2(N_WAYS)-1:0] access_way,
  output logic [$clog2(N_WAYS)-1:0] victim_way
);

  localparam int LEVELS    = $clog2(N_WAYS);
  localparam int TREE_BITS = N_WAYS - 1;

  logic [TREE_BITS-1:0] tree [NUM_SETS];

  // ------------------------
  // VICTIM SELECTION
  // ------------------------
  always_comb begin
    int node;

    node = 0;
    victim_way = '0;

    for (int lvl = 0; lvl < LEVELS; lvl++) begin
      if (tree[access_set][node] == 1'b0) begin
        victim_way[LEVELS-1-lvl] = 1'b0;
        node = 2*node + 1;
      end
      else begin
        victim_way[LEVELS-1-lvl] = 1'b1;
        node = 2*node + 2;
      end
    end
  end

  // ------------------------
  // UPDATE LOGIC
  // ------------------------
  always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
      for (int s = 0; s < NUM_SETS; s++) begin
        tree[s] <= '0;
      end
    end
    else if (access_valid) begin
      logic [TREE_BITS-1:0] next_tree;
      int node;

      next_tree = tree[access_set];
      node = 0;

      for (int lvl = 0; lvl < LEVELS; lvl++) begin
        logic dir;

        dir = access_way[LEVELS-1-lvl];

        // Store opposite direction as LRU
        next_tree[node] = ~dir;
```

```
        if (dir == 1'b0)
          node = 2*node + 1;
        else
          node = 2*node + 2;
      end

      tree[access_set] <= next_tree;
    end
  end

endmodule
```

解释

考点：这是计算机体系结构里的经典模块设计题，重点考察对微架构概念的理解能否落地成可综合的RTL。

题目要求参数化（可配置位宽/深度），写代码时应该用parameter/localparam而不是硬编码常量，保证模块可复用。

延伸考点：面试官通常会在这道题基础上继续追问边界条件（比如参数取极值时是否还正确）或者要求扩展功能，建议实现时就考虑清楚这些点，回答时主动提及。

Q80. Computer Architecture: Implement Miss Status Holding Register (MSHR) for Non-Blocking Cache

QUESTION

Design MSHR structure to track outstanding cache misses and enable non-blocking operation.

Each entry stores miss address and requester info. Support merging multiple requests to the same cache line. Implement allocation, deallocation, and matching logic.

Example:

CPU0 misses on addr A → allocates MSHR[0]

CPU1 misses on addr A → merges into MSHR[0]

Refill completes → both CPUs served, MSHR[0] freed

Constraints:

- Configurable number of MSHR entries

- Backpressure when all entries occupied

- Match on cache line address (ignore block offset)

ANSWER

```
module mshr #(
  parameter ENTRIES  = 4,
  parameter ADDR_W   = 32,
  parameter REQS     = 4,
  parameter OFFSET_W = 4
)(
  input  logic                          clk,
  input  logic                          rst_n,
  input  logic                          alloc_req,
  input  logic                          refill_done,
```

```
  input  logic [ADDR_W-1:0]             alloc_addr,
  input  logic [$clog2(REQS)-1:0]       requester_id,
  input  logic [$clog2(ENTRIES)-1:0]    refill_entry,
  output logic                          full,
  output logic                          hit,
  output logic                          issue_mem_req,
  output logic [$clog2(ENTRIES)-1:0]    alloc_entry,
  output logic [$clog2(ENTRIES)-1:0]    hit_entry,
  output logic [REQS-1:0]               merged_waiters
);
  // Use packed valid bitvector for Verilator comb sensitivity
  logic [ENTRIES-1:0]              valid_vec;
  logic [ADDR_W-OFFSET_W-1:0]     line    [ENTRIES];
  logic [REQS-1:0]                waiters [ENTRIES];
  logic [ADDR_W-OFFSET_W-1:0]     alloc_line;

  assign alloc_line     = alloc_addr[ADDR_W-1:OFFSET_W];
  assign merged_waiters = waiters[refill_entry];

  always_comb begin
    hit         = 1'b0;
    hit_entry   = '0;
    full        = 1'b1;
    alloc_entry = '0;

    for (int i = 0; i < ENTRIES; i++) begin
      if (valid_vec[i] && (line[i] == alloc_line) && !hit) begin
        hit       = 1'b1;
        hit_entry = i[$clog2(ENTRIES)-1:0];
      end
    end

    for (int i = 0; i < ENTRIES; i++) begin
      if (!valid_vec[i] && full) begin
        full        = 1'b0;
        alloc_entry = i[$clog2(ENTRIES)-1:0];
      end
    end
  end

  assign issue_mem_req = alloc_req && !hit && !full;

  always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
      valid_vec <= '0;
      for (int i = 0; i < ENTRIES; i++) begin
        line[i]    <= '0;
        waiters[i] <= '0;
      end
    end else begin
      if (alloc_req) begin
        if (hit) begin
          waiters[hit_entry][requester_id] <= 1'b1;
        end else if (!full) begin
          valid_vec[alloc_entry]               <= 1'b1;
          line[alloc_entry]                    <= alloc_line;
          waiters[alloc_entry]                 <= '0;
          waiters[alloc_entry][requester_id]   <= 1'b1;
        end
      end

      if (refill_done) begin
        valid_vec[refill_entry]   <= 1'b0;
```

```
        waiters[refill_entry]     <= '0;
      end
    end
  end
endmodule
```

解释

考点：这是计算机体系结构里的经典模块设计题，重点考察对微架构概念的理解能否落地成可综合的RTL。

MSHR的核心价值是让多个对同一缓存行的未决请求合并成一次真实的内存访问，命中判断和分配判断要分开处理，不能混在一起。

延伸考点：面试官通常会在这道题基础上继续追问边界条件（比如参数取极值时是否还正确）或者要求扩展功能，建议实现时就考虑清楚这些点，回答时主动提及。

Q81. Computer Architecture: Design Functional Unit Busy/Free Tracking Module

QUESTION

Implement a tracker for functional unit availability in a multi-issue processor.

Constraints:

- Bit-vector tracking: 1=busy, 0=free

- Define same-cycle allocate+free precedence

- Reset: all units free

ANSWER

```
module fu_tracker #(
  parameter N_FU = 4
)(
  input  logic                     clk,
  input  logic                     rst_n,
  input  logic                     issue_req,
  input  logic                     done_req,
  input  logic [$clog2(N_FU)-1:0] issue_fu,
  input  logic [$clog2(N_FU)-1:0] done_fu,
  output logic [N_FU-1:0]         busy,
  output logic                     any_available
);
  assign any_available = ~(&busy);

  always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
      busy <= '0;
    end else begin
      if (issue_req) busy[issue_fu] <= 1;
      if (done_req)  busy[done_fu]  <= 0;
    end
  end
endmodule
```

解释

考点：这是计算机体系结构里的经典模块设计题，重点考察对微架构概念的理解能否落地成可综合的RTL。

实现时要特别注意题目给出的具体数值示例，写完代码后最好口头过一遍这些例子验证逻辑是否自洽。

延伸考点：面试官通常会在这道题基础上继续追问边界条件（比如参数取极值时是否还正确）或者要求扩展功能，建议实现时就考虑清楚这些点，回答时主动提及。

Q82. Computer Architecture: Implement Pipeline Registers with Stall and Flush Controls

QUESTION

Design pipeline stage registers supporting stall (hold current value) and flush (insert NOP/bubble).

Behavior:

Stall=1 → register holds current value

Flush=1 → register overwritten with NOP

Both=1 → defined priority (document choice)

Constraints:

- Define stall+flush simultaneous priority

- Reset initializes to safe NOP state

- Fields: instruction, PC, control signals, decoded registers, immediates

ANSWER

```
module pipe_reg #(
  parameter W = 64
)(
  input  logic          clk,
  input  logic          rst_n,
  input  logic          stall,
  input  logic          flush,
  input  logic [W-1:0] d,
  output logic [W-1:0] q
);
  always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n)      q <= '0;
    else if (flush)  q <= '0;
    else if (!stall) q <= d;
  end
endmodule
```

解释

考点：这是计算机体系结构里的经典模块设计题，重点考察对微架构概念的理解能否落地成可综合的RTL。

流水线相关设计要明确stall和flush同时出现时的优先级，这是题目描述里特别要求要"文档化"的地方，说明这是一个容易产生歧义、必须显式定义的设计决策。

延伸考点：面试官通常会在这道题基础上继续追问边界条件（比如参数取极值时是否还正确）或者要求扩展功能，建议实现时就考虑清楚这些点，回答时主动提及。

Q83. Computer Architecture: Implement 2-Read 1-Write Register File

QUESTION

Design a register file with two asynchronous read ports and one synchronous write port.

Constraints:

- Parameterizable NUM_REGS and DATA_WIDTH

- Define read-during-write semantics (write-first or read-first)

- Optional: hardwired zero register (x0)

ANSWER

```
module regfile #(
  parameter W     = 32,
  parameter DEPTH = 32
)(
  input  logic                      clk,
  input  logic                      we,
  input  logic [$clog2(DEPTH)-1:0] wa,
  input  logic [$clog2(DEPTH)-1:0] ra1,
  input  logic [$clog2(DEPTH)-1:0] ra2,
  input  logic [W-1:0]              wd,
  output logic [W-1:0]              rd1,
  output logic [W-1:0]              rd2
);
  logic [W-1:0] regs [DEPTH];

  // x0 always reads zero
  assign rd1 = (ra1 == 0) ? '0 : regs[ra1];
  assign rd2 = (ra2 == 0) ? '0 : regs[ra2];

  always_ff @(posedge clk) begin
    if (we && wa != 0) regs[wa] <= wd;
  end
endmodule
```

解释

考点：这是计算机体系结构里的经典模块设计题，重点考察对微架构概念的理解能否落地成可综合的RTL。

题目要求参数化（可配置位宽/深度），写代码时应该用parameter/localparam而不是硬编码常量，保证模块可复用。

延伸考点：面试官通常会在这道题基础上继续追问边界条件（比如参数取极值时是否还正确）或者要求扩展功能，建议实现时就考虑清楚这些点，回答时主动提及。

Q84. Computer Architecture: MESI and MOESI Cache Coherence Protocols

QUESTION

What is the MESI cache coherence protocol? What is MOESI, and what's the difference between the two?

ANSWER

```
// MESI states (per cache line, per core):
//   M (Modified)  -- sole copy, DIRTY (differs from memory); this
//                    cache must eventually write it back.
//   E (Exclusive) -- sole copy, CLEAN (matches memory); can silently
//                    go to M on a local write, no bus transaction
//                    needed.
```

```
//   S (Shared)    -- may also exist in other caches, clean; a write
//                     requires invalidating other copies first.
//   I (Invalid)   -- no valid copy.
//
// Core mechanism: a core reading a line for the first time (miss, no
// other cache has it) goes to E; if another core then reads the same
// line, both transition to S (shared read); if a core in S writes,
// it broadcasts an invalidate (other S copies -> I) and becomes M.
//
// MOESI adds a fifth state, O (Owned):
//   O (Owned) -- this cache holds the ONLY valid, DIRTY copy, but
//                UNLIKE M, memory has NOT been updated yet, and OTHER
//                caches may also hold a (now-stale, but officially
//                "shared read") copy of this same data marked S. The
//                O-holder is the designated "owner" responsible for
//                supplying the correct data (via cache-to-cache
//                transfer) to any future reader, instead of forcing a
//                writeback-then-refetch through memory.
//
// The key difference: in MESI, when a core holding a MODIFIED line is
// re-read by another core, the common implementation forces a
// writeback to memory (or at least tends toward keeping memory
// synchronized) before/while supplying the data. In MOESI, the dirty
// data can be forwarded CORE-TO-CORE directly from the O-holder,
// with memory allowed to stay stale for as long as the O-holder exists
// -- avoiding an extra memory round trip every time dirty data is
// shared. This matters most in workloads where multiple cores
// frequently re-read data that was just written by another core.
```

解释

考点：MESI/MOESI是多核缓存一致性的经典题，考察能不能讲清楚每个状态的具体含义，以及MOESI相比MESI到底解决了什么效率问题，而不是背出几个字母分别代表什么。

状态转移的核心逻辑：独占读之后没人共享就是E，多个核心都读过就变成S，从S发起写操作需要先广播失效通知再变成M——这个转移过程本身跟之前详细展开过的两核读写场景完全一致。

MOESI的O状态到底解决了什么问题：MESI下如果一个核心持有M态的脏数据，被别的核心读取，通常需要先同步内存（写回+重新读取两次内存事务）；MOESI允许脏数据由owner核心直接转发给需要的核心，内存可以长期落后于真实值，只要owner还在就行，这在多核心频繁共享读写同一份数据的场景下能明显减少内存带宽消耗和延迟。

延伸考点：面试官可能会追问E状态存在的意义（如果没有E，独占读之后本地写也要走一次总线广播升级到M，E状态省掉了这次总线事务），以及如果owner核心被替换出缓存（evict）时O态的脏数据该怎么处理（必须先写回内存，否则数据就丢失了，这是O态相比M态多出来的一项额外责任）。

Q85. Computer Architecture: Set-Associative Cache vs Direct-Mapped vs Fully-Associative

QUESTION

What is a set-associative cache? How does it compare to direct-mapped and fully-associative organizations? (See the companion question on implementing a write-back set-associative cache with LRU/FIFO replacement for a concrete RTL design based on this concept.)

ANSWER

```
// Cache placement policy spectrum:
```

```
//   DIRECT-MAPPED: each memory address maps to EXACTLY ONE possible
//     cache location (line index = address bits modulo number of
//     lines). Simplest, fastest to check (no comparison needed beyond
//     one tag match), but suffers badly from CONFLICT MISSES -- two
//     frequently-used addresses that happen to map to the SAME line
//     will keep evicting each other, even if the rest of the cache is
//     completely empty.
//   FULLY-ASSOCIATIVE: a memory address can be placed in ANY cache
//     line at all. Eliminates conflict misses entirely (only capacity
//     misses remain), but requires comparing the tag against EVERY
//     single line in the cache simultaneously -- expensive in
//     hardware (a large parallel comparator array) and doesn't scale
//     to large cache sizes.
//   SET-ASSOCIATIVE: the middle ground. The cache is divided into SETS
//     (index bits select which set), and each set contains N "ways"
//     (N cache lines) -- an address can go into any of the N ways
//     WITHIN its designated set, but not anywhere else in the cache.
//     This is direct-mapped (N=1 way) and fully-associative (N = total
//     number of lines, only 1 set) as the two extreme special cases of
//     the SAME general structure.
//
// The trade-off N (associativity) controls: higher N reduces conflict
// misses (more flexibility in where an address can land) at the cost
// of more comparators per set and a more complex replacement decision
// (need a policy like LRU/FIFO/PLRU to pick which of the N ways to
// evict) -- most real CPU caches use N somewhere between 2 and 16,
// balancing hit rate against hardware complexity and access latency.
```

解释

考点：这是缓存组织方式的基础概念题，考察能不能把直接映射、全相联、组相联这三种方式放在同一个框架下理解，而不是孤立地记忆三个名词。

关键理解：组相联本质是直接映射和全相联的一般化——直接映射就是"每组只有1路"的组相联，全相联就是"只有1组、路数等于全部缓存行数"的组相联，理解了这个统一视角，就不需要分别记忆三套规则。

权衡的核心：路数越多，一个地址能放的位置越灵活，冲突缺失（conflict miss）越少，但需要的比较器数量和替换策略的复杂度也随之上升——这也是为什么几乎没有实际CPU缓存采用纯直接映射或者纯全相联，而是选择路数适中的组相联作为折中方案。

延伸考点：这道题跟之前"实现一个带LRU/FIFO替换策略的写回组相联缓存"那道RTL设计题是配套的——概念题回答清楚"是什么、为什么"，设计题落实"具体怎么实现"，两道题放在一起能体现从理论理解到工程实现的完整链条。

Q86. Computer Architecture: TAGE vs gshare Branch Predictors

QUESTION

What is the difference between the TAGE and gshare branch predictors? How does each one work?

ANSWER

```
// gshare (simpler, widely used as a baseline predictor):
//   Maintains a single Global History Register (GHR) -- a shift
//   register recording the taken/not-taken outcome of the last N
//   branches, globally across the whole program (not per-branch).
//   To predict a branch: XOR the GHR with (part of) the branch's PC,
//   and use the result to INDEX into a single Pattern History Table
```

```
//   (PHT) of 2-bit saturating counters. The counter's value predicts
//   taken/not-taken, with 2-bit hysteresis so a single mispredict
//   doesn't immediately flip the prediction.
//   Why XOR with PC (rather than just using GHR alone as the index):
//   this helps DIFFERENT branches that happen to share the same
//   recent global history pattern still land in different PHT entries
//   most of the time, reducing (but not eliminating) destructive
//   aliasing between unrelated branches.
//   Limitation: ONE fixed history length for every branch -- some
//   branches correlate strongly with only a SHORT recent history,
//   others need a much LONGER history to predict well, and gshare
//   can't adapt per-branch.
//
// TAGE (TAgged GEometric history length -- significantly more
// sophisticated, used in most modern high-performance CPUs):
//   Maintains MULTIPLE prediction tables, each indexed using a
//   DIFFERENT history length, with the lengths increasing
//   GEOMETRICALLY across tables (e.g. table1: 4 bits of history,
//   table2: 8 bits, table3: 16 bits, table4: 32 bits, ...) plus a base
//   bimodal predictor table using no history at all.
//   Each entry in these longer-history tables is TAGGED with a partial
//   hash (of PC + history) -- this lets the predictor distinguish a
//   genuine match from an aliasing collision, which is what makes
//   using many different, LONG history lengths practical without
//   drowning in aliasing noise.
//   To predict a branch: check ALL tables in parallel; among the
//   tables whose TAG actually matches (a confirmed hit, not aliasing),
//   use the prediction from the table with the LONGEST matching
//   history -- longer, correctly-matched history generally gives the
//   most specific/accurate prediction for branches with complex
//   correlations. If no longer-history table has a tag hit, fall back
//   to progressively shorter tables, down to the base predictor.
//
// Bottom line difference: gshare uses ONE fixed history length for
// every branch; TAGE adaptively picks the BEST-matching history length
// PER BRANCH from a whole family of geometrically-scaled tables, at
// the cost of significantly more hardware (multiple tables, tag
// storage, and an allocation policy deciding when to promote a branch
// into a longer-history table). This adaptivity is why TAGE
// consistently achieves higher prediction accuracy than gshare,
// especially on branches with complex, long-range correlations, and
// is why it (or a TAGE-derived predictor) is the dominant branch
// predictor family in modern high-performance processors.
```

解释

考点：TAGE和gshare都是基于历史相关性的分支预测器，考察对"用历史信息预测分支"这一大类预测器演进思路的理解，尤其是TAGE相比gshare到底解决了什么核心问题。

gshare的核心限制：只用一个全局历史寄存器、一张预测表，所有分支共享同一个"历史长度"，但不同分支需要的历史长度天然是不一样的——有些分支只跟最近几次的结果相关，有些需要更长的历史才能预测准，gshare没办法针对不同分支自适应地调整历史长度。

TAGE的核心创新：同时维护多张用不同（几何级数增长的）历史长度索引的预测表，每张表的表项都带一个tag用来确认命中是不是真的匹配（而不是历史碰撞导致的伪命中），预测时优先采用"命中且历史最长"的那张表的结果——本质上是让每个分支都能自动找到最适合它自己的历史长度，而不是被迫使用一个全局统一的长度。

两者关系的记忆方法：可以把gshare理解成TAGE的一个简化特例（只有一张固定历史长度的表，没有tag确认机制），TAGE则是在这个基础上"多开几张不同历史长度的表+加tag防止伪命中+按最长命中历史优先"这几个改进的集大成者，这也是为什么TAGE预测准确率明显更高，但硬件复杂度也高得多。

延伸考点：面试官可能会追问TAGE的"分配策略"具体怎么做（当一个分支被预测错误时，需要决定要不要给它在某张更长历史的表里新分配一个表项，这里有一套专门避免"污染"已有有用表项的启发式规则），以及为什么现代高性能CPU几乎都采用TAGE或者其变体而不是gshare（预测准确率对流水线深度较深、乱序执行范围较大的现代CPU性能影响巨大，哪怕准确率提升几个百分点也有明显的整体性能收益）。

Behavioral Questions

Q87. Behavioral: Ensuring Something You Created Worked Properly

QUESTION

Tell me about a time you were responsible for making sure something you had created worked properly. What did you do to ensure it worked as intended, and what was the outcome?

Q88. Behavioral: Responding to a Major Setback After Significant Effort

QUESTION

Tell me about a time you experienced a major setback after putting significant effort into something. How did you respond, and what was the outcome?

Q89. Behavioral: Using an AI Tool to Help Create Something

QUESTION

Tell me about a time you used an AI tool to help you create something, such as code or a project, among others. How did you work out what it could and could not do well, and what did you create or change before relying on its output?

Q90. Behavioral: Something You Created Did Not Work as Expected

QUESTION

Tell me about a time something you created did not work as expected. How did you work out why, and looking back, what might you have missed?

Q91. Behavioral: Creating Something With Other People (Team Role)

QUESTION

Tell me about something you created with other people. What was your role, and how did you help the team work efficiently together?

Q92. Behavioral: Explaining Something You Created to an Unfamiliar Audience

QUESTION

Tell me about a time you had to explain something you had created, in writing or in person, to someone who was unfamiliar with it. How did you make it easy to understand, and how did you know it was understood?

==========

END OF DOCUMENT

==========

==========

END OF DOCUMENT

==========